



PES
UNIVERSITY

B. TECH IN COMPUTER SCIENCE AND ENGINEERING

PROGRAM EDUCATIONAL OBJECTIVES

- Prepare and train students in theoretical foundations to work with cutting edge computing technologies and design solutions to complex engineering problems, making them ready to work in industrial environment.
- Develop all round skills such as team building, inter-personal skills, and leadership qualities in order to effectively communicate with engineering community and with society at large.
- Promote research culture through internships, research assistantships, research-oriented projects, sponsored and collaborative research and enable them to pursue higher studies in computer science and related fields.
- To inculcate social concern meeting the requirements of prospective employers and to develop an ability to innovate efficient computing solutions for a better society.
- Create professionally superior and ethically strong globally competent employees and entrepreneurs.

PROGRAM OUTCOMES

- Apply mathematical and theoretical principles in the modeling and design of high-quality computer-based systems using state-of-the-art computer technology.
- Conduct in-depth study of research literature in the area of Computer Science, analyse problems in order to arrive at substantiated conclusions using first principles of mathematics, and allied sciences.
- Design, implement and evaluate Computer Systems, programs and processes that meet partial/ complete specifications with concern for society, environment and culture.
- Design and conduct experiments, collect data, analyze and interpret the results to investigate complex engineering problems in the field of Computer Science.
- Apply state-of-the-art techniques and modern computer-based tools in prediction, comparison and modelling of complex engineering activities.
- Have a sound understanding of professional, legal, security and social issues and responsibilities in engineering activities involving Computer Science.
- Understand societal and environmental concerns and demonstrate responsibility in sustainable development of computer-based solutions.
- Be aware of ethical and professional responsibilities in engineering situations; make informed judgments regarding intellectual property and rights in relation to computer-based solutions in global, economic, environmental and societal contexts.
- Able to function effectively in teams to establish goals, plan tasks, meet deadlines, manage risk and produce high-quality technical solutions.
- Contribute and communicate effectively with the society, be able to write effective reports and design documents by adhering to appropriate standards, make effective presentations, give and receive clear instructions.
- Apply skills in clear communication, responsible teamwork and time management by, for example, managing a team or project and communicating with external stakeholders.
- Recognize the need for and demonstrate an ability to engage in continuing professional development in its broadest sense.

B.TECH IN COMPUTER SCIENCE AND ENGINEERING**I SEMESTER (2026-30 BATCH)**

Sl. No	Course Code	Course Title	Hours per week				Credits	Tools/Languages	Course Type
			L	T	P	S	C		
1	UE26CS151A	Python for Computational Problem Solving	4	0	2	5	5	Python interpreter 3.8 and above. IDLE and any IDE like Jupyter	FC

II SEMESTER (2026-30 BATCH)

Sl. No	Course Code	Course Title	Hours per week				Credits	Tools/Languages	Course Type
			L	T	P	S	C		
1	UE26CS151B	Problem Solving with C	4	0	2	5	5	Programming Language GCC Compiler, GNU Debugger	FC

III SEMESTER(2025-29 BATCH)

Sl. No	Course Code	Course Title	Hours per week				Credits	Tools/Languages	Course Type
			L	T	P	S			
1	UE25CS251A	Digital Logic and Computer Design	4	0	2	5	5	Logisim Evolution, Icarus Verilog, GTKWave, ModelSim / QuestaSim, gem5, PipelineSim.	CC
2	UE25CS252A	Data Structures and its Applications	4	0	2	5	5	C-Programming language	CC
3	UE25MA242A	Mathematical Foundation for AI & data Science	4	0	0	4	4	Python, MatLab	CC
4	UE25CS242A	Operating System	4	0	0	4	4	C, Linux/Unix OS for system call implementation.	CC
5	UE25CS243A	Computer Networks	4	0	0	4	4	Cisco Packet Tracer, Wireshark, Mininet Simulator, Python.	CC
6	UZ25UZ221A	CIE L1	2	0	0	2	2		ST
7	UE26MA201A *	Bridge Course Mathematics-I (Applicable for The Lateral Entry Students)	2	0	0	0	0		FC
Total			22	0	4	24	24		

*-Audit Course

IV SEMESTER(2025-29 BATCH)

Sl. No.	Course Code	Course Title	Hours per week				Credits	Tools/Languages	Course Type
			L	T	P	S	C		
1	UE25CS251B	Microprocessor and Computer Architecture	4	0	2	5	5	RISC-V GNU Toolchain, QEMU (riscv64-linux-user), Spike ISA Simulator, Ripes, Dinero IV Cache Simulator, Venus (Web-based).	CC
2	UE25CS252B	Machine Learning	4	0	2	5	5	Pytorch	CC
3	UE25CS241B	Design and Analysis of Algorithms	4	0	0	4	4	C-Programming Language .	CC
4	UE25CS242B	Cyber Security Essentials	4	0	0	4	4	Wireshark, Kali, Linux, Python.	CC
5	UE25CS243B	Full Stack Development – Theory and Practice	4	0	0	4	4	HTML, CSS, JS, ReactJS, NodeJS, ExpressJS, MongoDB, MySQL	CC
6	UZ25UZ221B	CIE L2	2	0	0	2	2		ST
7	UE26MA201B *	Bridge Course Mathematics–II (Applicable to Lateral Entry Students)	2	0	0	0	0		FC
Total			22	0	4	24	24		

V SEMESTER(2024-28 BATCH)

Sl. No.	Course Code	Course Title	Hours per week				Credits	Tools/Languages	Course Type
			L	T	P	S			
1	UE24CS351A	Database Management System	4	0	2	5	5	MySQL Workbench, Python, Redis, Neo4j	CC
2	UE24CS352A	Machine Learning	4	0	2	5	5	Pytorch	CC
3	UE24CS341A	Software Engineering	4	0	0	4	4	Git, GitHub, docker, Jenkins, gtest, pytest, Jira/OpenProject , SonarQube.	CC
4	UE24CS342AAX	Elective I	4	0	0	4	4		EC
5	UE24CS343ABX	Elective II	4	0	0	4	4		EC
6	UE24CS320A	Capstone Project-Phase-I	0	0	8	2	2		PW
Total			20	0	12	22	24		
Elective-I									
7	UE24CS342AA1	Advanced Algorithms	4	0	0	4	4	C or C++ Programming Language	EC
8	UE24CS342AA2	Data Analytics	4	0	0	4	4	R and Python, MS Excel tools	EC
9	UE24CS342AA3	Internet of Things	4	0	0	4	4	-	EC
10	UE24CS342AA4	Applied Cryptography	4	0	0	4	4	-	EC
11	UE24CS342AA5	Augmented and Virtual Reality	4	0	0	4	4	Co-pilot	EC
12	UE24CS342AA6	UI/UX design	4	0	0	4	4	Figma	EC
13	UE24CS342AA7	Topics in wireless and 5G Networks	4	0	0	4	4	Cisco Packet Tracer, OMNET++	EC
14	UE24CS342AA8	Quantum Computing	4	0	0	4	4	PennyLane,Qiskit Machine Learning	EC

15	UE24CS342AA9	Visualization of Data, Understanding and Interpretation in the Era of AI	4	0	0	4	4	Matplotlib, Seaborn, Plotly, Tableau / Power BI	EC
Elective–II									
16	UE24CS343AB1	Image Processing	4	0	0	4	4	MatLab, Python	EC
17	UE24CS343AB2	Big data	4	0	0	4	4	Hadoop, HDFS, Spark, Streaming Spark, HIVE, HBase, Mllib.	EC
18	UE24CS343AB3	Graph Theory and its Applications	4	0	0	4	4	NetworkX, PyTorch, Deep Graph,Library,Ne o4j,Gephi, Graphistry	EC
19	UE24CS343AB4	Modern Android App Development with Kotlin	4	0	0	4	4	Kotlin, Android Studio, Gradle	EC
20	UE24CS343AB5	Intelligent Vehicle :Connectivity & Cyber Security	4	0	0	4	4	Kali Linux, Wireshark, VehicleSpy, Python (for attack simulation and secure coding), SCAP tools, IDS/IPS simulation frameworks, AUTOSAR security stack, Sumo, Veins, Omnet++, CARLA.	EC
21	UE24CS343AB6	Computer Network security	4	0	0	4	4	SEED Ubuntu VM, Wireshark, Snort, Netwox, Scapy.	EC
22	UE24CS343AB7	ROS for Autonomous Systems.	4	0	0	4	4	ROS/ROS2, Python, C++	EC
23	UE24CS343AB8	Enterprise Business Systems	4	0	0	4	4	Oracle, Sales force, Open ERP	EC
24	UE24CS343AB9	Design of Secure Platform and Systems	4	0	0	4	4	QEMU,Linux, Python, C	EC

VI - SEMESTER(2023-27BATCH)

Sl. No	Course Code	Course Title	Hours per week				Credits	Tools/Languages	Course Type
			L	T	P	S			
1	UE24CS351B	Cloud Computing	4	0	2	5	5	Amazon AWS (or equivalent), AWS SkillBuilder, AWS Educate, Qwiklabs, Docker, Kubernetes, Jenkins, Zookeeper, Github, NoSQL database, Flask, Python, Go Lang. AI tools- AWS Sagemaker	CC
2	UE24CS352B	Object Oriented Analysis and Design	4	0	2	5	5	Star UML, Object Oriented Programming Language(Java)	CC
3	UE24CS341B	Compiler Design	4	0	0	4	4	LLVM/Clang Lex/flex and YACC/Bison	CC
4	UE24CS342BA X	Elective III	4	0	0	4	4		EC
5	UE24CS343BBX	Elective IV	4	0	0	4	4		EC
6	UE24CS320B	Capstone Project Phase – II	0	0	8	2	2		EC
Total			20	0	12	24	24		
Elective–III									
7	UE24CS342BA1	Supply Chain Management for Engineers	4	0	0	4	4	SCM applications and tools, ML tools, Case studies, Webresources.	EC
8	UE24CS342BA2	Algorithms for Information Retrieval	4	0	0	4	4	Pytorch, C, C++, Java	EC
9	UE24CS342BA3	Design of RTOS	4	0	0	4	4	ARM GCC, Open OCD, GDB, C assembly.	EC
10	UE24CS342BA4	Generative AI and its Applications	4	0	0	4	4	Python, Hugging Face, LM Studio, Kaggle, Google Colab, AI tools- DeepEval, TruLens	EC
11	UE24CS342BA5	Block Chain	4	0	0	4	4	Solidity, Ethereum, Hyperledger Fabric,	EC

12	UE24CS342BA6	Digital Forensics and Incident responses	4	0	0	4	4	Open source tools on Forensics.Cellebrite, Deepfake Detection Tools	EC
13	UE24CS342BA7	Digital Twins	4	0	0	4	4	Co-pilot	EC
14	UE24CS342BA8	Responsible AI	4	0	0	4	4		EC
15	UE24CS342BA9	Quantitative Trading Systems	4	0	0	4	4		EC
16	UE24CS342BA10	Quantum ML & AI	4	0	0	4	4	PennyLane and Qiskit	EC
Elective–IV									
17	UE24CS343BB1	Heterogeneous Parallelism	4	0	0	4	4	Pthread, OpenMP, CUDA, AI tools-Vivado HLS.	EC
18	UE24CS343BB2	Topics in Deep Learning	4	0	0	4	4	Pytorch, Tensorflow, Kaggle, Google Colab, Hugging Face models	EC
19	UE24CS343BB3	Database Technologies	4	0	0	4	4	MySQL, postgres, Oracle, Apache Spark, Apache Kafka, Amazon Kinesis.	EC
20	UE24CS343BB4	Machine Learning on Graphs	4	0	0	4	4	Pytorch, C, C++, Java	EC
21	UE24CS343BB5	Embedded Systems with Micropython	4	0	0	4	4	Microcontroller: Pyboard, ESP32, BBC micro:bit	EC
22	UE24CS343BB6	Information Security	4	0	0	4	4	SEED Labs VM, Scapy, Burp Suite, Metasploit, Nmap, etc.	EC
23	UE24CS343BB7	Mobile Autonomous Robot	4	0	0	4	4	ROS/ROS2, Python, C++	EC
24	UE24CS343BB8	Cyber security in Cloud	4	0	0	4	4	AWS-vulnerable platform	EC
25	UE24CS343BB9	Quantum Computer System Design	4	0	0	4	4	IBM Qiskit	EC
26	UE24CS343BB10	Video Processing & Understanding	4	0	0	4	4	Python with libraries like OpenCV, PyTorch, TensorFlow FFmpeg, OpenCV, Hugging Face Transformers (DeepSort with YOLO integration)	EC
27	UE24CS343BB11								
28	UE24CS343BB12	SOC & Threat Intelligence	4	0	0	4	4	Wazuh, Shuffle/ Python	EC

VII SEMESTER (2023-27 BATCH)

SI No.	Course Code	Course Title	Hours/week				Credits	Course Type
			L	T	P	S	C	
1	UE23CS441A	Capstone Project Phase-III	0	0	16	4	6	Project Work
2	UZ23UZ422A	Technical writing	2				2	Special Topic
Total					20	6	8	

VIII SEMESTER (2023-27 BATCH)

Sl. No.	Course Code	Course Title	Hours/week				Credits	Course Type
			L	T	P	S	C	
1	UE23CS421B	Capstone Project Phase-IV	0	0	8	4	4	Project Work
2	UE23CS461XB	Internship	0	0	16	8	8	Internship
Total			0	0	22	12	12	

Course Code	UE26CS151A	Course Title	Python For Computational Problem Solving				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	2	5
Semester	1	Type of Course	Foundation				
Tools/Languages	Python interpreter 3.8 and above. IDLE and any IDE like Jupyter	Desirable Knowledge	-				
Prelude	Python is an easy to learn, general-purpose, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.						
Course Objectives:	- Understand the syntax and semantics of Python programming language. - Apply the process of structuring the data using lists, tuples, sets and dictionaries. - Utilize the use of built-in functions to navigate the file system - Explore various paradigms of programming and implement the Object-Oriented Programming concepts in Python.						
Course Contents	Unit 1: Introduction to python and collections Block diagram of Computer, Computer Hardware & Software: Application Software, System Software, Operating System, Syntax, semantics and program translation, algorithm. Introduction to Python Programming Language, IDLE Python Development Environment, Output function, variables, types and id, input function, operators and expressions, Control structures , Strings , Lists. 14 Hours Unit 2: Collections & Functions Tuples, Sets, Dictionaries, File handling - Reading and writing files: Text, CSV and JSON . Functions: Definition, call, Positional and keyword parameter, Default parameters, Variable number of arguments. 14 Hours Unit 3: Functions, Functional Programming and Modules Recursion, Call-backs, Lambda function, Map, filter, and reduce, max, min, Zip, list comprehension ,Closure, Decorators, generators, Introduction to modules – Numpy, matplotlib , Pillow / PIL ,argparse ,re . 14 hours Unit 4: Testing ,Debugging and Object Oriented Programming Testing- Pytest , Function testing with Doctest, pdb debugger commands, Virtual environments and pip package management. Classes and objects - inheritance, polymorphism, iterators & generators, Error handling & Exceptions - try, except and raise, exception propagation. 14 Hours						
Laboratory	1	Python Development Environment using IDLE					
	2	Python Fundamentals: Input, Output and Expressions					
	3	Control Flow Programming in Python					
	4	Working with Python Collections					
	5	Processing Different Types of Data using Text, CSV and JSON Files.					
	6	Desktop GUI Development using WxPython					

	7 Modular Programming using Functions and Recursion 8 Functional Programming Constructs in Python 9 Object-Oriented Programming using Python
Text Book(s):	1. "Think Python: How to Think Like a Computer Scientist", Allen B. Downey, 3rd Edition, O'Reilly Media, 2024. (Available under CC BY-NC-SA license at https://alldowney.github.io/ThinkPython/) 2. "Automate the Boring Stuff with Python: Practical Programming for Total Beginners", Al Sweigart, 3rd Edition, No Starch Press, 2025. (Available under CC BY-NC-SA license at https://automatetheboringstuff.com/3e/) 3 "Python Programming: Using Problem-Solving Approach", Reema Thareja, Third Edition, Oxford University Press, 2025.
Reference Book(s):	1. "Introduction to Computer Science Using Python: A Computational Problem- Focus", Charles Dierbach, Wiley India Edition, John Wiley, 2015. 2. "Fluent Python", Luciano Ramalho, 2nd Edition, O'Reilly Media, 2022
Course Outcome	1. Demonstrate the understanding of Python syntax, control structures, strings, and collection data types to develop basic Python programs. 2. Apply functions, file handling techniques, and Python data structures to solve computational and data processing problems. 3. Utilize functional programming concepts, modules, and Python libraries such as NumPy, Matplotlib, Pillow, argparse, and regular expressions in application development. 4. Implement Object-Oriented Programming, testing, debugging, exception handling, and virtual environment management to build reliable and modular Python applications.

Course Code	UE26CS151B	Course Title	Problem Solving with C				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	2	Type of Course	Foundation Course- Lab Integrated				
Tools/Languages	Programming Language GCC Compiler, GNU Debugger	Desirable Knowledge					
Prelude	The Problem Solving with C course introduces students to fundamental techniques for solving Common computational problems. Students will learn to analyse problem statements, design appropriate algorithms, and implement efficient solutions. Emphasis is placed on using C language constructs to develop well-structured and maintainable programs, thereby laying a strong foundation in both problem-solving and programming skills.						
Course Objectives:	• Develop the ability to solve relevant and logical problems using a computing machine through algorithmic thinking. • Translate algorithmic solutions into programs by effectively utilizing C programming language constructs. • Acquire a solid understanding of C language syntax, semantics, and its supporting ecosystem. • Recognize and appreciate the nuances of C standards and understand their implications on program behaviour.						
Course Contents	Unit 1: Problem Solving Fundamentals Introduction, Salient Features of 'C', Program Structure, Variables, Data Types & arrange of values ,Qualifiers, Operators and Expressions, Control Structures, Input/Output Functions, Language Specifications -Behaviors, Single character input and output, Guidelines and CERT C Coding standards of variables, comments and readability of code 14 Hours Unit 2: Arrays and Functions Arrays–1D and 2D, Bound Checking with Arrays, Pointer and Pointer Arithmetic, Pointer to an array and Array of pointers, Functions, Recursion, Call back, Storage classes, Recursion, Searching, Sorting, Strings, String Manipulation Functions & Error handling, Command line arguments 14 Hours Unit 3: Dynamic Memory Management and User-Defined Types Dynamic Memory Management functions & Error handling, Structures, #pragma, Array of Structures, Pointer to structures, Passing Structure and Array of structure to a function, Bit fields, Unions, Enums, Lists, Stack, Queue, Priority Queue 14 Hours Unit 4: File Handling and Portable Programming File IO using redirection, File Handling functions of C, Searching and Sorting using Opaque Pointers, Variable Length Arguments, Environment variables, Preprocessor Directives, Header Guards and Conditional Compilation. 14 Hours						
Laboratory	Lab 1: Foundational Logic: I/O, Operators, and Control Structures Lab 2: Linear Data Handling: Arrays, Pointers, and Functional Abstraction Lab 3: String Processing and Dynamic Memory Management Lab 4: Composite Data Types: Structures and Arrays of Structures Lab 5: System Integration: File Redirection and Command Line Interface						

	Lab 6: Advanced File I/O: Searching and Sorting via Pointers to Structures Lab 7: Data Structures: Implementation of the Ordered List Lab 8: Memory Optimization and Preprocessor Directives
Text Book(s):	1. "The C Programming Language", Brian Kernighan and Dennis Ritchie, Prentice Hall PTR, 2nd Edition, 1988.
Reference Book(s):	1. "How To Solve It By Computer", R G Dromey, Pearson, 2011. 2. "C Programming: A Modern Approach", 2nd Edition by K.N. King. W. W. Norton & Company. 3. "C how to Program with case Studies Introducing Applications Programming and Systems Programming". Ninth Edition(Global Edition) by Paul Dietel and Harvey Deitel, 2022 4. "Modern C for Absolute Beginners", by Slobodan Dmitrović, 2nd Edition, June 2024 5. "Head First C" by David Griffiths and Dawn Griffiths, April 2012 (O'Reilly Media) 6. "Learn C the Hard Way": Zed Shaw's Hard Way Series, 1st Edition. 7. "C Puzzle Book" by Alan R. Fever, Pearson Education. 8. "Expert C Programming: Deep C Secrets" by Peter Van Der Linden, Pearson Education.
Course Outcome:	• Understand and apply algorithmic solutions to counting and other basic computational Problems using appropriate C construct. • Understand, analyse, and implement fundamental sorting and searching techniques. • Apply string manipulation and text processing methods using arrays, pointers, and functions. • Design and implement user-defined data types using structures, unions, and other mechanisms in C, and demonstrate the ability to read from and write to secondary storage in a portable manner.

Course Code	UE24CS251A	Course Title	Digital Logic and Computer Design				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	3	Type of Course	Core				
Tools/Languages	Logisim Evolution, Icarus Verilog, GTKWave, ModelSim / QuestaSim, gem5, PipelineSim.	Desirable Knowledge	Basic Electronics				
Prelude	This course bridges digital logic and computer architecture, guiding students from Boolean foundations through processor design. Covering combinational and sequential circuits, hardware description, arithmetic units, MIPS assembly, and pipelined microarchitecture, it builds the analytical skills essential for modern hardware and embedded system						
Course Objectives:	- To introduce the principles of Boolean algebra, logic gates, and combinational/sequential circuit design. - To enable students to design and analyse finite state machines and synchronous digital systems. - To provide a thorough understanding of digital building blocks including arithmetic circuits, memory arrays, and sequential elements. - To familiarise students with MIPS assembly language, machine language encoding, addressing modes, and program execution flow. - To develop competency in single-cycle, multicycle, and pipelined processor design with performance evaluation techniques.						
Course Contents	Unit 1: Combinational and Sequential Logic Design Introduction, Boolean Equations, Karnaugh maps, Combinational building Blocks. Introduction, Latches and Flip-Flops, Synchronous Logic Design, Finite state Machines. 14 Hours Unit 2: Digital Building Blocks Introduction, Arithmetic Circuits, Number Systems, Sequential Building Blocks, Memory Arrays 14 Hours Unit 3: Architecture Introduction, Assembly Language, Machine Language, Programming, Addressing Modes, Compiling, Assembling and Loading, Parallelism. 14 Hours Unit 4: Microarchitecture Introduction, Performance Analysis, Single-Cycle Processor, Multicycle Processor, Pipelined Processor. 14 Hours						
Laboratory	- Verilog Basics - Verilog Basics - Combinational Design: A complete ALU - Combinational Design: A complete ALU - Sequential Design - Sequential Design						

	7. Input/Output Devices 8. Finite-State Machines 9. Memories: A Register File and a Basic Datapath 10. A Full Single-Cycle CPU 11-13: Projects
Text Book(s):	1. "Digital Design and Computer Architecture" by David Money Harris & Sarah L. Harris, Second Edition, 2013, Morgan Kaufmann.
Reference Book(s):	1. M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, Hoboken, NJ, 2018. 2. John F. Wakerly, Digital Design: Principles and Practices, 5th ed., Pearson, Upper Saddle River, NJ, 2017. 3. David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed., Morgan Kaufmann, Waltham, MA, 2013.
Course Outcome	• Design and minimise combinational logic circuits using Boolean equations, Karnaugh maps, and standard building blocks. • Construct and analyse sequential circuits, latches, flip-flops, and finite state machines for synchronous digital systems. • Implement digital building blocks such as adders, multipliers, counters, shift registers, and memory arrays for real-world applications. • Write, assemble, and trace MIPS assembly and machine language programs using appropriate addressing modes and parallelism concepts. • Design and compare single-cycle, multicycle, and pipelined MIPS processor microarchitectures and evaluate their performance metrics.

Course Code	UE24CS252A	Course Title	Data structures & its Applications				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	3	Type of Course	Core				
Tools/Languages	C-Programming language	Desirable Knowledge	Problem Solving with C				
Prelude	This course provides a comprehensive introduction to fundamental data structure concepts, emphasizing both their theoretical foundations and practical applications. It begins with abstract representations of data and explores how these abstractions facilitate effective problem solving. The course then bridges the gap between theory and practice by demonstrating how these concepts can be concretely implemented using a programming language. Equal importance is given to understanding the underlying principles, their implementation techniques, and their real-world applications, enabling students to not only grasp the concepts but also apply them effectively in software development.						
Course Objectives:	- To introduce the fundamental concepts and methodologies for analyzing and designing data structures, fostering problem-solving skills for efficient data storage, retrieval, and manipulation. - To explore and utilize various data structures such as arrays, linked lists, stacks, queues, trees, heaps, and graphs, emphasizing their suitability for different computational tasks and operations. - To implement core operations—insertion, deletion, searching, and modification—across a variety of data structures in both linear and non-linear forms. - To design and develop applications using appropriate data structures, demonstrating the ability to select and apply the most suitable structure to solve real-world problems effectively.						
Course Contents	Unit 1: Linked List and Stacks Review of Structures, Pointers, Static and Dynamic Memory Allocation in C. Linked List : Single, Double, Circular (All insert, delete, other operations and applications). Multilist: Introduction to sparse matrix. Skip list. Case study: Representation of Polynomial expression & its operations. Stack: Introduction & implementation of a Stack using Arrays & Linked list. Applications of Stack: Execution of functions, Recursion, Conversion & Evaluation of an expression: Infix to postfix, Infix to prefix, Matching of Parenthesis. 16 Hours Unit 2: Queues and Trees Queues: Simple Queue, Circular Queue, Priority Queue, Double Ended Queue and their implementation using Arrays and Linked List. Applications of Queue: Case Study – Josephus problem, CPU scheduling. Trees: N-ary trees, Binary Trees, Binary Search Trees (BST) and Forest: definition, properties, conversion of an N-ary tree and a Forest to a binary tree. Traversal of trees: Preorder, Inorder and Postorder. 12 Hours Unit 3: Application of Trees and Introduction to Graphs Implementation of Binary Search Tree using arrays and dynamic allocation(Insert, delete, traversals and other operations and applications), Expression Tree, Threaded Binary Search Tree, Heap using arrays, Priority Queue using heap (min and max heap) . Balanced Trees: AVL Trees, Splay Tree. Graphs: Properties, Representation & Implementation of graphs - Adjacency matrix, Adjacency list. Graph traversal methods: Depth first search, Breadth first search techniques.						

	14 Hours Unit 4: Applications of Graphs, Suffix Tree and Hashing Applications of BFS & DFS: Connectivity of graph, finding the number of Connected Components, finding path in a network. Trie Trees: Implementation (insert, delete and search operations), Construction of suffix trie and tree. Hashing: Hash function, Hash table, Collision Handling: Implementation (Insert, Delete and other operations) of Separate Chaining, Linear, Quadratic Probing, Double Hashing and Rehashing. 14 Hours
Laboratory	1: Pointers, Structures, Dynamic Memory Allocation. 2: Linked List and advanced operations. 3: Stack and applications based on it. 4: Queue and applications based on it. 5: Binary Tree, Binary Search Tree and applications based on it. 6: Graph Data structure and applications based on it. 7: Hashing Techniques.
Text Book(s):	"Data Structures using C / C++", Langsum Yedidyah, Moshe J Augenstein, Aaron M Tenenbaum Pearson Education Inc, 2nd edition, 2015.
Reference Book(s):	"Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019
Course Outcome	• Select and apply appropriate data structures for solving problems in a variety of application domains. • Implement fundamental data structures and their operations using suitable programming constructs and techniques. • Utilize data structures effectively in competitive programming to design optimal and efficient solutions. • Design and develop efficient software systems by leveraging an in-depth understanding of data structure principles and their applications.

Course Code	UE24MA242A	Course Title	Mathematical Foundation for AI & data Science				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	3	Type of Course	Core				
Tools/Languages	Python, MATLAB	Desirable Knowledge					
Prelude	This course introduces the mathematical and statistical foundations required for data science, artificial intelligence, and computational engineering applications. The course covers linear systems, vector spaces, matrix decompositions, eigen analysis, and dimensionality reduction techniques such as Singular Value Decomposition (SVD) and Principal Component Analysis (PCA). It also introduces probability distributions, statistical estimation, hypothesis testing, and inference methods used in data-driven decision making. Emphasis is placed on developing analytical, computational, and modeling skills for solving real-world engineering and data science problems.						
Course Objectives:	- Understand matrix methods and vector space concepts used in AI and data representation. - Apply eigen-analysis and dimensionality reduction techniques for data processing. - Model uncertainty using probability distributions and statistical inference. - To enable students to apply hypothesis testing and statistical reasoning methods for decision-making and predictive modeling applications.						
Course Contents	Unit 1: Linear Systems and Structural Foundations Introduction to Linear Systems: Systems of Linear Equations, Gaussian Elimination, LU Decomposition, Matrix Rank, Consistency of Systems. Inverse of a Matrix-Gauss Jordon Vector Spaces: Vector Spaces and Subspaces, Four fundamental subspaces, Linear Independence, Basis, Dimension., Linear Transformation 14 Hours Unit 2: Matrix Decomposition Vector Operation and Matrix decomposition: Dot Product, Vector Norms, Orthogonality, Gram Schmidt process, Orthogonal Projections, Least Squares Approximation. Eigen Concepts: Eigenvalues, Eigenvectors, Diagonalization of Matrices. Covariance and Positive Definite Matrices: Covariance Matrix, Positive Definite Matrices, Tests for Definiteness. Singular Value Decomposition: Singular Value Decomposition (SVD) Principal Component Analysis: PCA, Feature Extraction, Dimensionality Reduction. 14 Hours Unit 3: Probability and Statistical Modeling (14 Hours) Introduction Population, sample, sampling (Random Sampling), Random variable, Probability Distribution: Binomial and Normal, The Central Limit Theorem and Applications. Point Estimation, MLE: Principles of Point Estimation – Mean Squared Error for Normal, Maximum Likelihood Estimate for Normal. Introduction to Multivariate Normal Distribution, MAP distribution. Confidence Intervals: Interval Estimates for Mean of Large and Small Samples, Student's t Distribution, Factors affecting Margin of Error. 14 Hours						

	Unit 4: Hypothesis Testing and Simple Linear Regression Hypothesis Testing: Introduction, Large-Sample tests for a Population Mean, Drawing conclusions from the results of Hypothesis tests, the relationship between confidence interval and hypothesis tests, Small-Sample Tests for a Population Mean, ANOVA, Chi-squared Test, Fixed Level Testing, Type I and Type II Errors, Power of a Test, Factors Affecting Power of a Test. 14 Hours
Text Book(s):	1. "Linear Algebra and its Applications", Gilbert Strang, Thomson Brooks/ Cole, 4th Edition, Second Indian Reprint 2007. 2. "Statistics for Engineers and Scientists", William Navidi, McGraw Hill Education, India, 6th Edition, 2024. 3. "Linear Algebra and its Applications", David. C lay, Publication by Pearson Education, 5th Edition, Reprint : 2021 4. Application : Applied projects for an introductory linear algebra class by Anna Zemlyanova.
Reference Book(s):	1. "Higher Engineering Mathematics" , B S Grewal, Khanna Publishers, 44th Edition, 2020, 2. "Linear Algebra, Schaum's outlines", Seymour Lipschutz and Marc Lipson, Tata McGraw- Hill publications, 4th Edition, 2009. 3. Montgomery, Douglas C., and George C. Runger. <i>Applied Statistics and Probability for Engineers</i>. 7th ed., Wiley, 2018. 4. "Data Science from Scratch", Joel Grus, O'Reilly, 1st Edition, 2015. 5. "Fundamentals of Data Science", Sanjeev J. Wagh, Manisha S. Bhende, and Anuradha D. Thakare, First edition, CRC Press, 2022.
Course Outcome	• Solve systems of linear equations and apply vector space concepts in data representation. • Apply eigenvalue methods, SVD, and PCA for dimensionality reduction and feature extraction. • Use probability distributions and estimation techniques for probabilistic modeling. • Apply hypothesis testing and interpret statistical inference results for large and small sample data

Course Code	UE25CS242B	Course Title	Operating Systems				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
Semester	4	Type of Course	Core Course				
Tools/Languages	C, Linux/Unix OS for system call implementation	Desirable Knowledge	Problem Solving with C				
Prelude	This course focuses on fundamental operating systems concepts including various algorithms and trade-offs for efficient management of resources such as CPU, Memory, Storage and I/ O. This course requires the student to have a desirable knowledge of Data Structures and its Applications.						
Course Objectives:	● Focus on fundamental Operating System concepts. ● Provide an understanding of various components of the Operating System (OS). ● Delve deeper into various algorithms and associated trade-offs for efficient resource management such as process, disk, and memory management. ● Introduce design principles and trade-offs in the design of Operating Systems.						
Course Contents	Unit 1 : Introduction and Process Management What Operating Systems Do, Computer-System Organization, Computer-System Architecture, Operating-System Structure & Operations, Operating-System Services; Process Concept: Process in Memory, Process State, Process Control Block (PCB), Process Creation and Termination, System Calls for Process Management, Life Cycle of I/O Requests. CPU Scheduling: Basic Concepts, Scheduling Criteria, Scheduling Algorithms: First-Come First-Served (FCFS), Shortest-Job-First (SJF), Priority Scheduling, Round-Robin Scheduling, Multilevel Queue Scheduling, Multilevel Feedback Queue Scheduling; Introduction to Shell Programming: Variables, Control Flow, Shell Scripts, Scheduler – cron. 14 Hours Unit 2 : IPC, Threads and Concurrency IPC: Shared Memory, Message Passing, Pipes – Named and Ordinary, System Calls for Shared Memory, Pipes and FIFOs, Linux Domain Sockets; Introduction to Threads, Types of Threads, Multicore Programming, Multithreading Models, Thread Control Block (TCB), Thread Creation, Thread Scheduling, Thread Libraries, POSIX Threads (Pthreads); Multicore Kernel Concepts: Symmetric Multiprocessing (SMP), Kernel Parallelism, CPU Affinity, Load Balancing, Multicore Scheduling; Mutual Exclusion and Synchronization: Software Approaches, Principles of Concurrency, Hardware Support, Mutex Locks, Semaphores; Scalable Synchronization: Spinlocks, Reader-Writer Locks, Lock Contention; Classic Synchronization Problems; Deadlocks: Principles of Deadlock, Deadlock Characterization, Deadlock Avoidance; Signals and Signal Handling. 14 Hours Unit 3: Memory Management Main Memory: Hardware and Control Structures, OS Support, Address Translation, Dynamic Loading, Dynamic Linking and Shared Libraries, Memory-Mapped Files and mmap(), Swapping, Fragmentation, Segmentation, Paging, Structure of Page Tables, Translation Lookaside Buffer (TLB), TLB Hits and Misses, Effective Access Time (EAT); Virtual Memory – Demand Paging, Page-Fault Handling, Copy-on-Write, Page Replacement, Allocation of Frames, Thrashing, NUMA (Non-Uniform Memory Access) Basics and Memory Locality, Clock Page Replacement. 14 Hours						

	Unit 4 : File Management, Storage Management and Security File Concept, Access Methods, Directory, Driver Development, Flash Drives, File-System sharing, File system protection, System calls to retrieve file attributes, file types, file operations, reading directories, File system implementation, Directory implementation, allocation methods, Free-Space Management, Efficiency and Performance, Mass storage structure, Disk Structure, Disk scheduling, Disk Management, RAID Structure, OS Security, Access Matrix, Implementation of the Access Matrix, Access Control, Revocation of Access Rights; Shell programming – awk, sed. 14 Hours
Text Book(s):	1. “Operating System Concepts”, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023. 2. “Advanced Programming in the Unix Environment”, Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
Reference Book(s):	3. “Operating Systems, Internals and Design Principles”, William Stallings, 9th Edition, Pearson, 2018. 4. “Modern Operating Systems”, Andrew S Tanenbaum, 3rd edition, Pearson, 2007. 5. “Learning the bash shell”, Cameron Newham, 3rd edition, O’Reilly, 2005.
Course Outcome	• Understand the principles and modules of Operating Systems. • Understand the design of various algorithms for scheduling and their relative performance. • Understand the concept of Deadlocks that typically occur in OS. Deadlocks - Avoidance and Detection. • Implement Operating Systems Concepts related to process management, Concurrent processes, Threads and Memory Management.

Course Code	UE24CS243A	Course Title	Computer Networks				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	4	Type of Course	Core				
Tools/Languages	Cisco Packet Tracer, Wireshark, Mininet Simulator, Python.	Desirable Knowledge					
Prelude	This is a foundation course on Computer Networking which focuses on building blocks of the Internet. We trace the journey of messages sent over the Internet from an application residing on one host machine (source) to another (Destination) using a top-down layered approach. The course contents are organized based on the TCP/IP Protocol stack.						
Course Objectives:	- To provide a comprehensive understanding of the fundamental principles and architectures of computer networks and the Internet. - To enable students to design, implement, and analyze network applications using socket programming and understand the role of application layer protocols. - To equip students with the knowledge to analyze and evaluate transport layer protocols, including their mechanisms for reliable data transfer and congestion control, and to introduce them to the concepts of Software Defined Networking (SDN). - To familiarize students with link layer protocols, error detection/correction techniques, and routing algorithms, enabling them to understand the operational aspects of local area networks and network routing.						
Course Contents	Unit 1: Computer Networks and the Internet, Application Layer Introduction to Computer Networks, Internet: A Nuts-and-Bolts Description, A Services Description, Protocol, The Network Edge: Access Networks, Physical Media, Introduction to physical layer devices, The Network Core, Packet Switching, Circuit Switching, A Network of Networks, Delay, Loss, and Throughput in Packet-Switched Networks, Overview of Delay in Packet Switched Networks – Queuing Delay and Packet Loss, End-to-End Delay, Throughput in Computer Networks, The OSI Model and the TCP/IP Protocol Suite, Protocol Layers, The OSI Model, TCP/IP Protocol Suite. Network Application Principles: Network Application Architectures, Processes Communication, Transport Services available to Applications, Transport Services provided by Internet. 14 Hours Unit 2: Application Layer, Transport Layer – UDP The Web, HTTP and HTTPS, Non persistent and persistent connection, HTTP Message Format, TLS 1.3, User Server Interaction: Cookies, Web Caching. DNS, The Internet's Directory Service: Services provided by DNS; How DNS works: DNS Records and messages; Peer to peer Applications - Overview; Socket Programming with TCP and UDP; Other Application Layer Protocols: FTP, SMTP, SNMP, Telnet, SSH. Introduction to Transport Layer Services: Relationship Between Transport and Network Layer, Overview of the Transport layer in the Internet, Multiplexing and Demultiplexing; Connectionless Transport UDP: UDP Segment Structure, UDP Checksum. Principles of Reliable Data Transfer: Building a Reliable Data Transfer Protocol, (Pipelined Reliable Data Transfer Protocol) RDT variants - Overview, Go-Back-N Protocol, Selective-Repeat; Connection Oriented Transport TCP: TCP Segment Structure, Flow Control, TCP Connection Management 14 Hours						

	Unit 3: Transport Layer – TCP, Network Layer and Internet Protocol TCP Congestion Control. Numerical on TCP congestion control mechanisms–TCP Tahoe, Reno, QUIC Protocol – with HTTP3 Overview of Network Layer: Forwarding and routing, what’s Inside a Router?, The Internet Protocol (IP) IPv4: Datagram Format, Fragmentation, Addressing, NAT, Introduction to SDN, Components and Architecture of SDN, OpenFlow and P4, Data Center Networking – Modern Clos/Fat-tree vs Traditional 3-Tier – Case Study. 14 Hours Unit 4: Network Layer and Internet Protocol, Link Layer and LAN Introduction to Network layer Protocols: DHCP, ICMP; IPv6 Protocol: Packet Format, Transition from IPv4 to IPv6; Introduction to Routing Algorithms: Link State: Dijkstra’s algorithm and Distance Vector: Bellman–Ford Algorithm. Link layer – Error–Detection and Correction techniques, Parity checks, Internet Checksum, Cyclic Redundancy Check, and Multiple Access Protocols: CSMA/CD, CSMA/CA; Switched LAN: Link layer addressing and ARP, Ethernet: Link–layer switches. Retrospective: A Day in the Life of a Web Page Request. Physical Layer – Purpose, Signals to Packets, Transmission media. Wireless LANs: IEEE 802.11 LAN architecture. 14 Hours
Text Book(s):	1. “Computer Networking: A Top – Down Approach”, James F. Kurose, Keith W. Ross, 8th Edition, Pearson Publication, 2022.
Reference Book(s):	1. “TCP IP Protocol Suite”, Behrouz Forouzan, 4th Edition, McGraw–Hill, 2010.
Course Outcome	- Analyze and apply fundamental networking concepts, including protocol layers, network architectures, and addressing schemes (IPv4, IPv6). - Evaluate and implement key application layer protocols (HTTP, DNS, SMTP) and understand their role in network communication. Apply socket programming concepts with TCP and UDP to develop network applications. - Design and analyze transport layer protocols (UDP, TCP), including reliable data transfer, flow control and congestion control mechanisms, and evaluate their impact on network performance. Additionally, gain introductory understanding of Software Defined Networking (SDN) concepts. - Explain and apply link layer protocols (Ethernet, Wi-Fi), error detection/correction techniques, and routing algorithms (Dijkstra, Bellman-Ford) for efficient network operation.

Course Code	UE24CS251B	Course Title	Microprocessor & Computer Architecture				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	4	Type of Course	Core				
Tools/Languages	RISC-V GNU Toolchain, QEMU (riscv64-linux-user), Spike ISA Simulator, Ripes, Dinero IV Cache Simulator, Venus (Web-based).	Desirable Knowledge	Digital Design and Computer Organization				
Prelude	Modern computing demands a deep understanding of how hardware and software co-evolve. This course explores computer architecture through the lens of the open-standard RISC-V instruction set, offering students a rigorous foundation in processor design, arithmetic logic, memory systems, and I/O organization. Spanning instruction-level programming to cache hierarchies and multiprocessor coherence, learners will bridge theoretical principles with practical hardware behavior. By examining real pipeline stages, floating-point standards, and memory technologies, students develop the analytical skills needed to design, evaluate, and optimize computing systems for the challenges of contemporary and future architectures.						
Course Objectives:	- Understand the RISC-V ISA, its instruction formats, register conventions, and ABI calling standards for writing correct low-level programs. - Apply algorithms for integer and floating-point arithmetic including Booth's algorithm, restoring/non-restoring division, and IEEE 754 representations. - Analyze the five-stage pipeline, identify structural, data, and control hazards, and evaluate mitigation strategies such as forwarding and branch prediction. - Examine memory hierarchy design including cache organizations, write policies, virtual memory, and the AMAT performance model. - Explore I/O mechanisms, multiprocessor architectures, MESI cache coherence, and SIMD parallelism at the system level.						

Course Contents	Unit 1 : RISC-V Instruction Set Architecture Introduction to RISC-V ISA: History, design principles, and comparison with CISC (x86), RISC-V Instruction Formats, Integer Arithmetic Instructions, Data Transfer Instructions, memory addressing, Control Flow Instructions, RISC-V Register File: 32 general-purpose registers, ABI calling conventions, pseudo-instructions, Procedure Calls: Stack frame, caller/callee-saved registers, leaf and non-leaf procedures. 14 Hours Unit 2 : Arithmetic for Computers Multiplication: Booth's algorithm, hardware multiplier, Division: Restoring and non-restoring algorithms, hardware divider, IEEE 754 single and double precision, special values, Floating-Point Arithmetic: Addition, subtraction, multiplication, rounding modes, RISC-V Floating-Point Extension (RV32F/RV64F/RV64D): FP registers, FP instructions, RISC-V M Extension: MUL, MULH, DIV, REM and their unsigned variants, Parallelism and Arithmetic: SIMD, subword parallelism concepts. 14 Hours Unit 3 : Pipelining Introduction to Pipelining: Five-stage pipeline, pipeline registers, throughput improvement, Pipeline Hazards: Structural hazards, data hazards (RAW, WAW, WAR), control hazards, Forwarding / Bypassing and Stalling: Hazard detection unit, forwarding paths, Branch Prediction: Static prediction, dynamic prediction (1-bit, 2-bit predictors), BTB. 14 Hours Unit 4 : Memory Hierarchy and I/O Systems Memory Hierarchy Overview: Temporal and spatial locality, memory technologies, Cache Memory: Direct-mapped, set-associative, and fully-associative caches, Cache Performance: Miss rate, hit time, miss penalty, AMAT formula, Cache Write Policies: Write-back, write-through, write-allocate, no-write-allocate, Cache Optimization: Multi-level Caches and Virtual Memory: L1/L2/L3 hierarchy, TLB, page tables, Main Memory and DRAM: SDRAM, DDR, bandwidth and latency considerations, Storage: HDD, SSD, flash memory, RAID levels overview, I/O Subsystems: Buses, programmed I/O, interrupt-driven I/O, DMA, Multiprocessors: SMP, cache coherence (MESI protocol), memory consistency. 14 Hours
Laboratory	1. Introduction to RISC-V Toolchain: Setting up RISC-V GNU toolchain (riscv64-unknown-elf-gcc), assembler (riscv64-unknown-elf-as), linker, and QEMU emulator. Compile, link, and execute a 'Hello, World' program in RISC-V assembly and C. 2. RISC-V Arithmetic & Logic Instructions: Write assembly programs using ADD, ADDI, SUB, AND, OR, XOR, SLL, SRL, SRA. Verify register contents using GDB (riscv64-unknown-elf-gdb) and QEMU. 3. Data Transfer and Memory Access: Programs for LW, SW, LH, LB, SH, SB instructions. Implement array storage and retrieval in memory. Use memory inspection commands in GDB. 4. Procedure Calls and Stack Management: Implement leaf and non-leaf procedure calls. Demonstrate caller/callee-saved register conventions. Implement factorial and Fibonacci using recursive calls. 5. Control Flow and Loops: Implement programs using BEQ, BNE, BLT, BGE, JAL, JALR for if-else, for-loop, while-loop, and do-while-loop structures in RISC-V assembly. 6. Sorting Algorithms in Assembly: Implement Bubble Sort and Selection Sort on integer arrays in RISC-V assembly. Profile execution using QEMU's built-in performance counters or Spike ISA simulator (spike + pk). 7. Floating-Point Operations: Programs using RISC-V F/D extensions: FADD.S, FSUB.S, FMUL.S, FDIV.S, FSQRT.S. Implement vector dot product using FP registers. Use Spike simulator with -- isa=rv64imafd. 8. Pipeline Simulation using Venus / Ripes: Load RISC-V assembly programs into Ripes (a visual pipeline simulator). Observe the 5-stage pipeline, data hazards, forwarding, and stalling in the pipeline diagram GUI. 9. Cache Simulation using Dinero IV / CacheSim: Run memory traces of programs (bubble sort, matrix multiply) through Dinero IV cache simulator. Compare hit/miss rates for direct-mapped vs set-associative vs fully-associative caches with varying sizes.

	10. RISC-V Processor Design using Logisim / Digital: Implement the single-cycle RISC-V datapath (ALU, register file, control unit, instruction memory, data memory) in Logisim-evolution. Test with sample RISC-V machine code programs. 11. Project 12. Project 13. Project.
Text Book(s):	1. David A. Patterson and John L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 2nd Edition, Morgan Kaufmann (Elsevier), 2021. ISBN: 978-0-12-820331-6.
Reference Book(s):	1. John L. Hennessy and David A. Patterson, Computer Architecture: A Quantitative Approach, 6th Edition, Morgan Kaufmann, 2019. ISBN: 978-0-12-811905-1. 2. Andrew Waterman and Krste Asanovic (Eds.), The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA (Document Version 20191213), RISC-V Foundation, 2019. Available at: riscv.org/specifications. Online References : • CS61C: Great Ideas in Computer Architecture (RISC-V) — UC Berkeley, lecture videos on YouTube and inst.eecs.berkeley.edu/~cs61c • RISC-V International Learning Resources — learn.riscv.org • MIT OpenCourseWare 6.004: Computation Structures — ocw.mit.edu • Venus RISC-V Simulator (Web) — venus.cs61c.org • Ripes Pipeline Simulator — github.com/mortbopet/Ripes • RISC-V GNU Toolchain — github.com/riscv-collab/riscv-gnu-toolchain • Logisim-Evolution — github.com/logisim-evolution/logisim-evolution
Course Outcome	• Write and trace RISC-V assembly programs using correct instruction formats, pseudo-instructions, and stack-based procedure call conventions. • Perform manual arithmetic operations using hardware algorithms and convert numbers to and from IEEE 754 single/double precision formats. • Trace instruction execution through a pipelined datapath, detect hazards, and assess the effectiveness of forwarding, stalling, and prediction techniques. • Calculate AMAT for various cache configurations, compare write policies, and reason about TLB and page table interactions in virtual memory systems. • Evaluate I/O strategies (DMA, interrupt-driven), RAID trade-offs, and cache coherence states under the MESI protocol in multiprocessor environments.

Course Code	UE24CS352A	Course Title	Machine Learning				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	4	Type of Course	Core Course				
Tools/Languages	Pytorch	Desirable Knowledge	Mathematical Foundation for AI & data Science				
Prelude	Machine Learning has become a core technology behind applications such as recommendation systems, virtual assistants, autonomous vehicles, fraud detection, and intelligent search engines. This course introduces the fundamental concepts and algorithms of Machine Learning, including Decision Trees, KNN, SVM, Bayesian Learning, clustering, neural networks, introductory deep learning, Reinforcement Learning, and MLOps. The course aims to provide students with both theoretical foundations and practical understanding of modern ML techniques used in intelligent systems. A desirable background in Mathematical Foundation for AI & data Science is recommended.						
Course Objectives:	- To understand the fundamental concepts of Machine Learning, performance evaluation metrics, and supervised learning techniques such as Decision Trees and KNN. - To study Artificial Neural Networks, Support Vector Machines, and ensemble learning methods for classification and prediction tasks. - To analyze probabilistic learning methods including Bayesian Learning, Gaussian Mixture Models, Hidden Markov Models and Unsupervised Learning - To explore Reinforcement Learning, introductory Deep Learning concepts, and MLOps practices for real-world ML systems.						
Course Contents	Unit1: Introduction, Performance Metrics, Classification with Decision trees and KNN: Introduction to ML: Machine Learning and its Models, Concept Learning: Concepts of Hypotheses, Version space, inductive bias, Performance metrics: Accuracy, Precision, Recall, Sensitivity, Specificity, AUC, ROC, Bias-Variance decomposition, Bias Variance trade-off, Cross validation strategies, Decision Trees: Basic algorithm(ID3) for classification, Decision boundary for Decision Trees(x-y axis), Hypothesis search and Inductive bias, Issues in Decision Tree Learning – Over fitting, Solutions to over fitting, dealing with continuous values, Logistic Regression, Instance-based learning: k-nearest neighbor learning (Classification & Regression), Decision boundary for KNN. 14 Hours Unit2: Artificial Neural Networks (ANN), Support Vector Machines (SVM), Boosting: Artificial Neural Networks: Introduction, Perceptrons, Multi-layer networks and Back-propagation, Activation functions (Step, Sigmoid, Tanh, ReLU), Various Optimizers (GD, SGD, Momentum-based, Adagrad, Adam) Support Vector Machines – Margin and maximization, SVM - The primal problem, the Lagrangian dual, SVM – Solution to the Lagrangian dual, (Hard Margin and Soft Margin, Classification ONLY), Kernel Functions :Linear, Polynomial (Derivation only for linear function).Combining weak learners, improving performance with Gradient Boost, Random Forest . 14 Hours Unit3: Bayesian Learning, HMM and Unsupervised Learning Bayesian Learning –Bayes theorem, Maximum likelihood, Bayes optimal classifier, Naïve Bayes Classifier, Expectation maximization and Gaussian Mixture Models. Hidden Markov Models (HMMs), Hierarchical vs. non-hierarchical clustering, Agglomerative and divisive clustering, K-means clustering, Bisecting K-means, K-Means as special case of Expectation Maximization,						

	14 Hours Unit 4: ML deployment and advanced learning paradigms Introduction to Reinforcement Learning: agent, environment, states, actions, rewards, policy, value functions, Bellman equation, and Q-learning fundamentals., Introduction to Deep Learning, Introduction to Convolution operation and Convolution Neural Network (CNNs). (Parameter calculation, Max pooling, Avg pooling). Hyper parameter tuning methods: Grid Search, Random Search, Bayesian Optimization, Introduction to MLOps: MLOps Overview, ML Lifecycle & Production Challenges, Data & Feature Pipelines, Experiment Tracking & Model Versioning, Model Deployment & Serving, CI/CD/CT for ML Systems, Inference Systems, Monitoring, Drift & Observability, ML Orchestration. 14 Hours
Laboratory	Applications using: 1. Decision Trees. 2. ANN 3. SVM 4. Naïve Bayes 5. HMM 6. Clustering/PCA/SVD 7. CNN model
Text Book(s):	1. "Machine Learning", Tom Mitchell, McGraw Hill Education (India), 2013 (2017-reprint). 2. "Introduction to Machine Learning", Ethem Alpaydin, MIT Press Ltd. 4th Edition, 2020 3. Designing Machine Learning Systems – Chip Huyen, 2022
Reference Book(s):	1. "Machine Learning: The Art and Science of Algorithms that Make Sense of Data", Peter Flach, Cambridge University Press (2012). 2. "Artificial Intelligence: A Modern Approach (3rd Edition)", Stuart Russel and Peter Norvig, Pearson, 2009. 3. "Neural Networks in Deep Learning", Charu Agarwal, Springer International Publishing AG , part of Springer Nature 2018. 4. "Pattern Recognition and Machine Learning", Christopher Bishop, Springer (2nd Printing), 2011
Course Outcome	• Apply supervised learning algorithms and performance evaluation techniques to solve classification and regression problems. • Design and implement ANN, SVM, and ensemble-based models for predictive analytics applications. • Apply Bayesian methods, Hidden Markov Models, and clustering algorithms to model uncertainty and perform unsupervised learning tasks on real-world datasets. • Understand and apply reinforcement learning, deep learning, CNNs, hyperparameter tuning, and MLOps practices to build, deploy, and monitor scalable machine learning systems.

Course Code	UE25CS241B	Course Title	Design & Analysis of Algorithm				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	4	Type of Course	Core				
Tools/Languages	C-Programming language	Desirable Knowledge	Problem Solving with C				
Prelude	In the evolving landscape of computing, algorithms form the backbone of every efficient solution—from simple searching tasks to complex decision-making systems. The course Design and Analysis of Algorithms introduces students to the art and science of crafting efficient, scalable, and elegant solutions to computational problems. At its core, this course emphasizes not just <i>how</i> to solve a problem, but <i>how efficiently</i> it can be solved. Students will explore fundamental problem-solving strategies such as brute force, divide-and-conquer, decrease-and-conquer, greedy methods, and dynamic programming, gaining insight into when and why each approach is most effective. The course also highlights the importance of evaluating algorithms through time complexity, enabling learners to make informed design decisions.						
Course Objectives:	- To understand the fundamentals of algorithmic problem solving and analyze the efficiency of algorithms using asymptotic notations, while applying brute force techniques to solve basic computational problems. - To design and analyze algorithms using decrease-and-conquer and divide-and-conquer strategies, and evaluate their efficiency through recurrence relations and complexity analysis. - To apply transformation techniques, space-time tradeoffs, and greedy strategies for designing efficient algorithms for optimization and graph-based problems. - To develop dynamic programming solutions for complex problems and understand the limitations of algorithmic computation, including NP-completeness and strategies such as backtracking and branch-and-bound.						
Course Contents	Unit1: Introduction and Brute Force Algorithms: Fundamentals of Algorithmic Problem Solving, Important Problem Types. Analysis of Algorithm Efficiency: Analysis Framework, Asymptotic Notations and Basic Efficiency Classes, Mathematical Analysis of Non-Recursive and Recursive Algorithms. Brute Force: Selection Sort, Bubble Sort, Sequential Search, Brute Force String Matching, Exhaustive Search 14 Hours Unit2: Decrease – and – Conquer & Divide-and-Conquer Decrease-and-Conquer: Decrease by constant number algorithms - Insertion Sort, Topological Sorting, Algorithms for Generating Combinatorial Objects, Decrease-by-a-Constant-Factor Algorithms – Fake Coin Problem, Russian Peasant Multiplication, Josephus problem, Decrease-by-Variable-Size Algorithms – Computing a median and the selection problem. Divide-and-Conquer: Master Theorem, Merge Sort, Quick Sort, Binary Search, Binary Tree Traversals, Complexity analysis for finding the height of BST, Multiplication of Large Integers, Strassen’s Matrix Multiplication. 14 Hours Unit3: Transform-and-Conquer Space and Time Tradeoffs & Greedy Technique Transform and Conquer: Pre-sorting, Heap Sort, Red-Black Tree Construction and Time complexity Analysis for insert and search operation, 2-3 Trees and B Tree: insertion, deletion, searching, and time complexity analysis. Space and Time Tradeoffs: Sorting by Counting, Input Enhancement in String Matching - Horspool’s and Boyer-Moore Algorithms. Greedy Technique: Prim’s Algorithm, Kruskal’s Algorithm and union-find algorithm, Dijkstra’s Algorithm, Huffman Trees 14 Hours Unit4: Limitations, Coping with the Limitations of Algorithm Power & Dynamic Programming Dynamic Programming: Computing a Binomial Coefficient, The Knapsack Problem and Memory Functions,						

	Warshall's and Floyd's Algorithms. Limitations of Algorithm Power: Lower-Bound Arguments, Decision Trees, P, NP, and NPComplete, NP-Hard Problems. Coping with the Limitations of Algorithm Power: Backtracking, Branch and Bound. 14 Hours
Text Book(s):	1. "Introduction to the Design and Analysis of Algorithms", Anany Levitin, Pearson Education, Delhi(Indian Version), 3rd Edition, 2017.
Reference Book(s):	1. "Introduction to Algorithms", Thomas H Cormen, Charles E Leiserson, Ronald L Rivest and Clifford Stein, PrenticeHall India, 3rd Edition, 2009. 2. "Fundamentals of Computer Algorithms", Horowitz, Sahni, Rajasekaran, Universities Press, 2nd Edition, 2007. 3. "Algorithm Design", Jon Kleinberg, Eva Tardos, Pearson Education, 1st Edition, 2006.
Course Outcome	- Analyze the efficiency of algorithms using asymptotic notations and apply brute force techniques to solve fundamental computational problems. - Design and implement algorithms using decrease-and-conquer and divide-and-conquer paradigms, and evaluate their performance using recurrence relations and complexity analysis. - Apply transformation techniques, space-time tradeoffs, and greedy methods to design efficient solutions for sorting, searching, and graph problems. - Develop dynamic programming solutions for optimization problems and analyze computational limitations using concepts such as NP-completeness, backtracking, and branch-and-bound.

Course Code	UE25CS242B	Course Title	Cybersecurity Essentials				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	4	Type of Course	Core				
Tools/Languages	Wireshark, Kali Linux, Python	Desirable Knowledge	Computer Network				
Prelude	This course will cover the essentials of Cyber security, and students will learn about the characteristics of security principles, technologies, and procedures to secure networks, software and other assets of an organization. They will also gain an insight into Risk Management, Incident Management, Cryptography, Operations Security, Digital Forensics etc						
Course Objectives:	- To introduce students to the fundamentals of cybersecurity, network security concepts, attack methodologies, and security best practices for protecting digital systems and networks. - To provide understanding of common web vulnerabilities, secure web communication mechanisms, and ethical security assessment techniques through concepts aligned with OWASP and MITRE frameworks. - To familiarize students with cyber laws, data privacy principles, global regulatory frameworks, and legal challenges associated with cybercrimes and digital data protection. - To expose students to the role of Artificial Intelligence in cybersecurity, including AI-driven security applications, AI risks, adversarial machine learning concepts, and emerging security challenges in modern AI systems.						
Course Contents	Unit 1 – Introduction to Cybersecurity Introduction : CIA principles, Vulnerability, Threat and Risk, Anatomy of an Attack, Attack Landscape, Real-life Cyber-attacks, Security vs. Privacy, Security Cornerstones, Cyber security Framework, Security Mindset, Security Policy and Culture, Security Pillars, Security Principles, Defense in depth, Cyber security best practices: Strong password, Social engineering techniques. Basics of Network Security : NAT, Socket Programming using python, Packet Construction inside kernel , packet sending tools (netcat, telnet, ssh, /dev/tcp, /dev/udp, ping). Packet Sniffing and Spoofing: Network Interface Card (NIC), Sniffing and spoofing using Scapy. 14 Hours Unit 2: Web Vulnerabilities Web basics - Browser–Server Communication Model, HTTP vs HTTPS, GET vs POST methods (security implications), Cookies & Session Management, OWASP Top 10, MITRE Att&ck Framework, Cross-Site Request Forgery (CSRF – concept), Cross-Site Scripting (XSS – concept), SQL Injection (concept & prevention). Introduction to CTF. 14 Hours Unit 3: Cyber Law & Data Privacy Cyber Laws in digital age: impacts, risks, X-Border Data Flow, Cyber Crimes - What is it, types of crimes, Behavioral Aspects (Technology intoxication), Cyber Crimes against Individuals, Institution and State, Cyber terrorism, Cyber Warfare issues and legal challenges. Discuss and compare regulations like Indian IT Act GDPR, PDPA, CCPA, DPDPA 2023 etc. Legal Provisions to handle Cyber Crimes, Cyber Law and Compliance issues in Industry, International Approach to regulate cyberspace. Data privacy : Fundamentals of Data Privacy, Data Classification, Privacy eco-system, Privacy evolution in today's context, trends, driving factors and its Global importance, The Data Life Cycle, Privacy Principles and Framework, Privacy in Digital Age, Privacy Risks (what, how to identify, when). Case Study						

	14 Hours Unit 4: AI Security: Applications of AI in Cyber Security, AI Capabilities - Understanding current AI technological abilities, Scope and limitations of modern AI systems, AI Risks and X-Risks - Analysis of long-term AI development trajectories. Bias and Stereotypes, Handling unexpected inputs and scenarios. Adversarial ML Taxonomy and attacks (Prompt injection, Model misuse risks, Data poisoning (basic idea)) 14 Hours
Text Book(s):	1. Internet Security: A Hands-on Approach, Wenliang Du, 3rd Edition, 2022 2. Virginia Dignum, "Responsible Artificial Intelligence: How to Develop and Use AI in a Responsible Way" Springer Nature, 04-Nov-2019; ISBN-10 : 3030303705, ISBN-13 : 978-3030303709 3. Natraj Venkatraman, Ashwin Shriram, Data Privacy: Principles and Practice, CRC Press, 2017
References/ Reference Book	1. Richard M. Thompson II, Emily C. Barbour and Alison M. Smith, Understanding Privacy and Data 2. Protection: What You Need to Know, Nova Publishers New York, 2014 3. Data Privacy: A Runbook for Engineers Book by Nishant Bhajaria https://ico.org.uk/ https://www.iapp.org/ https://commission.europa.eu/law/law-topic/data-protection_en https://www.dsci.in/ https://meity.gov.in/ https://www.linklaters.com/ https://www.dlapiper.com/ https://www.dataguidance.com/
Course Outcome	● Apply AI techniques for cybersecurity use cases such as threat detection and analysis. ● Evaluate strengths, limitations, and risks of AI systems in security contexts . ● Identify and analyze adversarial attacks including prompt injection and data poisoning . ● Assess robustness and reliability of AI models under different conditions . ● Implement responsible and ethical AI practices in cybersecurity systems.

Course Code	UE25CS243B	Course Title	Full Stack Development – Theory and Practice				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	4	Type of Course	Core Course				
Tools/ Languages	HTML, CSS, JS, ReactJS, NodeJS, ExpressJS, MongoDB, MySQL	Desirable Knowledge	None				
Prelude	Full Stack Development is a hands-on, project-oriented course that takes students through every layer of a modern web application — from data modelling and SQL to reactive frontends and server-side APIs. Beginning with relational database design and advancing through asynchronous JavaScript, React, Node.js, Express, and MongoDB, the course mirrors the actual workflow of a professional development team. Students progress logically from foundational web and database concepts, through intermediate SQL and component-based UI development, to building and connecting production-ready backend services — emerging with the practical skills to design, develop, and integrate a complete, data-driven web application.						
Course Objectives:	• To ground students in web architecture and relational database fundamentals — covering three-tier application design, ER modelling, schema mapping, and core SQL • To extend database competency into advanced SQL and NoSQL paradigms with document-oriented storage with MongoDB, while being awareness of graph, vector, and time-series databases. • To develop fluency in client-side web development — from HTML, CSS3 and JavaScript and building functional, component-based user interfaces with React, hooks, and client-side routing. • To equip students with the skills to build and expose RESTful APIs using Express.js and to wire a React frontend to a Node.js backend, producing a fully integrated, deployment-ready full stack application.						

Course Contents	Unit 1 - Introduction to Full stack and Database Concepts Web architecture: Introduction, Web Application Architectures (3-tier) and the technology stacks. Introduction to databases: Purpose and advantages of DBMS, database application architecture. Data modeling: Entity-Relationship model, ER diagrams, mapping ER to relational schemas. SQL basics: Data types, DDL (CREATE, ALTER, DROP), DML (SELECT, INSERT, UPDATE, DELETE), constraints and keys. Intermediate SQL: Multi-table joins (inner, left, right), aggregate functions, GROUP BY and HAVING 14 hours Unit 2 - Advanced DBMS and NoSQL Databases Advanced Database: Sub-queries and Correlated queries, Triggers, Functions and Procedures. Normalisation: Functional dependencies, 1NF, 2NF, 3NF — identifying and resolving anomalies. Introduction to NoSQL: NoSQL vs relational databases, key-value stores, document stores — when to use what. NoSQL DB Types: Types of NoSQL DB - Graph databases, Vector Databases, Time Series Databases. MongoDB basics: Documents and collections, CRUD operations. 14 hours Unit 3 - Web Fundamentals and React HTML5 and CSS3: HTML Semantic elements, forms and inputs, CSS. JavaScript essentials: Built-in objects, DOM manipulation, Events. Async JavaScript: Callbacks, promises, async/await; consuming REST APIs with Axios, JSON syntax, working with JSON in JavaScript. Introduction to React: React installation (Vite), JSX, Functional components, props, state with useState. React hooks: useEffect for side effects and data fetching, React Router for navigation 14 hours Unit 4 - Advanced React, Backend and Database Integration React forms and event handling: Controlled components, form validation, passing state and handlers from parent to child components. Express.js: Web service concepts, REST API design, routing, middleware, error handling, Drivers for SQL and MongoDB connectivity. Full stack integration: Connecting React frontend to Express backend via REST APIs, handling CORS, environment configuration 14 hours
Text Book(s):	1. "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. 2. "Web Development with Node and Express" by Ethan Brown, O'Reilly, 2nd ed., 2019/2020 3. "Full-Stack React Projects" by Shama Hoque, Packt, 2nd ed., 2020
Reference Book(s):	1. Database Management Systems, R Ramakrishnan, J Gehrke, 3rd Edition, McGraw Hill, 2002. 2. "Node.js Design Patterns" by Casciaro & Mammino, Packt, 4th ed., 2025 3. "Learning React, Functional Web Development with React and Redux", Alex Banks and Eve Porcello, O'Reilly Media, 2nd Edition, 2020 4. "An Introduction to Database Systems" by C. J. Date, Pearson Education, 8th Edition, 2003
Course Outcome	• Explain the three-tier web application model, design entity-relationship diagrams for real-world data scenarios, map them to relational schemas, and write SQL queries using DDL, DML, multi-table joins, and aggregate functions. • Write advanced SQL using subqueries, correlated queries, triggers, and stored procedures; apply normalisation up to 3NF to resolve data anomalies; and perform CRUD operations in MongoDB, articulating when a NoSQL store is preferable to a relational database. • Build semantic, interactive web pages with HTML5 and CSS3; manipulate the DOM and handle events with JavaScript; use async/await and Axios to consume REST APIs; and develop React applications with functional components, props, state, useEffect, and React Router. • Design and implement controlled React forms with validation; build RESTful Express.js APIs with structured routing, middleware, and SQL/MongoDB drivers; and integrate a React frontend with a Node.js/Express backend — managing CORS and environment configuration — to deliver a working, full stack web application.

Course Code	UE24CS351A	Course Title	Database Management System				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	5	Type of Course	Core Course				
AI Tools /Tools/Lang uages	MySQL Workbench, Python, Redis, Neo4j	Desirable Knowledge	-				
Prelude	This course aims to introduce the foundational and advanced concepts of database management systems. It provides a comprehensive understanding of relational algebra, SQL, normalization, transaction management, NoSQL databases, and emerging database technologies. It prepares students to analyse data requirements, design efficient database solutions, and implement optimized queries with modern data platforms.						
Course Objectives:	● To introduce fundamental concepts of database systems, relational algebra, and basic SQL querying. ● To develop the ability to use advanced SQL techniques and understand the strategies for efficient query processing and optimization. ● To equip students with normalization principles, functional dependencies, and transaction management to ensure data integrity. ● To understand the modern data management systems including NoSQL, graph and vector databases.						

Course Contents	Unit 1: Introduction to Database Management and SQL Purpose, View of data, Database design- Introduction to databases, Database application architecture, E-R Model, reducing ER to a relational schema. Relational operations (Algebra), Unary Operations - Unity, Binary, Aggregate Functions, Grouping, SQL overview, Data definition, Structure of SQL queries, constraints and keys. 14 Hours Unit 2: Advanced SQL in Database Design: Database Modification, Basic Operations, Set Operations, Null Values, Aggregate Functions, Nested Subqueries, Corelated queries , Join expressions, Views, Triggers, Functions, Procedures, CTE, windows functions and full text search functions. 14 Hours Unit 3: Design Concepts and Implementation Introduction to strategies of Query processing and Query optimization, Functional Dependencies, Inference Rules, Normal Forms Based on Primary Keys (1NF, 2NF, and 3NF), General Definitions of Second and Third Normal Forms, Boyce-Codd Normal Form, Higher Normal Forms. 14 Hours Unit 4: Next-Gen Data Management Database transactions, Concurrency control, Locking, Introduction to NoSQL databases, Key-value database, Graph databases (Neo4j), Introduction to Vector databases, Vector search with Relational and NOSQL databases. Web Application development using Next.js and SQL 14 Hours
Laboratory	1. Draw an ER diagram for a given problem statement and Conversion of an ER diagram into Relational schema 2. DDL and DML 3. SQL – Joins: inner, outer; Sub queries: correlated and uncorrelated 4. SQL – Creating Functions and Procedures 5. SQL – Creating Triggers, Transaction isolation 6. NoSQL database queries Neo4J and Redis 7. Vector database 8. Web-Application that includes: • Backend with Database using SQL (MySQL) and NoSQL (any NoSQL database) • Frontend with Next.js created using LLM

Text Book(s):	1: “Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017. 2: “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. 3: Essential Concepts of Vector Databases: Mastering Vector Databases for Advanced Data Management by Paulo Dichone, 1st edition, 2024 ,Packt publisher.
Reference Book(s):	1. Database Management Systems, R Ramakrishnan, J Gehrke, 3rd Edition, McGraw Hill, 2002. 2. Data on the Web: From Relations to Semistructured Data and XML, S Abiteboul, P Buneman, D Suciu, Morgan Kauffman, 1999. 3. “An Introduction to Database Systems” by C. J. Date, Pearson Education, 8th Edition, 2003
Course Outcome	● Demonstrate an ability to explain the structure of a database system, ER diagrams, relational algebra, and write basic SQL queries involving constraints and keys ● Construct and optimize SQL queries including nested subqueries, joins, views, triggers, and stored procedures ● Apply normalization techniques using functional dependencies and design schemas, and ensure the implementation of transaction and concurrency control mechanisms. ● Create applications of graph and key value databases, and develop basic application using vector database

Course Code	UE25CS352A	Course Title	Machine Learning				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	5	Type of Course	Core Course				
Tools/Languages	Pytorch	Desirable Knowledge	Mathematics for Computer Science Engineers, Linear Algebra & its applications.				
Prelude	Machine Learning has become a core technology behind applications such as recommendation systems, virtual assistants, autonomous vehicles, fraud detection, and intelligent search engines. This course introduces the fundamental concepts and algorithms of Machine Learning, including Decision Trees, KNN, SVM, Bayesian Learning, clustering, neural networks, introductory deep learning, Reinforcement Learning, and MLOps. The course aims to provide students with both theoretical foundations and practical understanding of modern ML techniques used in intelligent systems. A desirable background in Mathematical Foundation for AI & data Science is recommended.						
Course Objectives:	- To understand the fundamental concepts of Machine Learning, performance evaluation metrics, and supervised learning techniques such as Decision Trees and KNN. - To study Artificial Neural Networks, Support Vector Machines, and ensemble learning methods for classification and prediction tasks. - To analyze probabilistic learning methods including Bayesian Learning, Gaussian Mixture Models, Hidden Markov Models and Unsupervised Learning - To explore Reinforcement Learning, introductory Deep Learning concepts, and MLOps practices for real-world ML systems.						
Course Contents	Unit1: Introduction, Performance Metrics, Classification with Decision trees and KNN: Introduction to ML: Machine Learning and its Models, Concept Learning: Concepts of Hypotheses, Version space, inductive bias, Performance metrics: Accuracy, Precision, Recall, Sensitivity, Specificity, AUC, ROC, Bias-Variance decomposition, Bias Variance trade-off, Cross validation strategies, Decision Trees: Basic algorithm(ID3) for classification, Decision boundary for Decision Trees(x-y axis), Hypothesis search and Inductive bias, Issues in Decision Tree Learning – Over fitting, Solutions to over fitting, dealing with continuous values, Logistic Regression, Instance-based learning: k-nearest neighbor learning (Classification & Regression), Decision boundary for KNN. 14 Hours Unit2: Artificial Neural Networks (ANN), Support Vector Machines (SVM), Boosting: Artificial Neural Networks: Introduction, Perceptrons, Multi-layer networks and Back-propagation, Activation functions (Step, Sigmoid, Tanh, ReLU), Various Optimizers (GD, SGD, Momentum-based, Adagrad, Adam) Support Vector Machines – Margin and maximization, SVM - The primal problem, the Lagrangian dual, SVM – Solution to the Lagrangian dual, (Hard Margin and Soft Margin, Classification ONLY), Kernel Functions :Linear, Polynomial (Derivation only for linear function).Combining weak learners, improving performance with Gradient Boost, Random Forest . 14 Hours Unit3: Bayesian Learning, HMM and Unsupervised Learning Bayesian Learning –Bayes theorem, Maximum likelihood, Bayes optimal classifier, Naïve Bayes Classifier, Expectation maximization and Gaussian Mixture Models. Hidden Markov Models (HMMs), Hierarchical vs. non-hierarchical clustering, Agglomerative and divisive clustering, K-means clustering, Bisecting K-means, K-Means as special case of Expectation Maximization, 14 Hours Unit 4: ML deployment and advanced learning paradigms Introduction to Reinforcement Learning: agent, environment, states, actions, rewards, policy, value						

	functions, Bellman equation, and Q-learning fundamentals., Introduction to Deep Learning, Introduction to Convolution operation and Convolution Neural Network (CNNs). (Parameter calculation, Max pooling, Avg pooling). Hyper parameter tuning methods: Grid Search, Random Search, Bayesian Optimization, Introduction to MLOps: MLOps Overview, ML Lifecycle & Production Challenges, Data & Feature Pipelines, Experiment Tracking & Model Versioning, Model Deployment & Serving, CI/CD/CT for ML Systems, Inference Systems, Monitoring, Drift & Observability, ML Orchestration. 14 Hours
Laboratory	Applications using: 1. Decision Trees. 2. ANN 3. SVM 4. Naïve Bayes 5. HMM 6. Clustering/PCA/SVD 7. CNN model
Text Book(s):	1. “Machine Learning”, Tom Mitchell, McGraw Hill Education (India), 2013 (2017-reprint). 2. “Introduction to Machine Learning”, Ethem Alpaydin, MIT Press Ltd. 4th Edition, 2020 3. Designing Machine Learning Systems – Chip Huyen, 2022
Reference Book(s):	1. “Machine Learning: The Art and Science of Algorithms that Make Sense of Data”, Peter Flach, Cambridge University Press (2012). 2. “Artificial Intelligence: A Modern Approach (3rd Edition)”, Stuart Russel and Peter Norvig, Pearson, 2009. 3. “Neural Networks in Deep Learning”, Charu Agarwal, Springer International Publishing AG , part of Springer Nature 2018. 4. “Pattern Recognition and Machine Learning”, Christopher Bishop, Springer (2nd Printing), 2011
Course Outcome	• Apply supervised learning algorithms and performance evaluation techniques to solve classification and regression problems. • Design and implement ANN, SVM, and ensemble-based models for predictive analytics applications. • Apply Bayesian methods, Hidden Markov Models, and clustering algorithms to model uncertainty and perform unsupervised learning tasks on real-world datasets. • Understand and apply reinforcement learning, deep learning, CNNs, hyperparameter tuning, and MLOps practices to build, deploy, and monitor scalable machine learning systems.

Course Code	UE24CS341A	Course Title	Software Engineering				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Core Course				
Tools/Languages	Git, GitHub, docker, Jenkins, gtest, pytest, Jira/OpenProject, SonarQube	Desirable Knowledge	-				
Prelude	In this course, students learn and gain practical experience with software engineering principles, techniques, and tools. Topics in this course include requirements analysis, specification, architecture, design, programming style, testing, quality management, security validation, devOps, teamwork, and software project management. The practical experience centers on a semester-long team project, in which a software development project is carried through all the stages of the software life cycle. Particular emphasis is placed on performing all SDLC activities similar to industry setup. Assignments further provide a hands-on experience in using state-of-the-art techniques and tools.						
Course Objectives:	- Learn Software Engineering concepts. Ensure the relevance and need of an engineering approach to software development. Expose students to the software project management and Life Cycles. - Expose students to agile development methodology using standard tools and associated specifications - Enable the students to understand the principles of Software Configuration Management and importance of software quality for Software Product Development. - Expose students to the continuous integration and continuous development, build, test and release of software products through a semester long project.						
Course Contents	Unit 1: Introduction to Software Engineering and Requirements Engineering Introduction, Case studies and motivation for SE, DevOps and CI/CD tools introduction and setup, Software development lifecycles overview (waterfall etc) and motivation for agile methodology. Agile approach of software development, Contrasting Agile and Plan driven approaches, Agile SCRUM model and exposure to other Agile approaches like Lean Agile, XP etc. Hands on Jira training. Security development lifecycle. Requirements Engineering: Requirement's introduction and properties, Feasibility, Requirements Elicitation, Analysis and modelling, Specification and Verification, Requirement Management and Requirements Traceability; Introduction to UML and development of use cases. Validation planning and security validation plan. 16 Hours Unit 2: Software Project Management and Software Architecture, Design and Quality Fundamentals, Software Project Management Lifecycle, Planning activities, Choice of Lifecycles, Project Organization, Software Estimation, Scheduling, Risk Management, Quality Management, Gantt charts basics, Monitoring of Execution and Control, Project Closure. RACI matrix with example Software Architecture: Case study for motivation of SW Arch, Architectural View, Styles, Architectural and Design Patterns. Modularity, interface definition, microservices intro, Security architecture, Agile context. Software Design: Modularity, importance of interfaces (API and ABI), Error definition, handling and management, Forward and backward compatibility, usage of UML. Quality and Test case development: Case studies and motivation, types of testing, technical debt, Test cases documentation, Black and white box testing. 12 Hours Unit 3: AI, Implementation, SCM, DevOps AI for SW Engineering and SE for AI/ML systems. CI/CD pipeline and tools for development, Introduction						

	to software construction, Coding principles, standards and guidelines, Factors for effective coding - Defensive, Secure and Testable programming, Code Review / Peer Review, Security code reviews; logging, tracing, debugging techniques. Test case development and dynamic analysis, functional and non functional testing, security testing, Automation; mutation testing to validate test suite Software Configuration Management (SCM): Elements of a Configuration Management System, Configuration Items, Baselines, Repositories, Branch Management, Build Management, Install, Change and Release Management, Patching and Patch Management 16 Hours Unit 4: Usability, Quality and System Testing ICase study discussion, Industry talk, Usability, UX design, Code coverage, branch coverage; Ethics and Software Engineering; Opensource Introduction; SW maintenance and support; Security and Privacy aspects; Measurements and Matrix for Quality; Risk management for Quality Security Validation – Penetration Testing, Fuzzing; System testing – Regression testing, Integration testing, Demo and student presentations for the mini-project 12 Hours
Text Book(s):	1. “Software Engineering”, International Computer Science Series, Ian Somerville, Pearson Education, 10th Edition, 2021 2. “Software Engineering: A Practitioner’s Approach”, Roger S Pressman, McGraw Hill, 9th Edition, 2019
Reference Book(s):	1. “Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling” by By Jennifer Davis, Ryn Daniels, O’ Reilly Publications, 2018 2. “Software Engineering: Principles and Practice”, Hans van Vliet, WileyIndia, 3rd Edition, 2010. 3. “Foundations of Software Testing ", Aditya Mathur, Pearson, 2008. 4. “Software Testing, A Craftsman’s Approach ", Paul C. Jorgensen, Auerbach, 2008. 6:IEEE SWEBOK, PMBOK, BABOK and Other Sources from Internet 5. “Software Testing – Principles and Practices”, Srinivasan Desikan and Gopalaswamy Ramesh, Pearson, 2006.
Course Outcome	• Relate to the challenges of Software Development and Software Engineering as a methodical approach for development. Use state-of-the-art tools and techniques for large-scale software systems development. • Use Software project management ideas and apply suitable architecture and design approaches for software development • Implement the major software development methods in practical projects including CI/CD, various devOps tools etc • Follow industry accepted documentation and review processes to develop a high quality SW project.

Course Code	UE24CS342AA1	Course Title	Advanced Algorithms				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 1				
Tools/Languages	C or C++ Programming Languages	Desirable Knowledge	Design and Analysis of Algorithms.				
Prelude	Algorithm Design and Analysis is a fundamental and important part of computer science. The course on Advanced Algorithms introduces the learner to advanced techniques for design and analysis of algorithms and explores a variety of applications. This course requires the student to have a desirable knowledge of Design and Analysis of Algorithm.						
Course Objectives:	• Enable the learner with basics of Amortized Complexity Analysis of Data Structures. • Empower the learner with String Matching/ Prediction Algorithms and hone the problem-solving skills of the learner by introducing them to Flow Networks, Bipartite Matching and DFT. • Introduce the learner to Number Theoretic Algorithms. • Enable the learner with design strategy of Dynamic Programming, Randomized Algorithms and Approximation Algorithms.						
Course Contents	Unit 1: Basics of Complexity Asymptotic Notations, Standard notations and common functions, Recurrences and Solution of Recurrence equations- The Substitution method, the Recurrence tree method, the Master method, Amortized Complexity Analysis: Aggregate, Accounting and Potential Methods.NP-Completeness, NP Reduction. Computational efficiency, optimization techniques, memory usage, cache behaviour, and overall system-level performance 14 Hours Unit 2: String Matching Algorithms, Suffix Trees and Maximum Flow, Naive String Match, Rabin_Karp, String matching with Finite State Automata, and Knuth_Morris_Pratt Algorithms, Suffix Arrays, Suffix Trees - Applications of Suffix Trees, Regular Expression Searches Using Suffix Trees, Flow Networks, The Ford-Fulkerson method, The Edmonds-Karp algorithm, Maximum Bi-Partite Matching. 14 Hours Unit 3: Polynomials and FFT &Number-Theoretic Algorithms Polynomials and FFT: Representation of Polynomials, Efficient Polynomial Multiplication, DFT and FFT, Efficient Implementation of FFT. Number-Theoretic Algorithms: Elementary notions; GCD, Modular Arithmetic, Solving modular linear equations, Modular Inverse, The Chinese remainder theorem, Powers of an element; RSA cryptosystem; Primality testing; Integer factorization. 14 Hours Unit 4: Dynamic Programming Randomized Algorithms and Approximation Algorithms Elements of Dynamic Programming, Problems - Coin- Row, Rod-Cutting, Matrix-Chain Multiplication, Longest Common Subsequence. Randomized Algorithms: Hiring Problem, Indicator random variables, Approximation Algorithm: Vertex Cover Problem, TSP, The Subset Sum Problem. 14 Hours						
Text Book(s):	1. "Introduction to Algorithms", T H Cormen, C E Leiserson, R L Rivest and C Stein, MIT Press, 4th Edition, 2022.						
Reference Book(s):	2. "The Algorithm Manual", Steven Skiena, Springer. ISBN: 9788184898651, 2nd Edition, Springer, 2008 3. "Randomized Algorithms", R Motwani and P Raghavan, Cambridge University Press, 2013.						

	4. Communication Complexity (for Algorithm Designers) , Tim Roughgarden 5. Kraken: Memory-Efficient Continual Learning for Large-Scale Real-Time Recommendations, Minhui Xie, Kai Ren†*
Course Outcome	• Perform Amortized Analysis of complex Data Structures. • Apply String Matching Algorithms to solve string related problems and implement Max Flow and FFT Algorithms. • Apply Number Theoretic concepts in designing Cryptographic Algorithms. • Solve complex problems using Dynamic Programming, Randomized Algorithms and Approximate Algorithms.

Course Code	UE24CS342AA2	Course Title	Data Analytics				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 1				
Tools/Languages	R ,Python and MS Excel tools	Desirable Knowledge	Mathematics for Computer Science				
Prelude	The course explores the data analytics lifecycle: question formulation, data collection and cleaning, exploratory, analysis, visualization, statistical inference, prediction, and decision-making. Focuses on building analytical models using key principles and techniques. This course requires the student to have a desirable knowledge of Statistics for Data Science.						
Course Objectives:	- Assess a dataset for the type of data, inter-relationships between features and interpret the data meaningfully - Understand the assumptions that underlie regression models and determine the appropriate model and parameters for a problem - Recognize patterns such as trend and seasonality in a time series data and design appropriate forecasting models. - Understand the nuances of recommender systems, build and validate appropriate models for recommendations.						
Course Contents	Unit 1: Exploratory data analysis, ANOVA and correlation analysis Introduction to business analytics, Data cleaning and dimensionality reduction, ANOVA, correlation analysis, hidden variables. 14 hours Unit 2: Regression Analysis Simple linear regression (alternative computational approaches and an emphasis on model diagnostics and validation), multiple linear regression (alternative computational approaches and an emphasis on model interpretation, diagnostics and validation) and variations (such as ridge regression and lasso regression), an introduction to multivariate and nonlinear regression. 14 Hours. Unit 3: Time Series Analysis and Markov Chains Time series data components, moving average and exponential smoothing methods, regression models for forecasting, AR model identification using ACF/ PACF, concept of stationarity – its importance, testing for stationarity and converting a non-stationary signal to stationary, ARMA and ARIMA modelling, Discrete Markov chains, Markov chains with Absorbing States, expected duration to reach a state from other states, computing the retention probability and customer lifetime value, confounding variables. 14 Hours Unit 4: Recommender Systems Collaborative filtering, knowledge-based filtering: constraint-based and case-based filtering, a brief introduction to text classification/ clustering for content-based filtering, mining of association rules and evaluation of recommender systems, A/B testing. 14 Hours						
Text Book(s):	1. “Business Analytics, The Science of Data-Driven Decision Making”, U. Dinesh Kumar, Wiley, second edition, 2022 2. “Recommender Systems: The Textbook”, Charu C. Agarwal, Springer 2016.						
Reference Book(s):	1. Introduction to Statistical Learning (ISL) Authors: Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani Publisher: Springer (2nd Edition, 2021) 2. Data Mining: Concepts and Techniques by Jiawei Han, Micheline Kamber and Jianei, The Morgan Kaufmann Series in Data Management Systems, Elsevier publications, 3rd Edition, 2012.						

	3. The Elements of Statistical Learning, Trevor Friedman, Robert Tibshirani and Jerome Hastie, Data Mining, Inference and Prediction, Springer 2001. 4. Practical Data Science with R, Nina Zumel and John Mount, Manning Publications, 2014.
Course Outcome	• Perform exploratory data analysis on a given set of data and identify/ interpret correlation between features. • Narrow down a subset of regression techniques to model data and evaluate the efficacy of these models. • Analyse a time series and build appropriate time series models for prediction and evaluate them. • Analyse data to infer underlying patterns, select the appropriate techniques and formulate recommendations.

Course Code	UE24CS342AA3	Course Title	Internet of Things				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	TyAApe of Course	Elective				
Tools/Languages		Desirable Knowledge	Computer Networks				
Prelude	The Internet of Things (IoT) is no longer just about connecting devices—it is about building intelligent, autonomous, and secure cyber-physical systems. From smart homes and healthcare to industrial automation and agriculture, IoT systems operate at the intersection of embedded systems, networking, cloud computing, and security. This course provides a full-stack understanding of IoT, starting from the “things” (sensors, actuators, microcontrollers) and progressing through communication protocols, edge intelligence, cloud orchestration, and security fundamentals.						
Course Objectives:	- Understand the architecture, components, and functional stack of IoT systems. - Develop embedded applications using microcontrollers and standard interfacing protocols. - Analyze and apply IoT communication protocols for different use cases. - Design data pipelines with edge computing and cloud integration for IoT applications.						
Course Contents	Unit 1 : Foundations – The "Things" and Embedded Logic. Introduction to IoT, IoT Traffic Model, IoT Connectivity, IoT Architecture Basics: Perception (sensors), Network (gateways), and Application layers, IoTWF Model, The Core IoT Functional Stack. The Things in IoT - Sensors, Actuators and smart Objects: Sensors - Its Types, Sensors & Its Performance Metrics, Sensor Selection, Actuators and Its Types MEMS, Smart Objects. Microcontrollers (MCU): Deep dive into ESP32 and ARM Cortex-A, ARM Cortex-M architectures. Embedded C & Python: Programming GPIOs, Interfacing (I2C, SPI, UART), and Interrupt handling. 14 Hours Unit 2 : The Network – Connectivity & Protocols Short-Range Protocols: BLE (Bluetooth Low Energy), Zigbee, and the new Matter/Thread standards (essential for 2026). Long-Range Protocols: LoRaWAN and NB-IoT for industrial/agricultural use cases. Application Protocols: MQTT (Publish/Subscribe), CoAP, and HTTP/WebSockets. Lightweight Encryption: Standard Cryptography vs. Lightweight Introduction to ASCON (NIST standard) and how it differs from heavy AES. Network Obfuscation: Hiding IoT devices behind gateways and using non-standard ports. 14 Hours Unit 3 : Edge computing & Cloud Orchestration for IoT Introduction to IoT Analytics , IoT Data Pipelines & AI Integration, Edge Computing: Introduction to TinyML—running machine learning models on microcontrollers. Data Pipelines: Time-series databases (InfluxDB) and real-time visualization (Grafana). Cloud Integration: Connecting to AWS IoT Core or Azure IoT Hub for device management. Predictive Maintenance: Using sensor data to predict hardware failure before it happens. 14 Hours Unit 4 : Introduction to IoT Security The IoT Attack Surface: Identifying vulnerabilities at each layer (Physical, Network, and Cloud). Common Vulnerabilities (OWASP IoT Top 10): A beginner-friendly look at weak passwords, insecure network services, and lack of update mechanisms. Zero Trust Architecture: Traditional network security vs Zero Trust, Limitations of perimeter-based security, Zero Trust principle: “Never Trust, Always Verify”, Types of device authentication methods.						

	Identity-based security vs network-based security. Firmware Hacking : Using Binwalk to look inside a firmware file for secrets. X.509 certificates - Introduction to X.509 certificate, Structure and components of digital certificates, Public key infrastructure (PKI) basics, Digital signatures and trust chain. 14 Hours
Laboratory	Lab 1: Sensor Interfacing & Data Acquisition Lab 2: Communication Protocols Lab 3: Edge Processing & TinyML Lab 4: Cloud Integration & Visualization Lab 5: IoT Security & Firmware Analysis
Text Book(s):	1. IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things * Authors: David Hanes, Gonzalo Salgueiro, Patrick Grossetete, Robert Barton, and Jerome Henry (Cisco Press) 2. Designing the Internet of Things * Authors: Adrian McEwen and Hakim Cassimally.
Reference Book(s):	1. Advances in the Internet of Things: Challenges, Solutions, and Emerging Technologies 2. Internet of Things Security Attacks, Tools, Techniques and Challenges Preeti Mishra, Senthil Kumar Jagatheesa perumal,
Course Outcome	● Explain IoT architecture, components, and system-level design. ● Develop embedded IoT applications using microcontrollers and communication interfaces. ● Select and implement appropriate IoT communication protocols for given scenarios. ● Design IoT data pipelines with edge processing and cloud integration. ● Analyze IoT security threats and apply basic mechanisms for secure system design

Course Code	UE24CS342AA4	Course Title	Applied Cryptography				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective				
Tools/Languages		Desirable Knowledge					
Prelude	Applied Cryptography focuses on the design, analysis, and implementation of cryptographic techniques used to secure modern digital systems. This course covers the complete spectrum—from classical ciphers and mathematical foundations to modern symmetric and asymmetric algorithms, cryptographic hash functions, and real-world security applications. It also introduces practical attack models, hardware security concepts, and emerging areas such as post-quantum cryptography. The course emphasizes both theoretical understanding and practical security thinking, enabling students to analyze and build secure systems.						
Course Objectives:	- Understand the fundamental principles and mathematical foundations of cryptography. - Analyze and implement symmetric and asymmetric cryptographic algorithms. - Apply cryptographic techniques for data confidentiality, integrity, and authentication. - Evaluate cryptographic protocols and identify potential vulnerabilities and attacks. - Gain exposure to real-world applications, hardware security, and emerging cryptographic trends.						
Course Contents	Unit 1: Foundations, Number Theory & Classical Ciphers. Cryptography Basics: Security attacks (Passive vs. Active), Services, and Mechanisms. The CIA Triad, Threat models, Attack types: MITM, replay, side-channel, Cryptography vs security Mathematical Toolkit I: Modular arithmetic, GCD (Euclidean & Extended), Euler's Totient Function (ϕ), and Fermat's Little Theorem. Classical Techniques: Substitution Ciphers : Caesar, Monoalphabetic and Polyalphabetic :Vigenere, Playfair, and Hill Cipher. Transposition Ciphers: Rail Fence, Row-Columnar, and Double Columnar. Security Models: Semantic Security (IND-CPA, IND-CCA1, IND-CCA2). Distinguishing games and the "Random Oracle Model". Shannon's Theorem, Confusion and Diffusion, Entropy, and the concept of "Perfect Secrecy" (One-Time Pad). 14 hours Unit 2 : Symmetric Key Cryptography & Modern Standards: Block Ciphers: Introduction, Structure of Feistel Networks ,Substitution-Permutation Networks (SPN), Feistel Networks Vs Substitution-Permutation Networks DES/3DES: Design and why they are deprecated. AES: Introduction to Galois field, Introduction to AES, Detailed rounds (SubBytes, ShiftRows, MixColumns, KeyExpansion). Modes of Operation: Electronic Codebook (ECB), Cipher Block Chaining (CBC), and Modern Authenticated Encryption (GCM & CCM). Randomness : PRNGs vs. TRNGs and their role in key generation. 14 hours Attack on Block ciphers : Differential & Linear Cryptanalysis, Padding Oracle Attacks, Side-Channel Attacks. 14 hours Unit 3: Asymmetric Cryptography & Data Integrity The Public Key Paradigm: Symmetric vs. Asymmetric trade-offs. Mathematical Toolkit II:, Prime number generation (Miller-Rabin), Chinese Remainder Theorem ,						

	Discrete Logarithm Problem (DLP), and Finite Fields (GF(2n)). Public Key Algorithms: RSA: Key generation, encryption, and OAEP padding. Diffie-Hellman: Key exchange and the Man-in-the-Middle (MITM) vulnerability. Elliptic Curve Cryptography (ECC): Point addition, doubling, and the security benefits of smaller key sizes (ECDH, ECDSA). Digital Signatures : Concept and security goals , RSA signatures , ECC-based signatures, PKI Infrastructure. Cryptographic Hashes: Properties (Pre-image, Collision resistance), Merkle-Damgård construction, SHA-256 14 hours Unit 4: Applied Crypto, Attacks & Future Trends Entity Authentication: Passwords (Salting/Hashing), Challenge-Response, and Zero-Knowledge Proofs (ZKP) basics. Blockchain Fundamentals (Cryptographic Perspective): Use of cryptographic hash functions for data integrity, block linking, and immutability; role of digital signatures in transaction authentication, ownership, and non-repudiation. Intro to Hardware Security: The "Physical Attacker" model., Side-Channel Attacks: How systems leak secrets via Timing and Power Consumption (Introduction to DPA). Hardware Trust: TPMs (Trusted Platform Modules) and PUFs (Physically Unclonable Functions). Post-Quantum Cryptography (PQC): Why Quantum computers threaten RSA/ECC; introduction to Lattice-based crypto (Kyber/ML-KEM). 14 hours
Laboratory	Lab 1: Classical Cipher Implementation & Cryptanalysis Lab 2: AES & Modes of Operation Lab 3: Public Key Cryptography & Key Exchange Attack Lab 4: Hash Functions & Digital Signatures Lab 5: OpenSSL & TLS Handshake Analysis Case Study , CTF
Text Book(s):	1. Introduction to Modern Cryptography (3rd or 4th Edition) Authors: Jonathan Katz and Yehuda Lindell. 2. Handbook of Applied Cryptography - Alfred J. Menezes , Paul C. van Oorschot 3. Understanding Cryptography – Paar & Pelzl
Reference Book(s):	1.Serious Cryptography - Jean-Philippe Aumasson
Course Outcome	● Explain cryptographic concepts, security models, and underlying mathematical principles. ● Implement and analyze symmetric and asymmetric encryption algorithms. ● Apply cryptographic primitives for secure communication and data protection. ● Evaluate cryptographic systems against common attacks and vulnerabilities. ● Interpret emerging trends such as hardware security and post-quantum cryptography.

Course Code	UE24CS342AA5	Course Title	Augmented and Virtual Reality				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective				
Tools/Languages	Co-Pilot,	Desirable Knowledge	C, Python, C++ Programming				
Prelude	This course presents an introduction to virtual and augmented reality technologies, with an emphasis on designing and developing interactive virtual and augmented reality experiences using blender and Unity3D.						
Course Objectives:	● Introduce the use of geometric transformations on graphics objects, their application in composite form and its implementation. ● Impart the basics of computer graphics and Introduce graphics programming using OpenGL and Graphics Pipeline. ● Introduce Virtual and Augmented Reality essentials and understanding human physiological aspects with respect to virtual reality applications and user interfaces. ● Understanding of use of Artificial Intelligence in the field of Virtual and Augmented Reality and applying the concepts to create creative virtual experiences.						
Course Contents	Unit 1: Geometric Objects and Transformations Scalars, Points and Vectors, Three-Dimensional Primitives, Coordinate Systems and Frames, Modelling a Coloured Cube, Overview of 2D Transformations: Rotation, Translation and Scaling, Affine transformations, Transformation in Homogeneous Coordinates, Concatenation of Transformations, OpenGL Transformation Matrices, Interfaces to Three Dimensional Applications, Quaternion's 14 Hours Unit 2: Graphical System and Programming and 3D Modelling. The Programmer's Interface, Graphics Architectures, Programmable Pipelines. Graphics Programming: Programming Two Dimensional Applications. The OpenGL: The OpenGL API, Primitives and Attributes, Colour, Viewing, Control Functions, Polygons, Viewing, Control Functions, the gasket Program, Polygon and Recursion, The Three-dimensional gasket, Creating Scene on ThreeJS, Working with Three.Js Material and Geometries, Animations, Adding Physics, Working with Blender, Three.JS with React and WebXR 14 Hours Unit 3: Augmented and Virtual Reality. Introduction to Augmented Reality: Definition and Scope, A Brief History, Examples, Requirements and Characteristics: Methods of Augmentation, Spatial Display Models, Visual Display, Stationary Tracking Systems, Mobile Sensors. Introduction: What is Virtual Reality, Modern VR Experience. Bird's Eye View: hardware, Software. Eye movement and its implications or VR. Tracking: 2D and 3D orientation, Tracking Position and Orientation, Tracking Attached bodies. 14 Hours Unit 4: IO modalities, AI, and Behaviour in VR Computer Vision and Augmented Reality; marker tracking, Multiple-Camera Infrared Tracking, Natural Feature Tracking by Detection, Incremental Tracking, Simultaneous Localization and Mapping, Outdoor Tracking, Interaction: Output and input modalities, Haptic interaction and Multimodal interaction. 3D Scanning of environments. Reactive AI: Adaptability, Complexity and Universality, Feasibility, More Intelligence in the System: Deliberative AI, Reinforcement learning through interaction, Imitation Learning through human demonstration. 14 Hours						

Text Book(s):	1. “Interactive Computer Graphics - A top-down approach with shader-based OpenGL”, Edward Angel and Dave Shreiner, Pearson Education, Sixth edition, 2012. 2. Steven M. LaValle. Virtual Reality. Cambridge University Press, 2017, http://vr.cs.uiuc.edu/ (Links to an external site.) (Available online for free) 3. Creating Augmented and Virtual Realities, by Erin Pangilinan, Steve Lukas, Vasanth Mohan, PUBLISHED BY:O'Reilly Media, Inc.PUBLICATION DATE:March 2019
Reference Book(s):	1. “Interactive Computer Graphics: A Top -Down Approach with WebGL”, Edward Angel, Pearson Education, 7th Edition, 2015. 2. Unity Game Development in 24 Hours , Geig, Mike. Sams Teach Yourself . Pearson Education, 2014. 3. “OpenGL Programming Guide”: Mason Woo, Jackie Neider, Tom Davis, Dave Shrenier: 3rd Edition, openGL version 1.2, Addison Wesley, 1999. 4. Blender 3D Basic, Gordon Fisher, PACKT Publishing, 2nd Edition (Note: For working with recent version the course material for UNIT 4 can be substituted with appropriate web content. 5. D. Schmalstieg and T. Höllerer. Augmented Reality: Principles and Practice. Addison-Wesley, Boston, 2016, ISBN-13 978-0-32-188357-5
Course Outcome	• Apply techniques and methods of augmenting virtual objects in real space. • Demonstrate the fundamentals of computer graphics and display pipeline systems and use OpenGL for complex 3D graphical visualization and demonstrate its applications. • Apply techniques and tool to design a immersive virtual reality experience and graphics in greater depth to more complex aspects of Image Processing, Tracking and Human Computer Interfaces, etc. • Apply AI models and methods for creating dynamic interactive and adaptable virtual spaces.

Course Code	UE24CS342AA6	Course Title	UI/UX design				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 1				
Tools/Languages	Figma	Desirable Knowledge	-				
Prelude	This course introduces the fundamental concepts of Human Computer Interaction (HCI) and UI/UX Design, focusing on designing interactive, user friendly, and accessible digital systems. It covers user centered design principles, interaction design, prototyping, usability evaluation, accessibility, and emerging trends in HCI. The course enables students to understand user behavior, create effective user interfaces, and evaluate user experiences using modern design tools and techniques.						
Course Objectives:	- Explain core HCI concepts, including usability, user experience (UX), and interaction design principles. - Apply user-centered design (UCD) methodologies to prototype and evaluate interactive systems by exploring industry-standard design guidelines, responsive design practices, modern UI/UX design tool (Figma) - Conduct usability testing and heuristic evaluations to assess interface effectiveness via UX evaluation methods, accessibility standards, data-driven analysis techniques, and ethical design practices. - Analyze advanced HCI concepts such as AI-driven interfaces, multimodal interaction, and ethical issues in human-centered computing.						
Course Contents	Unit 1: Foundations of HCI and User Experience Design Definition, Importance, and Goals of HCI, Human Factors in HCI: Perception, Cognition, Memory, Mental Models, Interaction Models: Gulf of Execution & Evaluation, Norman’s Model, User-Centered Design (UCD) Process UX Research Methods: Interviews, Surveys, Observation UX Synthesis Techniques: Affinity Diagramming, Personas, Scenarios, Empathy Mapping, User Journey Mapping, Task Analysis Information Architecture: Navigation Design, Content Hierarchy Design Principles: Affordances, Feedback, Consistency, Universal Design Principles 14 Hours Unit 2: Interaction Design and Prototyping Interaction Styles: Command Line, GUI, Voice Interfaces, Gesture-Based Interfaces, Design Thinking and Ideation Techniques Visual Interface Design Principles: Typography, Color Theory, Layout and Spacing, Visual Hierarchy Wireframing and Prototyping: Low-Fidelity Prototypes, High-Fidelity Prototypes, Figma as a Prototyping Tool, Interface Design Guidelines: Google Material Design Guidelines, Apple Human Interface Guidelines Responsive and Mobile-First Design Case Studies in Interaction and UI Design 14 Hours Unit 3: Usability Evaluation, Accessibility, and UX Testing Usability Principles - Learnability, Efficiency, Memorability, Error Prevention, User Satisfaction Evaluation Methods - Heuristic Evaluation, Cognitive Walkthrough, User Testing, Think-Aloud Protocol, A/B Testing UX Metrics and Analytics: Task Completion Rate, Error Rate, Time on Task, SUS (System Usability Scale) Web & Mobile Accessibility: Web Content Accessibility Guidelines (WCAG), Inclusive Design, Assistive Technologies Analyzing Usability Data: Metrics, Surveys, Interviews, Feedback Analysis Ethical Design and Dark Patterns in UX						

	14 Hours Unit 4: Emerging Trends in UI/UX & Future of HCI AI in HCI: Chatbots, Recommender Systems, Personalized User Experience. Conversational UI and Voice User Interfaces (VUI) Virtual & Augmented Reality (VR/AR) Interfaces, Brain-Computer Interfaces (BCI) & Wearable Computing Cross-Platform and Cross-Cultural UI/UX Design Ethical & Social Implications of HCI Emotional Design and Persuasive Design, Design for Globalization and Accessibility Challenges, Future Trends in Human-Centered Computing 14 Hours
Text Book(s):	1. Interaction Design: Beyond Human-Computer Interaction, Helen Sharp, Jennifer Preece, Yvonne Rogers, 5th Edition (2023), Wiley 2. "Human Computer Interaction", Dix A., Finlay J, Abowd G. D. and Beale R., 3rd Edition, Pearson Education, 2005.
Reference Book(s):	1. "Designing the User Interface: Strategies for Effective Human-Computer Interaction", Ben Shneiderman, Catherine Plaisant, 6th Edition, Pearson Education, 2016. 2. "Don't Make Me Think, Revisited: A Common Sense Approach to Web and Mobile Usability", Steve Krug(Author), 3rd Edition, 2014 (e-book) 3. Measuring the User Experience by Thomas Tullis and William Albert, Morgan Kaufmann, Second edition (2013)
Course Outcome	• Understand the fundamental principles of Human-Computer Interaction (HCI) and its importance in designing user-friendly systems. • Analyze user needs and apply interaction design methodologies to develop effective interfaces. • Evaluate usability and accessibility aspects in interactive systems using various HCI evaluation techniques. • Explore emerging trends in HCI, including AI-driven interfaces, VR/AR, and ethical considerations in interaction design.

Course Code	UE24CS342AA7	Course Title	Topics in Wireless and 5G Networks				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	1	4	4
Semester	5	Type of Course	Elective Course				
Tools/Languages	Cisco Packet Tracer, OMNET++	Desirable Knowledge	Computer Networks				
Prelude	This course offers an in-depth understanding of wireless communication systems, covering WLAN, WPAN, and cellular networks from 1G to 5G. Students will explore key topics like wireless access methods, modulation schemes, MIMO, LTE, and 5G network architecture. The course also delves into emerging areas such as Network slicing, OpenRAN, SDN/NFV, 5G security, and applications in IoT, AI/ML, and digital twin. With a forward-looking perspective on 6G, the course balances theoretical knowledge with practical skills, preparing students for careers in wireless communication and next-generation mobile networks.						
Course Objectives:	- Understand fundamental wireless technologies, IEEE 802.11 standards, and access methods. - Learn modulation techniques, cellular architecture, and evolution of mobile communication 1G to 5G. - Explore 4G and 5G architecture, protocol stack, authentication, and signaling mechanisms. - Introduce 5G applications, security, SDN/NFV, and emerging trends toward 6G.						
Course Contents	Unit 1: Overview of Wireless communication Introduction, Different Wireless Networks – WWAN, WBAN, WPAN, WLAN , Different Wireless Standards of 802.11, Wireless Local Area Networks: IEEE 802.11, 802.11 Frame Format, Basic Access Methods – CSMA/CA, ALOHA, Distribution Coordinating Function (DCF), DIFS, SIFS, Point Coordination Function (PCF), Multiple Access for Wireless Systems – FDMA, TDMA, CDMA, OFDMA, 802.11 Physical Media, WPAN Technologies: Bluetooth, Bluetooth Protocol Stack, Bluetooth Low Energy, NFC, LORAWAN, WiMAX, WiFi-6 & WiFi-7, Adhoc Routing Protocols – Overview, MANET & VANET, Wireless networking and security issues in 802.11 – Static filtering based on MAC address, Wireless Security Protocols - WPA2/WPA3, 802.1X/EAP, Wireless Link Characteristics, RFID: Concept, frequency band, classification of RFID tags, applications. 14 hours Unit 2: Advanced Wireless Technologies and Cellular Standards Wireless Modulation Schemes – Amplitude Modulation (AM), Phase Modulation (PM), Frequency Modulation (FM), Binary Phase-Shift Keying (BPSK), Quadrature Phase-Shift Keying (QPSK), Frequency Spectrum, Cellular Concept –basic mobile communication, Mobile systems concept: mobile terminal, base station (BS), mobile switching centre (MSC), Frequency Reuse, Techniques: Multiple-Input and Multiple-Output (MIMO), mmWave, massive MIMO, Small Cells, Handoff, Mobile Radio standards – AMPS, GPRS, GSM, UMTS, CDMA 2000, Evolution of Wireless Network generations (1G, 2G, 3G, 4G, 5G), How does a mobile connectivity work? Global System for Mobile Communication (GSM), 3G UMTS – architecture, Need for 3G, 4G and 5G technologies – architectural difference 14 hours Unit 3: Fundamentals of Cellular System 4G – LTE – EPC architecture, LTE protocol stack, UMTS vs LTE, Configuring LTE Control and Data Plane, Authentication Key Agreement in 4G LTE, Introduction to 5G, 5G Architecture, features, specifications and other procedures, 5G-SA/NSA, 5G vs 4G architectural differences, Simple call registration flow in 5G – AKA Authentication, Session establishment in 5G call flow, Mobility in 5G, Basics of Voice over IP (SIP, RTP) 16 hours Unit 4 : 5G Applications and Evolution to 6G OpenRAN, Network Slicing, Long Range Communication- Satellite Communication, VoLTE, IMS, SDN, NFV, Private 5G networks, Application use cases: Digital Twin, IoT, AI/ML, AI/ML in RAN, Satellite Communications - NTN, 5G Security and Beyond, Mobile Forensics, 5G Security, Introduction to 6G						

	12 hours
Laboratory	1. Bluetooth Communication using Cisco Packet Tracer 2. Wireless Security Implementation 3. RFID 4. Implementation of Cellular topology using Cisco Packet Tracer 5. 5G call setup using OMNET++
Text Book(s):	Disclaimer: There is no fixed text book. Lectures will be drawn from several sources (books, articles, papers) as most available texts do not cover all of the course material. The text book listed covers some of the topics that are relevant to the material presented. 1. “Computer Networking: A Top Down Approach”, James Kurose and Keith Ross, Pearson, 8th Edition
Reference Book(s):	Books, Articles and Papers
Course Outcome	At the end of the course, the student will be able to: ● Explain wireless standards, access protocols, and security in wireless networks. ● Learn cellular standards based on different wireless technologies ● Analyze 4G and 5G network architectures and key protocols. ● Evaluate 5G applications, network slicing, and the transition to 6G technologies.

Course Code	UE24CS343AA8	Course Title	Quantum Computing				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 1				
Tools/Languages	PennyLane, Qiskit Machine Learning	Desirable Knowledge	-				
Prelude	This course is meant to give an overview of the field of quantum technologies and make the students familiar with the state-of-the-art in all four verticals. The emphasis is not on depth in this course, but on covering the exciting aspects of the field.						
Course Objectives:	- The general physical principles of realising qubits for computation - The various hardware implementations of qubits for computation - The basic ideas of quantum sensing - The applications of quantum sensing						
Course Contents	Unit-1: Logic of Qubits Quantum bit, Super Position, Measurement, Bloch Sphere Mapping, Quantum Gates & Circuits. Quantum States, Inner Products, Quantum Gates, Outer Products 14 hours Unit 2: Multiple Quantum bits and Quantum Programming States and Measurement, Entanglement, Quantum Gates and Adders, Universal Quantum Gates, Quantum Error Correction. Quantum programming using Qiskit and other programming languages. 14 hours Unit 3: Entanglement and Quantum Protocols Measurements, Bell Inequalities, Monogamy of Entanglement, Superdense coding, Quantum Teleportation, Quantum Key Distribution. . 14 hours Unit 4 Quantum Algorithms & Applications Deutsch & Deutsch-Jozsa, Simon's Algorithm, Grover's Algorithm, Shor's Algorithm 14 hours						
Text Book(s):	"Introduction to Classical and Quantum Computing" by Thomas G. Wong, Published by Rooted Grove, 2022.						
Reference Book(s):	"Quantum Computing for Computer Scientists" by Noson S. Yanofsky & Mirco A. Mannucci, Cambridge University press, 2008.						
Course Outcome	- The various hardware implementations of qubits for computation - Calculate the state of a quantum system using matrix mechanics and Dirac notation. - Execute quantum teleportation and superdense coding protocols. - Evaluate the efficiency of Grover's and Deutsch-Jozsa algorithms compared to classical limits.						

Course Code	UE24CS343AA9	Course Title	Visualization of Data, Understanding and Interpretation				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 1				
Tools/Languages	Matplotlib, Seaborn, Plotly, Tableau / Power BI	Desirable Knowledge	--				
Prelude	This course introduces the principles and practices of modern data visualization, interpretation, and AI-assisted analytics for effective communication of insights. It covers foundational and advanced visualization techniques, storytelling with data, interactive dashboards, and ethical visualization practices						
Course Objectives:	- Understand the principles of data visualization, visual perception, cognition, and effective visual communication. - Apply appropriate visualization techniques for exploratory data analysis, statistical interpretation, and AI-driven analytics. - Design static, interactive, multimedia, and narrative visualizations using modern visualization tools and frameworks. - Analyze multidimensional and machine learning data using advanced visualization and AI-assisted analytical techniques. - Evaluate visualization effectiveness, ethical considerations, uncertainty, and reliability in AI-assisted visualization systems.						
Course Contents	UNIT 1: Foundations of Visualization, Data Interpretation Introduction to data visualization, Role of Visualization in AI-driven Analytics, Visual encoding (marks & channels), Human perception & cognition, Gestalt principles, Color theory and design, Static vs Interactive Visualizations, Data preprocessing & Exploratory Data Analysis (EDA), Identifying trends, correlations and patterns, Multivariate visualization, Statistical thinking, Perceptual effectiveness of charts, Data-ink ratio and chart junk, Misleading visualizations and ethical concerns. 14 Hours UNIT 2: Advanced Visualization & Story Telling. Advanced visualizations: Networks, node-link diagrams, treemaps, saliency visualization. Dimensionality reduction visualization: PCA, t-SNE, UMAP. Geospatial visualization, Feature importance visualization, Uncertainty visualization, Interactive dashboards, Coordinated multiple views. Storytelling with data, Narrative visualization, AI-generated reports and summaries, Audience-centric design, Annotation & highlighting, Ethical visualization and accessibility. 14 Hours UNIT 3: Multimedia and Interactive Visualization Multimedia visualization concepts, Image, video and audio visualization, Scientific visualization fundamentals, Scalar and vector field visualization, Volume visualization, Medical and simulation data visualization, 3D visualization and rendering basics, Animation and motion visualization, Temporal visualization, Graph drawing and network layout techniques (force-directed and hierarchical layouts), Visualization pipelines and rendering process, Interactive visualization techniques: brushing, linking, filtering, zooming and details-on-demand, Visual analytics and sensemaking, Visualization evaluation methods, Usability testing and cognitive load analysis, Dashboard design principles and accessibility in visualization systems. 14 Hours UNIT 4: AI-Assisted Visualization Automated visualization tools: Auto chart recommendation systems, Natural Language to Visualization (NL2Vis), conversational visualization systems, text-to-chart generation, Automated generation of dashboards, Data summarization and interpretation using LLMs for EDA insights, Introduction to human-						

	in-the-loop AI in visualization, AI-assisted pattern recognition, Anomaly detection, Bias in AI-driven insights, Model drift visualization, Uncertainty visualization in AI models, Visualization of ML outputs: predictions, probabilities and confidence measures, Explainable AI (XAI) visualizations, Monitoring AI systems using visual analytics, Misuse and limitations of automated insights. 14 Hours
Text Book(s):	1. C. O. Wilke, <i>Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures</i>. Sebastopol, CA, USA: O'Reilly Media, 2019. 2. C. N. Knaflic, <i>Storytelling with Data: A Data Visualization Guide for Business Professionals</i>. Hoboken, NJ, USA: John Wiley & Sons, 2015. 3. C. Ware, <i>Information Visualization: Perception for Design</i>, 4th ed. Burlington, MA, USA: Morgan Kaufmann (Elsevier), 2020.
Reference Book(s):	1. T. Munzner, <i>Visualization Analysis and Design</i>. Boca Raton, FL, USA: CRC Press, 2014. 2. S. Few, <i>Information Dashboard Design: Displaying Data for At-a-Glance Monitoring</i>, 2nd ed. Burlingame, CA, USA: Analytics Press, 2013. 3. A. Cairo, <i>The Art of Insight: How Great Visualization Designers Think</i>. Hoboken, NJ, USA: John Wiley & Sons, 2023. 4. E. R. Tufte, <i>The Visual Display of Quantitative Information</i>, 2nd ed. Cheshire, CT, USA: Graphics Press, 2001. 5. J. Dykes, A. Slingsby, and J. Wood, <i>Hands-On Data Visualization</i>. Birmingham, U.K.: Packt Publishing, 2018.
Course Outcome	• Explain the fundamentals of visualization, visual encoding, human perception, cognition, and statistical thinking for data interpretation. • Perform exploratory data analysis and develop effective static and interactive visualizations to identify trends, correlations, and patterns. • Apply advanced visualization techniques for multidimensional, temporal, geospatial, multimedia, and scientific data visualization. • Design interactive dashboards, narrative visualizations, and AI-assisted visual analytics systems for decision-making and communication. • Evaluate visualization quality, ethical issues, uncertainty, explainability, and the reliability of AI-generated insights in real-world applications.

Course Code	UE24CS343AB1	Course Title	Image Processing				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective - 2				
Tools/Languages	MatLab, Python	Desirable Knowledge	Design and Analysis of Algorithms.				
Prelude	Digital Image Processing deals with processing images that are digital in nature. Improving the quality of images for human perception and understanding, extracting useful information for decision making and efficient storage are some of the driving factors behind image processing techniques/algorithms. The course on Digital Image Processing introduces the learner to various image processing techniques, algorithms and their applications.						
Course Objectives:	- Understand the principles underlying imaging and assess the quality of an image based on its visual content, histogram and statistical measures - Gain an insight to image transforms and enhancement techniques in the frequency domains - Be able to apply morphological operations and segmentation techniques on monochrome and color images - Learn different feature extraction techniques for classification						
Course Contents	Unit 1: Introduction and Enhancement in the Spatial Domain What is Digital Image Processing, examples of fields that use DIP, Fundamental Steps in Digital Image Processing, elements of visual Perception, Basic Concepts in Sampling and Quantization, Representing Digital Images, Spatial and Gray-level Resolution, Zooming and Shrinking Digital Images, Some Basic Relationships Between Pixels, Linear and Nonlinear Operations. Image Enhancement in the Spatial Domain: Some Basic Gray-level Transformations, Histogram Processing, Enhancement Using Arithmetic/Logic Operations, Basics of Spatial Filtering, Smoothing Spatial Filters, Sharpening Spatial Filters, and Combining Spatial Enhancement Methods. 14 Hours Unit 2: Image Enhancement in the Frequency Domain, Transforms and Compression Frequency Domain: Introduction to the Fourier Transform and the Frequency Domain, Smoothing Frequency-Domain Filters, Sharpening Frequency Domain Filters, Homomorphic filtering. Image transforms (such as, DCT, Harr, DWT) for applications. Image Compression: Fundamentals - Image Compression Models, Some encoding techniques (including, block processing and transform-based techniques). 14 Hours Unit 3: Morphological Image Processing, Preliminaries of segmentation and Color Image Processing Dilation and Erosion, Opening and Closing, the Hit-or-Miss Transformation, Some Basic Morphological Algorithms. Image Segmentation: Detection of Discontinuities, Edge Linking and Boundary Detection, Thresholding, Region-Based Segmentation. Color Models, Color Transformations, Smoothing and Sharpening. 14 Hours Unit 4: Feature extraction, pattern classification Boundary Preprocessing, Boundary Feature Descriptors, Region Feature Descriptors, Transform, Image pattern classification using minimum distance classifier and prototype matching, deep convolutional neural networks and some of its recent variations for computer vision applications (UNet for segmentation, ResNet for feature extraction, YoLo for						

	object detection) and recent advances in computer vision. 14 Hours
Text Book(s):	1. “Digital Image Processing”, Rafael C Gonzalez and Richard E Woods, Prentice Hall, 4th Edition, 2018.
Reference Book(s):	1. Stan Birchfield, “Image Processing and Analysis”, 2017, First Edition, CL Engineering, ISBN number: 978-1285179520 2. “Digital Image Processing and Analysis”, Scott E. Umbaugh, CRC Press, 2023. 3. “Digital Image Processing”, S. Jayaraman, S. Esakkirajan, T. Veerakumar, McGraw Hill Ed. (India) Pvt. Ltd., 2020. 4. “Digital Signal and Image Processing”, John Wiley, 2012
Course Outcome	• Assess the quality of an image and identify the appropriate processing technique that needs to be applied in the spatial domain. • Apply image enhancement techniques frequency (Fourier) domain and interpret images in the transform domain. • Design and evaluate methodologies for segmentation of region of interest from binary, gray scale or color images. • Use appropriate feature extraction techniques for tasks such as classification and be able to work with data-driven approaches (deep learning architectures) for computer vision applications.

Course Code	UE24CS343AB2	Course Title	Big Data				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective - 2				
Tools/Languages	Hadoop, HDFS, Spark, Streaming Spark, HIVE, HBase, MLib.	Desirable Knowledge	Data Structures and its Applications.				
Prelude	This course provides a comprehensive introduction to fundamental data structure concepts, emphasizing both their theoretical foundations and practical applications. It begins with abstract representations of data and explores how these abstractions facilitate effective problem solving. The course then bridges the gap between theory and practice by demonstrating how these concepts can be concretely implemented using a programming language. Equal importance is given to understanding the underlying principles, their implementation techniques, and their real-world applications, enabling students to not only grasp the concepts but also apply them effectively in software development.						
Course Objectives:	• Introduce Big Data technologies. • Introduce computational and storage technologies for Big Data Ecosystem and infrastructure. • Introduce algorithms for processing Big Data and programming tools focusing on practical issues in working with Big Data. • Learn application of Big Data techniques to various real-life problems.						
Course Contents	Unit 1 : Introduction Big Data definition, Challenges and opportunities with Big Data, Data intensive scientific discovery and the role of Big Data, History, Distributed Storage - Google File System, HDFS, Computation model, Map Reduce architecture, YARN introduction, Google Big Query. Big Data Algorithms – matrix multiplication and PageRank, Issues with Hadoop, Complexity of Big Data algorithms – Communication Cost complexity model 14 Hours Unit 2 : Big data infrastructures (Compute/Storage) Hadoop Ecosystem - Apache HIVE, HBase, Cassandra, Apache Iceberg: A high-performance table format for data lakes that simplifies data ingestion by providing SQL-like ACID transactions and schema evolution. Apache Sea Tunnel: A very high-performance distributed data integration tool that supports synchronizing huge amounts of data between hundreds of data sources. Apache Druid - a high-performance analytics database for both real-time and historical data. Apache Hudi - brings real-time updates and incremental processing to data lakes Apache Airflow - a workflow orchestration platform for managing ETL pipelines and complex data workflows 14 Hours Unit 3 : In memory computation Spark and Scala/ PySpark programming model, Transformations and Actions, Spark SQL, Relational operators with Spark, Spark architecture – RDD, Data Frames, Wide and Narrow dependencies, Streaming Algorithms –sampling, set membership – Bloom Filters counting, counting unique elements – Flajolet Martin Algorithm. 14 Hours Unit 4 : Streaming analysis and advanced analytics on Big Data Streaming analytics use cases, Kafka – use cases, architecture, Streaming Spark, Apache Flink, , Spark MLLib – working examples, Spark GraphX, practical examples of Big Data success stories 14 Hours						

Text Book(s):	1: "Hadoop: The Definitive Guide", Tom White, O'Reilly, 4th edition, 2015. 2: Md. Rezaul Karim_ Sridhar Alla - Scala and Spark for Big Data Analytics_ Explore the concepts of functional programming, data streaming, and machine learning-Packt Publishing - ebooks Account (2017) 3: Stream Processing with Apache Spark - Gerard Maas, Francois Garillot, 2019
Reference Book(s):	1: "Mining of Massive Datasets",Anand Rajaraman,Jure Leskovec,Jeffrey D. Ullman, Cambridge Press, 2014. 2: "Big Data Analytics Beyond Hadoop: Real-Time Applications with Storm, Spark, and More Hadoop Alternatives",Vijay Srinivasa Agneeswaran, Pearson Education, 2014. 3: "Big Data Analytics", Rajkamal, Preeti Saxena, 1st Edition, McGraw Hill Education, 2019. 4: "Big Data Simplified", Sourabh Mukherjee, Amit Kumar Das, Sayan Goswami, 1stEdition, Pearson, 2019.
Course Outcome	• Explore various characteristics of Big Data Problems. • Evaluate principles and design alternative computational/storage technologies for Big Data. • Design Big Data applications using available infrastructure for Big Data through practical assignments. • Applying Big Data techniques in real life problems through a group-based project.

Course Code	UE24CS343AB3	Course Title	Graph Theory and its Applications				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 2				
Tools/Languages	NetworkX, PyTorch, Deep Graph, Library, Neo4j, Gephi, Graphistry	Desirable Knowledge	Problem Solving with C, Data Structures and its Applications.				
Prelude	This course focuses on mathematical structure to model the relations between the objects. It also discusses about the basics of graph theory together with a wide range of applications to different branches of Science and Technology, and to real-world problems. The course includes principles of Graph database and social networks.						
Course Objectives:	- Familiarize students with concepts and abstraction of Graph Theory. - Up skill students with computer representation of graphs and algorithms. - Introduce students with advanced concepts in Graph Theory including Vizing's and Brooks' Theorems. - Apply principles of graph theory for social network analysis and dynamic processes.						
Course Contents	Unit 1: Introduction, Paths, Cuts, Planar Graphs, and Vizing's Theorem Introduction- Representation and Traversals, Walks, Paths, Circuits - Euler graphs, Hamiltonian paths and circuits - Directed graphs, Digraphs and binary relations - Trees: Properties, Rooted trees, Binary trees, Spanning trees - Cut sets, Fundamental circuits and cut sets - Connectivity and Separability - Network flows, Isomorphism - Vizing's Theorem 14 Hours Unit2: Graph Coloring, Graph Applications, and Brooks' Theorem Planar Graphs – Different representations - Chromatic number, Chromatic partitioning, Chromatic polynomial - Matching, Covering - Four Colour Problem - Register allocation using graph coloring - Minimum Spanning Trees - Shortest Path Problem - Articulation Points - Network Flow Algorithms - Chinese Postman Problem - Time Table Problem - Strongly Connected Components - Brooks' Theorem on Colorings, Dynamic Graph Algorithms- Dynamic Connectivity 14 Hours Unit3: Graph Databases and Graphs in Social Networks Graph Database: RDBMS vs Graph Database - Neo4j Basics: Graph Data Model (Nodes, Properties, Relationships, Labels) - Basic CQL Commands, Cypher Query optimization, Knowledge Graphs- Build knowledge graphs using Neo4j, Graphs in Social Networks: Centrality Measures: Degree, Betweenness, Closeness, Eigenvector, PageRank - Measures of Cohesiveness: Degree Distribution, Diameter, Transitivity - Connected Components (in/out/weak/strong combined) - Groups and Substructures: Clique, N-clique, N-core. 14 Hours Unit4: Graphs in Social Networks and Social Graph Dynamics Community Detection: Cluster vs Community, Overlapped Communities (CPM), Girvan-Newman Algorithm, Louvain Algorithm - Generative Models: Random Graphs- Evolution and properties, Small World: Milgram's Small World Experiment, Watts-Strogatz Model, Barabási-Albert Model – Information diffusion in Social Graphs: Herd Behavior, Diffusion of Innovation (Bass Model), Epidemic Modeling, Information Cascades 14 Hours						

Text Book(s):	1. "Graph Theory: With Application to Engineering and Computer Science", Narsingh Deo, Prentice Hall of India, 2017. 2. "Graph Databases: New Opportunities for Connected Data", Ian Robinson, Jim Webber, Emil Eifrem, 2nd Edition, O'Reilly Publications, 2015. 3. "Networks, Crowds, and Markets: Reasoning about a Highly Connected World", David Easley, Jon Kleinberg, Cambridge University Press, 2010. 4. "Social Media Mining", Reza Zafarani, Mohammad Ali Abbasi, Huan Liu, Cambridge University Press, 2014.
Reference Book(s):	1. "Graph Theory", F. Harary, Addison-Wesley, 1969. 2. "Discrete and Combinatorial Mathematics", Ralph P. Grimaldi & B.V. Ramana, 5th Edition, PHI/Pearson Education. 3. "Graph Theory with Applications", J. A. Bondy and U. S. R. Murthy, Elsevier Science Publishing Co. 4. "Graph Representation Learning", William L. Hamilton, Morgan and Claypool Publishers, 2020. 5. "Introduction to Graph Theory", Douglas B. West, 2nd Edition, Prentice Hall.
Course Outcome	• Understand and model real-world problems using graph theory concepts. • Implement efficient graph representations and algorithms. • Apply graph coloring theorems like Vizing's and Brooks' to theoretical and practical problems. • Analyze and model social networks and dynamic processes using graph principles.

Course Code	UE24CS343AB4	Course Title	Modern Android App Development with Kotlin				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective				
Tools/Languages	Kotlin, Android Studio, Gradle	Desirable Knowledge	Programming Skills, Any Object oriented Programming Language preferably Java				
Prelude	This course introduces students to Android application development using Kotlin and modern Android development tools and libraries. Students will gain hands-on experience in developing interactive mobile applications using contemporary Android technologies. The course is designed to equip students with the knowledge and practical skills required for developing real world Android applications, enabling them to pursue advanced mobile computing projects, internships, research activities, and industry careers in mobile application development.						
Course Objectives:	- To introduce the Android development environment, Kotlin programming fundamentals, object-oriented concepts, functional programming techniques, and essential programming constructs required to build modern Android applications. - To understand modern Android UI development with declarative framework - jetpack Compose, Android application components, designing responsive user interfaces, managing state, implementing navigation, and handling lifecycle-aware UI behavior. - To learn Android application communication, background processing, data persistence, and resource management by using intents, broadcast receivers, services, local storage mechanisms, and application resources in Android applications. - To familiarize students with web communication, application deployment, and advanced Android capabilities by integrating networking libraries, device hardware features, on-device AI and deploying Android applications to Google Play Store.						
Course Contents	Unit 1: Introduction to Android Setting up Development Environment Introduction to Kotlin Variables and Primitive Data Types, Nullable Variables, Conditional Statements, For and While Loops, and Loop Control Statements, Functions and Arguments, Functional Programming with Lambdas, Creating Collections and Operations, Classes and Objects, Inheritance, Data Classes, Interfaces, Exception Handling 14 hrs Unit 2: Android Application Fundamentals – Activity, Service, BroadcastReceiver, ContentProvider Jetpack Compose Setup - UI Elements such as Text, Image, Buttons, Layouts including Column, Row Modifiers, Alignment and Arrangement, State Management - remember, mutableStateOf Custom Composables, Forms, Lists & Lazy Components - Using LazyColumn, LazyRow and Card, Using Scaffold, Compose Navigation, Side Effects & Lifecycle 14 hrs Unit 3. Intents, BroadcastReceivers, Services: Types of Intents, Intent Filter, Sending Broadcast, Receiving Broadcast, Registering BroadcastReceivers, Service life Cycle Data Persistency: SharedPreferences, Room Database Resources: Types of Resources, Creating and using resources 14 hrs Unit 4. Web Communication: Using Third Party Networking Libraries -using Retrofit, Downloading Images - using Glide App Deployment: Application Signing, Deploying APP in Play Store Introduction to Advanced Android Features and AI Integration: Camera, Sensors, Firebase Integration, ML kit GenAI APIs						

	14 hrs
Text Book(s):	1. Android Studio Meerkat Essentials - Compose Edition: by Neil Smyth, Payload Media, 2025. ISBN:978-1-965764-12-1
Reference Book(s):	1. Head First Android Development by Dawn Griffiths, David Griffiths - O'Reilly Media, 2nd Edition, 2021 2. Head First Kotlin by Dawn Griffiths, David Griffiths - O'Reilly Media, 2019 3. Building Modern Android Apps with Jetpack Compose and Kotlin by Peter Späth, Apress publisher, 1st Edition, 2023 4. Kotlin Programming Language (Web Source) 5. Android Mobile App Developer Tools - Android Developers
Course Outcome	• Apply Kotlin programming fundamentals, object-oriented principles, and functional programming concepts to develop basic Android applications using the Android development environment • Develop responsive Android user interfaces using Jetpack Compose by implementing UI components, state management, navigation, and lifecycle-aware application behavior. • Implement Android application communication, background services, data persistence, and resource management techniques using intents, broadcast receivers, services, local storage, and application resources • Build and deploy Android applications by integrating web communication libraries, device hardware features, and AI-powered functionalities, and publish applications to the Google Play Store.

Course Code	UE24CS343AB5	Course Title	Intelligent Vehicles: Connectivity and Cybersecurity				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective				
Tools/Languages	Kali Linux, Wireshark, VehicleSpy, Python (for attack simulation and secure coding), SCAP tools, IDS/IPS simulation frameworks, AUTOSAR security stack, Sumo, Veins, Omnet++, CARLA.	Desirable Knowledge	Computer Networks, Operating Systems				
Prelude	Automotive technology is undergoing a rapid transformation with the emergence of intelligent and connected vehicles. Modern vehicles integrate advanced sensing, communication, and computing capabilities that enable features such as advanced driver assistance, autonomous operation, and vehicle-to-everything (V2X) connectivity. While these advancements enhance safety, efficiency, and user experience, they also introduce new cybersecurity risks and attack surfaces within the vehicle ecosystem. Ensuring the security and resilience of intelligent vehicle systems has therefore become a critical requirement for the automotive industry. This course is designed to provide students with a comprehensive understanding of the architecture of intelligent and connected vehicles and the cybersecurity challenges associated with them. The syllabus introduces vehicle communication networks, connectivity technologies, potential cyber threats, and the security mechanisms required to protect automotive systems. By combining concepts from intelligent vehicle technologies and automotive cybersecurity, the course equips students with the knowledge and practical perspective needed to address emerging security challenges and contribute effectively to the development of secure intelligent mobility systems.						
Course Objectives:	● To introduce the architecture and key technologies of intelligent and connected vehicles, including sensors, electronic control units, and in-vehicle communication networks. ● To provide an understanding of vehicle connectivity technologies such as V2X communication and their role in modern intelligent mobility systems. ● To enable students to identify and analyze cybersecurity threats and vulnerabilities in intelligent and connected vehicle environments. ● To equip students with knowledge of security mechanisms, protection strategies, and industry standards used to secure automotive systems and communication networks.						
Course Contents	Unit 1: Introduction to Automotive Cybersecurity and Intelligent Autonomous Vehicle: ● Overview of the Vehicle Architecture ● Impact of Cybersecurity on intelligent vehicle ● In Vehicle Communication Protocols: CAN, LIN, FlexRay, Automotive Ethernet ● Classification of Autonomous Vehicle Based on Driving Levels (ADAS), State of the Art of Intelligent and Autonomous Vehicle Technologies, Battle for Adoption, Market Demand of Automotive Cyber Security. ● Attack Surfaces in Modern Vehicles (ECUs, Telematics, Sensors, OTA) ● Case Studies: Jeep Cherokee Hack, Tesla Autopilot Vulnerabilities 14 Hours Unit 2: Cyber Security in In-Vehicle Communication and Inter-Vehicle Communication ● In-Vehicle System, In-Vehicle Communication, In-Vehicle Network Architecture and Topology, Functional Safety and Cybersecurity, ● In-Vehicle Cybersecurity Issues and Challenges, ● Cyber Security in In-Vehicle Network (IVN). ● Internet of Vehicles (IoV),						

	- Machine Learning in Vehicular Networks, - Vehicular Social Network 14 Hours Unit 3: AUTOSAR Embedded Security in Vehicles, Threat Modeling and Risk Assessment: - Practical Threat Modeling and Security Controls - Overview of Threat Analysis and Risk Assessment (TARA) - STRIDE Model, Attack Trees, and MITRE ATT&CK for Vehicles - Asset Identification and Impact Analysis - Mapping Threats to Risk Levels and Determining Risk Treatment Options - Comparison of HARA (Safety) vs. TARA (Security) Approaches 14 Hours Unit 4: Secure System Design and Implementation, Standards, Regulations, and Lifecycle Management: - Automotive Cybersecurity Standards & ISO 26262/SAE 21434 - Principles of Secure Embedded System Design - In-vehicle Cryptographic Techniques: Secure Boot, Key Management, TLS, MACsec - Intrusion Detection Systems (IDS) in Automotive Context - Penetration Testing of Automotive Interfaces (CAN fuzzing, ECU spoofing) - Secure Coding Guidelines for Automotive Software - Secure Diagnostics and Access Control Mechanisms - UNECE WP.29 Cybersecurity Regulations: Compliance and Certification - Cybersecurity Management Systems (CSMS) and Organizational Processes - Post-production Monitoring, Incident Response, and OTA Security Updates - Cybersecurity Assurance Levels (CALs) vs. ASILs in Functional Safety - Industry Practice: OEM Cybersecurity Frameworks and Supplier Requirements 14 Hours
Text Book(s):	“Automotive Cybersecurity Engineering”, Dr. Ahmed M.K. Nasser, Springer, 2023. “Automotive Cyber Security Introduction, Challenges, and Standardization” by Shiho Kim • Rakesh Shrestha, Springer, 1st Ebook, 2020.
Reference Book(s):	Hans-Leo Ross, <i>Safety for Future Transport and Mobility: Engineering, Technology and Management</i>, Springer, 2021.
Course Outcome	- Understand the fundamental principles of automotive cybersecurity, including key vehicle communication protocols and typical attack surfaces in modern vehicles. - Analyze and model threats and risks in automotive systems using structured methodologies such as TARA, STRIDE, and MITRE ATT&CK. - Apply secure design principles, cryptographic techniques, and secure coding practices to develop resilient automotive software and communication systems. - Evaluate and simulate real-world attack scenarios on automotive networks using industry tools (e.g., CANoe, Wireshark, VehicleSpy, Python) to test intrusion detection and mitigation strategies. - Interpret and implement cybersecurity standards and regulations such as ISO/SAE 21434 and UNECE WP.29 across the vehicle development lifecycle.

Course Code	UE24CS343AB6	Course Title	Computer Network Security				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective				
Tools/Languages	SEED Ubuntu VM, Wireshark, Snort, Netwox, Scapy	Desirable Knowledge	Computer Networks				
Prelude	This course will give an overview and conceptual understanding of network related security aspects. Students will have opportunities to dwell well into technical “how to” with hands-on sessions and case study discussions.						
Course Objectives:	- To provide an overall view of Network Security and introduce the concept of packet analysis. - To understand the security problems in the design and implementation of the TCP, IP/ICMP, ARP protocols. - To learn the vulnerabilities in DNS protocol and to implement and experiment with Firewall rules. - To provide an overview of network management techniques and implementation of VPN. To understand concepts of risk management and security aspects of wireless networks.						
Course Contents	Unit 1: Introduction, Packet Sniffing & Spoofing, MAC Layer Attacks, Network Layer Attacks Packet Sniffing and Spoofing using C: Socket programming using C, Endianness, BSD packet filter (BPF). Packet sniffing using Raw sockets, Constructing spoofed raw ICMP packets and UDP packets. Python vs Scapy, MAC layer and attacks: The MAC layer Attack : ARP protocol, ARP cache poisoning attacks, MITM using ARP cache poisoning, Countermeasure. MACsec. Network layer attacks : IP Fragmentation, IPsec. 14 Hours Unit 2: TCP Attacks and DNS Attacks Attacks on the TCP protocols: TCP overview, Send and receive buffers, SYN flood attack: TCP 3-way handshake, the SYN flooding attack, Launching the attack using Netwox and C, Countermeasure. TCP reset attack: TCP reset attack on Telnet, SSH and video streaming connections. TCP session hijacking attack: TCP session and session hijacking, Launching the attack, Hijacked TCP connection. Reverse shell: working, redirecting IO to TCP connection, Creating reverse shell. Countermeasure. DNS Attacks: DNS hierarchy, zones and servers, DNS query process, Constructing DNS request and response using Scapy, DNS attacks: Overview, Local DNS cache poisoning attack 14 Hours Unit 3: Firewall, IDS, IPS, Honeypots Remote DNS cache poisoning attack (Kaminsky attack), Reply forgery attacks from malicious DNS servers, DNSSEC, TLS. Case Study – 1 Firewall: Introduction, Requirements of a firewall, Firewall characteristics and Access policy, Types of firewalls, NG firewall, Shortcomings, Firewall location and configuration: DMZ networks, Firewall topologies. Introduction, Build a simple firewall, Netfilter, iptables firewall in Linux, Stateful firewall and connection tracking, Application/Proxy firewall and Web proxy, Evading firewalls. Intrusion Detection and Prevention: Intruders, Intrusion detection, Analysis approaches, Host-based intrusion detection, Network-based intrusion detection, Distributed or hybrid intrusion detection, Honeypots, Example system: Snort. 14 Hours Unit 4: Virtual Private Networks, Wireless security Virtual Private Network: Introduction, Why VPN, analogy, and tunnelling. Overview of TLS/SSL VPN: Establishing a tunnel, Forwarding, and releasing IP packets, TLS/SSL VPN details. Building, Setup and Testing VPN. Bypassing Firewall using VPN. Case Study – 2. Wireless Security: Communications and 802.11 WLAN standards: Wired Equivalent Privacy (WEP), Wireless Protected Access (WPA), IEEE 802.1x, 802.11i/ WPA2, Wireless Network Threats. Brief introduction to SOC and SIEM. 14 Hours						

Text Book(s):	Internet Security: A Hands-on Approach, Wenliang Du, 3rd Edition, 2022 “Computer Security: Principles and Practice”, William Stallings and Lawrie Brown, 3rd edition, 2015
Lab Manual	PESU Lab Manual using VM’s with Industry tools like Wireshark, Kali linux, Netcat, Python Scapy, C, GDB, GCC, Snort, Nessus and others
Course Outcome	● Sniff packets from clients and analyse them to extract important info such as headers, passwords etc. ● Launch DoS and MITM attacks using various protocol vulnerabilities and mitigate them. ● Configure firewalls on Linux machines and exploit vulnerabilities on DNS protocol. ● Design and implement VPN for a secure connection over the internet .Master in wireless network security systems in depth and perform effective network management.

Course Code	UE24CS343AB7	Course Title	ROS for Autonomous Systems				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective Course				
Tools/Languages	ROS/ROS2, Python, C++	Desirable Knowledge	-				
Prelude	To introduce the foundational principles of autonomous systems, focusing on mobile robotics and programming using ROS (Robot Operating System). The course integrates theoretical understanding with simulation-based hands-on experience.						
Course Objectives:	- To introduce students to the fundamentals of autonomous robotics, including key components such as sensors, actuators, and controllers. - To familiarize students with the Robot Operating System (ROS) environment, including node creation, topic communication, and workspace setup. - To enable practical skills in integrating and programming common sensors and actuators in simulated robotic platforms. - To develop an understanding of robot perception, environment modeling, and mapping techniques such as SLAM. - To provide exposure to robot navigation using ROS frameworks and modern learning-based methods like reinforcement and imitation learning.						
Course Contents	Unit 1: Introduction to Autonomous Robotics & Simulation Tools What is a robot? Definitions, types, and criteria, Sensors, actuators, controllers, Autonomous vs. teleroobot systems; Environment classification. Overview of ROS, its architecture, and installation, RSVP: Robot Scenario Visual Planning; Hands-on Activities: Installing ROS and setting up workspace, Creating and launching ROS nodes (publisher/subscriber). 14 Hours Unit 2: Sensor Integration, Motor Control and Simulation Programming sensors: Color, ultrasonic, IMU, camera, Motor/servo characteristics and programming, Sensor packages and interfacing in ROS; Hands-on Activities: Simulating robot with LiDAR/camera in Gazebo, Sensor visualization with RViz and ROS topics, Motor movement control in a ROS simulation 14 Hours Unit 3: Localization Environment perception: Coordinate frames, TFs, Introduction to mapping: occupancy grids, SLAM, RSVP and SPACES methodology for decision making; Hands-on Activities: Building a 2D map using SLAM, Simulating perception using TurtleBot3 in Gazebo 14 Hours Unit 4: Planning and Navigation Localizing the robot in a map, ROS Navigation Stack-hardware requirement-navigation packages, path planning, motion planning of robot – software requirement and configuration. Navigation using Reinforcement learning and Imitation learning. 14 Hours						
Laboratory	1: ROS/ROS2 installation and setup 2: Visualization with Rviz, Turtlesim Node & Teleop Key 3: Create URDF File and Model of Mobile Robot, & Simulating Robot in Gazebo 4: RViz Sensor Visualization, -ROS Sensor Packages, IMU Integration -Camera-based Simulation 5: Publishing IMU and Encoder Data, -Simulating LiDAR sensor 6: Visualizing Coordinate Transforms in Rviz, -TF Tree Debugging and Dynamic Frames						

Text Book(s):	1. Lentin Joseph, Robot Operating System (ROS) for Absolute Beginners: Robotics Programming Made Easy, 1st Edition, APress, 2018 2. Jonathan Cacace; Lentin Joseph, Mastering ROS for Robotics Programming: Design, build, and simulate complex robots using the Robot Operating System, 2nd Edition, Packt Publishing, 2018.
Reference Book(s):	1. Hughes, C. and Hughes, T., Robot programming: a guide to controlling autonomous robots. Que Publishing, 2016 2. Quigley, M., Gerkey, B. and Smart, W.D., Programming Robots with ROS: a practical introduction to the Robot Operating System. " O'Reilly Media, Inc.", 201
Course Outcome	● Identify and describe the essential components and classifications of autonomous robotic systems. ● Configure and implement ROS-based simulations involving sensor-actuator integration using standard tools like Gazebo and RViz. ● Apply SLAM and environment modeling techniques for autonomous robot perception and mapping in simulated environments. ● Develop and test navigation algorithms using the ROS Navigation Stack with path and motion planning capabilities. ● Demonstrate foundational knowledge of learning-based navigation strategies such as reinforcement and imitation learning in robotics.

Course Code	UE24CS343AB8	Course Title	Enterprise Business Systems for Engineers				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective – 2				
Tools/Languages	Oracle, Sales force, Open ERP	Desirable Knowledge	-				
Prelude	The course in Enterprise Business Systems (EBS) offers students a comprehensive understanding of integrated business management systems and their role in streamlining organizational processes. Through theoretical insights, application examples, and case studies, students will understand the functionalities, implementation methodologies, challenges and strategic implications of ERP systems in modern enterprises including associated systems like Supply Chain Management [SCM] and Customer Relationship Management [CRM].						
Course Objectives:	- To learn the strategic importance of Enterprise Resource Planning, Supply Chain and CRM systems in the industry. - To learn the basics of ERP, the modules of ERP, their purpose and functions. - To learn the key selection criteria and issues & risks of ERP implementation. - To be aware of current trends like Role of Analytics, AI, Cloud and other technologies employed in EBS.						
Course Contents	Unit 1: Introduction to Enterprise Business Systems: Introduction to EBS, Value Chain Framework, Problems with Disintegrated Data in an Organization, Evolution of ERP Systems, Role of ERP Systems in an Organization, Scope of ERP Systems, General Model of Business and Role of ERP, Major ERP Players, Implementations Types – On-prem, Cloud, Hybrid, Two-tier. Life Cycle of an ERP Implementation Project: Life Cycle of an ERP Project, ERP Project Teams, Implementation Methodologies, Deployment Methods. Benefits and Cost of an ERP System: Benefits, Cost of an ERP Implementation, Cost-Benefit Analysis. 14 hours Unit 2: Change Management, Re-Engineering and BPM: Change Management: Change Management: People Issues: Are You Ready for ERP, Factors that Influence Pre-implementation Attitude, How to Enhance Attitude?, Change Management Strategies to Handle Organizational Issues, Creating a Change Management Strategy to Handle Organizational Issues, Tools for Assessing the Organizational Changes. Re-Engineering: Processes and their Characteristics, Life Cycle of a BPR Project, Life Cycle of an IT-driven BPR Project, Re-engineering Examples, Case Studies. Business Process Modelling and Business Modelling: BPM Introduction, Business Process Hierarchy, Standards for Business Processes and Modelling, Process Modelling Maturity and Multi-Dimensional Modelling, Process Modelling Software, Business Modelling, Integrated Data Modelling. 14 hours Unit 3: ERP Functional Modules: Human Capital Management: Human Capital Management: HCM Systems, Leading HR Solutions from ERP Vendors, Strategic Vs. Operational HR Processes and HR Outsourcing, Employee Health and Safety. Financial Management: ERP Financial Application, Financial Modules in Detail. Procurement and Inventory Management: Procurement, Inventory Management. Production Planning and Execution: Understanding MRP II Concepts, How ERP PP module supports MRP II Processes, Critical Master Data Elements, Managing different Production Scenarios.. 14 hours Unit 4: ERP Selection and Emerging Trends: ERP Solution and Vendor Selections, Selection Criteria, Request for Proposal (RFP), Gap Analysis, ERP Selection Team. Managing an ERP Project: Critical Success Factors, Project Risks, Key Performance Metrics and Data Migration. ERP and Related Technologies: Supply Chain Management (SCM), Customer Relationship Management (CRM), Product Life Cycle Management (PLM), Data Warehousing and Business						

	Intelligence (DW-BI). Introduction to Commercial ERP Related Software: Indian ERP Market, SAP, Oracle, MS Dynamics, Salesforce CRM. Implementations in India; Articles and Case Studies on ERP Trends like Analytics, Cloud, Digital Transformation, Embedding AI & ML capabilities, Automation, Block-chain, Cybersecurity, IoT, Sustainability and other technology integrations. Explore emerging trends like Autonomous Agents, Gen-AI and Web resources like Gartner Magic Quadrant and Hype Cycle for market stats. 14 hours
Text Book(s):	1. "Enterprise Resource Planning- Text & Cases", Rajesh Ray, Tata McGraw Hill, New Delhi, 2019
Reference Book(s):	1. "Supply Chain Management: Strategy, Planning and Operation", Sunil Chopra, Pearson Education, 8th Edition, 2025. 2. "Modern ERP: Select, Implement, and Use Today's Advanced Business Systems" by Marianne Bradford; Pearson, 4th Edition, 2020. 3. "Enhancing Enterprise Intelligence: Leveraging ERP, CRM, SCM, PLM, BPM, and BI", Vivek Kale, Auerbach Publications, 2016. 4. Ultimate Guide to ERP by TechTarget
Course Outcome	• Identify typical functionality of ERP, SCM & CRM sub-systems. • Apply criteria to select ERP Package & Consulting Partner. • Systematically develop plans for an ERP Implementation project. • Identify critical success factors and associated risks of an ERP Implementation.

Course Code	UE24CS343AB9	Course Title	Design of Secure Platform and Systems				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	5	Type of Course	Elective Course				
Tools/Languages	QEMU, Linux, Python, C	Desirable Knowledge	Operating Systems, Microprocessor architecture				
Prelude	Have you ever wondered what happens between the moment you power on a computer and when the operating system finally takes control? Or how hyperscale datacenters securely manage the power consumption and data transfer of thousands of interconnected processors and accelerators? The Platform and Systems Engineering elective is designed to bridge the critical gap between traditional computer architecture and operating systems. Moving beyond the isolated CPU core, this course immerses you in the reality of modern computing, where platforms are defined by complex System-on-Chip (SoC) architectures, high-speed coherent interconnects like CXL, advanced power and thermal management, and hardware-enforced security boundaries. By taking this course, you will transition from writing software that merely runs <i>on</i> a system to engineering the platform itself. You will gain hands-on, highly sought-after industry skills using open standards like RISC-V, UEFI, and SPDm. Through rigorous labs and project, you will experiment with all the technologies here. If you aspire to work at the cutting edge of cloud infrastructure, embedded systems, IOT, or silicon design, this course will equip you with the systemic perspective needed to succeed.						
Course Objectives:	- To introduce the foundational concepts of platform initialization, firmware development, and hardware discovery mechanisms (such as ACPI and Device Tree) to successfully bridge the hardware-software interface. - To explore advanced System-on-Chip (SoC) architectures, focusing on modern high-speed interconnects, PCIe fundamentals, and cache-coherent memory expansion technologies like Compute Express Link (CXL). - To examine the physics of computing power and teach the implementation of energy-efficient dynamic voltage and frequency scaling (DVFS) alongside software-based power management governors. - To provide a comprehensive understanding of hardware-level security, including cryptographic identity attestation (DICE, SPDm), secure boot protocols, and Trusted Execution Environments (TEEs) on open architectures like RISC-V.						
Course Contents	Unit 1: The Platform — Boot, Firmware, and Discovery - The Hardware-Software Interface: Toolchains and Cross-Compilation. The anatomy of an ELF binary. Linker scripts and linking a binary to a specific physical memory address. The C/Assembly Interface: Application Binary Interface (ABI), calling conventions, and stack frame setup. - Advanced I/O and Direct Memory Access: Memory-Mapped I/O (MMIO) - using volatile pointers in C to manipulate hardware registers. Direct Memory Access (DMA) - how peripherals independently address memory. - Applied Cryptography for Systems Engineers: Crash course in cryptography required for secure boot and attestation. Cryptographic Hash Functions (SHA-256/384). Asymmetric Digital Signatures (RSA/ECDSA). Public Key Infrastructure (PKI) and X.509 Certificates. Threat modeling: Defining the Trusted Computing Base (TCB) against physical vs. software attackers. - Introduction to Platform Architecture & The Boot Process: The "Platform" definition (SoC + Board + Firmware). The Boot Sequence: Power-On Reset (POR), ROM, First Stage Bootloader (SPL), Second Stage (U-Boot/GRUB). Legacy BIOS (Real Mode) vs. UEFI (Protected Mode). Reset Vector, Memory Mapping, BSS initialization, Stack setup in Assembly						

- Hardware Description: ACPI vs. Device Tree: How the OS discovers hardware Plug-and-Play, Enumeration, I/O Resource Management
- ACPI: Tables (RSDP, XSDT, DSDT, MADT). The AML bytecode interpreter in the OS.
- Device Tree: The DTS syntax (nodes, properties, interrupts, reg). DTB compilation (dtc). Overlays for dynamic hardware.
- Virtualization and IO virtualization: Introduction to virtualization; CPU, memory virtualization, Introduction to the IOMMU (Input-Output Memory Management Unit) for DMA remapping, and SR-IOV (Single Root I/O Virtualization) distinguishing Physical Functions (PFs) from Virtual Functions (VFs).

14 Hours

Unit 2: SoC Architecture and Interconnects

- Advanced Interrupts and Coherency: Moving beyond basic CPU faults. Hardware interrupt controllers (ARM GIC, x86 APIC) and Message Signaled Interrupts (MSI/MSI-X).
- Cache Coherency: Snooping vs. Directory-based coherency. Deep dive into the MESI state machine (Modified, Exclusive, Shared, Invalid).
- The System Fabric (NoC and Interconnects): Moving data on the chip. Bus Protocols: AMBA (Advanced Microcontroller Bus Architecture), AXI (Advanced eXtensible Interface). Arbitration, Split transactions, Burst transfers, Coherency (ACE protocol).
- PCIe Fundamentals, IDE security Intro: Root Complex, Endpoints, Switches, and Bridges. Bus/Device/Function (BDF) addressing. The Threat Model: Physical interposers and malicious switches. Why standard PCIe is vulnerable to observation and injection.
 - Link IDE: Encryption between two directly connected ports.
 - Selective IDE: End-to-end encryption through switches.
- Compute Express Link (CXL): The Limitation of PCIe- The lack of cache coherency and the latency of DMA. CXL Architecture - Multiplexing three protocols on the PCIe PHY.
 - CXL.io: Discovery, configuration, and legacy I/O (identical to PCIe).
 - CXL.cache: Allowing devices to cache host memory. The MESI protocol states across a serial link.
 - CXL.mem: Allowing the host to access device memory.
- CXL 2.0/3.0 - Device Types:
- Type 1: Accelerators without local memory (e.g., SmartNICs). Use CXL.io + CXL.cache.
- Type 2: Accelerators with local memory (e.g., GPUs). Use all three protocols.
- Type 3: Memory Expanders. Use CXL.io + CXL.mem.

14 Hours

Unit 3: Power and Performance Engineering

- The Physics of Computing and Performance Monitoring: Electrical engineering basics for CS students: Voltage, Frequency, and Capacitance. The CMOS Power Equation ($P = C \cdot V^2 \cdot f$) and why voltage scaling yields quadratic power savings.
- Telemetry: Hardware Performance Monitoring Counters (PMCs) and Model-Specific Registers (MSRs). How the CPU physically counts cache misses and cycles.
- Power Physics and Management Fundamentals: CMOS Power Equation. Dynamic vs. Static Power. Clock Gating, Power Gating.
 - DVFS: Dynamic Voltage and Frequency Scaling.
 - States: C-States (Idle), P-States (Performance), S-States (Sleep/Suspend)
- Software Power Governors and Scheduling: Linux cpufreq subsystem. Governors: performance, powersave, ondemand, schedutil. Energy Aware Scheduling (EAS) for asymmetric cores (big.LITTLE). Load tracking (PELT), Capacity awareness, Thermal throttling
- Profiling and Telemetry (RAPL & Perf): Measuring power in software. Intel RAPL (Running Average Power Limit). The powercap framework. Using perf for hardware events (cycles, instructions, cache misses) and turbostat.

14 Hours

Unit 4: Platform Security

	• Roots of Trust and Measured Boot: RoT for Measurement (RTM), RoT for Storage (RTS), and RoT for Reporting (RTR). Measured Boot and TPM - Extending hash measurements into Platform Configuration Registers (PCRs). The difference between "Secure Boot" (Verification) and "Measured Boot" (Recording). Supply Chain Security - The problem of counterfeit hardware and mutable firmware attacks. • Device Identifier Composition Engine (DICE): The TCG DICE architecture for IoT and constrained devices. The Power-on-Latch mechanism, Unique Device Secret (UDS), and how the firmware uses a Key Derivation Function (KDF) to combine the UDS with the hash of the next software layer to generate a Compound Device Identifier (CDI). • Security Protocol and Data Model (SPDM): A standard protocol for device authentication over any transport (PCIe, MCTP, USB). Foundation for zero-trust architectures by enabling secure chip-to-chip (S2C) communication and verifying attached peripherals before they are utilized. SPDM protocol overview. • Introduction to Trusted Execution Environments (TEE): Isolation Models. Least Privilege, Attack Surface Reduction, Side-channel risks. ○ ARM TrustZone: System-wide isolation (Secure vs Normal world). The NS-bit on the AXI bus. ○ RISC V TEE models. • RISC V Process-Level TEEs: Keystone and Penglai: Architecture of process-level secure enclaves on open architectures. • Keystone: The Security Monitor (SM) executing in M-mode, using PMP to carve out isolated, contiguous physical memory regions for the enclave (Runtime + User App), inaccessible to the hosting Linux OS. • Penglai: User-mode enclave systems, leveraging optional hardware extensions like sPMP (Supervisor PMP) and Guarded Page Tables for scalable secure resources. Integration with OpenSBI dynamic domains. • Hardware Vulnerabilities and Mitigations: When Hardware Fails. Speculative Execution attacks (Spectre, Meltdown). Rowhammer (DRAM disturbance). Speculation barriers, Cache flushing, Kernel Page Table Isolation (KPTI). 14 Hours
Text Book(s):	NA
Reference Book(s):	NA
Course Outcome	• Initialize and debug bare-metal systems and firmware bootloaders, effectively configuring hardware descriptions via Device Trees to hand off control to an operating system. • Analyze and configure modern interconnect topologies, demonstrating an understanding of how data moves across PCIe buses and how CXL facilitates memory pooling and coherency. • Profile system workloads to measure power consumption accurately, and design software-based thermal and power management solutions using telemetry tools and Linux governors. • Architect and evaluate platform security models by implementing hardware roots of trust, device attestation protocols (SPDM), and secure isolated enclaves.

Course Code	UE24CS351B	Course Title	Cloud Computing				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	6	Type of Course	Core Course				
Tools/Languages	Amazon AWS (or equivalent), AWS SkillBuilder, AWS Educate, Qwiklabs, Docker, Kubernetes, Jenkins, Zookeeper, Github, NoSQL database, Flask, Python, Go Lang. AI tools- AWS Sagemaker	Desirable Knowledge	Computer Networks, Operating Systems.				
Prelude	The cloud computing course introduces not only the various technologies that go into building a cloud native application, but also how cloud systems are designed. The student is introduced to various tools and design techniques/trade-offs. It also gives a flavour for the business relevance/ethics of using cloud computing. This course requires the student to have a desirable knowledge of Computer Networks and Operating System.						
Course Objectives:	● Introduce the rationale behind the cloud computing revolution and cloud native application Architecture ● Explore the concepts of Virtualization and Containerization. ● Explore the concepts of Distributed storage and its various techniques ● Design distributed systems for scalability and expose the student to various trade-offs in designing cloud architectures.						
Course Contents	Unit 1 : Distributed System refresher Parallel computing, Grid computing, Introduction to technology challenges with Distributed & Cloud computing, ephemeral nature of compute nodes and its implications in distributed systems, CAP Theorem, Microservices and differences with the traditional monolithic model; challenges of migrating monolithic applications, deployment models, Multi-cloud, Communication using Message queues- Pub Sub model. 14 Hours Unit 2 : OS refresher and control plane for Cloud Hypervisor - Types, Para virtualization and Transparent virtualization, Software - Trap and Emulate virtualization, Software - Binary translation, Goldberg Popek principles for Virtualization, Hardware - AMDv/Intel, Memory - Shadow page tables, Memory - Nested page tables, IO, VM Migration, Cluster coordination – consensus, , Cluster coordination - leader election, distributed locking, Case Study: Zookeeper/Raft - distributed consensus infrastructure. Edge Computing and Fog Computing - principles and paradigms, Orchestration with Kubernetes. Identity and Access Management (IAM). 14 Hours Unit 3 : Distributed Storage Types of Cloud storage - Block, Object stores, Replication, lag, multileader replication, Leaderless replication, Partitioning - key-value data, Consistent hashing, Partitioning - rebalancing partitions, Request routing, Consistency Models, CAP Theorem, Transactions, SAGA, Two-phase commit. 14 Hours Unit 4: Applications on Cloud and Security Cloud service models IaaS/PaaS/SaaS, , Microservices, REST, Multitenancy, Multitenant databases. reverse proxies, containerization and automation through Docker and Jenkins. Cloud security requirements - physical/virtual security, security design patterns, Container security features to work with Docker and Kubernetes, Cloud Threats – DDoS, Economic Denial of Sustainability. 14 Hours						

Laboratory	1. Migrating a monolithic e-commerce application to a microservices architecture 2. Building a Task Management Application with Raft Consensus Algorithm and MySQL. 3. Microservices communication using RabbitMQ 4. Building a Distributed Key-Value Store with etc 5. Back Up service using docker and Kubernetes 6. Building an E-commerce Microservices Application on Cloud using Docker, Kubernetes, Jenkins, and Git
Text Book(s):	1. “Distributed and Cloud Computing”, Kai Hwang, Jack Dongarra, Geoffrey Fox. ISBN: 978-0-12- 385880-1, Morgan Kaufmann, 2012. 2. “Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems”, Martin Kleppmann. O’Reilly, 2016. 3. “Fog and edge computing: principles and paradigms” / edited by Rajkumar. Bhuyya, Satish Narayana Srirama. Description: Hoboken, NJ, USA: John Wiley & Sons, 2019
Reference Book(s):	1. Docker in Action”, Jeff Nickoloff, Manning Publications, 2016. 2. “Cloud Native DevOps with Kubernetes”, John Arundel and Justin Domingus, O’Reilly, 2019. 3. “Moving to the clouds: Developing Apps in the new world of cloud computing”, Dinkar Sitaram and Geetha Manjunath. Syngress, 2011.
Course Outcome	• Comprehend the technical and business rationale behind cloud computing. Decide the model of cloud computing to use for solving a particular problem. • Implement Micro service architecture through Containers and Orchestration tools. Analyse virtual machines and containers. • Experiment with cloud storage Models such as object stores, key value stores. • Apply the critical constraints such as Performance, scalability and security to the designed distributed system.

Course Code	UE24CS352B	Course Title	Object Oriented Analysis and Design				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	6	Type of Course	Core Course				
Tools/Languages	Star UML, Object Oriented Programming Language(Java)	Desirable Knowledge	Star UML, Object Oriented Programming Language(Java)				
Prelude	In this course students will learn to perform Analysis on a given domain and come up with an Object- Oriented Design (OOD). Various techniques will be discussed and practiced which are commonly used in analysis and design phases in the software industry. Unified Modelling Language (UML) will be used as a tool to demonstrate the analysis and design ideas and an object-oriented programming language such as Java would be used to implement the design. The theory is supplemented with implementations which are demonstrated/ practiced in class which provides the hands-on experiences of implementing the patterns.						
Course Objectives:	- Familiarize students with static and dynamic models of object-oriented analysis and modelling using the unified modelling language (UML) - Introduce students to object oriented programming concepts in Java. - Make students appreciate the importance of system architecture design in software development. - Introduce the students to understand the importance of GRASP and SOLID design principles.						
Course Contents	Unit 1: Advanced OO, Object Oriented Analysis and Static Models and Diagrams OOA: Requirements, Modeling and Analysis, Introduction to UML, Use Case Modelling: Use Cases Diagrams. Class Modelling: UML Class Diagrams, OO relationships, CRC Diagrams, Component model, Deployment model, Activity Modelling: UML Activity Diagrams and Modelling, Guidelines. Behaviour Modelling: Sequence Diagram, UML State Machine Diagrams and Models, Advanced State Models 14 Hours Unit 2: Object Orientated Programming Object-oriented Programming: JVM, Abstraction, Encapsulation, Composition, Class-Attributes, Behaviour, Objects, and Methods. Interface and Implementation: Role of Constructors and Garbage Collector, Parameter Passing, Value Type and Reference Type, Overloading of Methods, Class Attributes and Behavior: Difference between Class Methods and Instance Methods, Inheritance: method overriding and Interface, Abstract Class, Concepts of Single Rooted Hierarchy -Object Class in Java, Collection, ArrayList and Map; Multithreading 14 Hours Unit 3: Design principles OO Development process, System Design and Frameworks, Architectural patterns, MVC architectural pattern, GRASP and its application to Object Design, Creator, Information Expert, Low Coupling, Controller, High Cohesion, Polymorphism, Pure Fabrication, Indirection and Protected Variations .SOLID: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion OO Design Principles and Sample Implementation of Patterns in Java. 14 Hours Unit 4: OO Design patterns Introduction to Design Patterns, selection and usage of a design pattern, Creational Design Patterns Theory and implementation in Java: Singleton, Factory, Builder and Prototype, Structural Patterns – Adapter, Façade, Proxy and Flyweight, Behavioral Patterns – Chain of Responsibilities, Command, Observer, Strategy.						

	14 Hours
Laboratory	• Lab assignment on Use case diagram. • Lab assignment on Class diagram. • Lab assignment on Activity and State diagrams. • Lab assignment on Java fundamentals. • Lab assignment on Java Advanced features (Inheritance, Composition, etc.). • Self-Learning Assignment on Java Serialization and Multithreading • Hands-on Assignment on MVC Framework. • Assignment on Design Patterns. • Mini Project using MVC Framework and incorporating all learning of the course.
Text Book(s):	1. "Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development", by Craig Larman, 3rd Edition, Pearson 2015. 2. "Software Architecture Patterns" by Mark Richards, 2nd Edition, OReilly 2022 3. "Java the Complete Reference", Herbert Schildt ,McGraw-Hill ,13th Edition, 2024
Reference Book(s):	1. "Object-Oriented Modelling and Design with UML", Michael R Blaha and James R Rumbaugh, 2nd Edition,Pearson 2007. 2. "Design Patterns: Elements of Reusable Object-Oriented Software" by Erich Gamma, 3. Richard Helm, Ralph Johnson, and John Vlissides, 1st Edition, Pearson 2015.
Course Outcome	• Construct static models, use cases and class models, followed by analyzing the dynamics of the system using activity, sequence, state and process models. Depict the architecture of a software system by using component and deployment models • Use the concepts of classes and objects of object-oriented programming in Java to model a complex system • Use GRASP and SOLID principles in the design of software application and apply Creational software design patterns for variety of application scenarios • Apply Structural and Behavioural software design patterns for variety of application scenarios.

Course Code	UE24CS341B	Course Title	Compiler Design				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Core Course				
Tools/Languages	LLVM/Clang, Lex/flex and YACC/Bison.	Desirable Knowledge	Data Structures and its Application, Automata Formal Languages & Logic.				
Prelude	The design and implementation of programming languages continue to evolve as new paradigms, tools, and techniques emerge. Languages and toolchains that fail to adapt become obsolete, making an understanding of compilers essential for the modern Computer Science engineer. Compiler design provides the foundation needed to interpret language constructs, implement language features, and contribute to the evolution of programming languages. A solid understanding of compiler functioning enhances a student's ability to grasp programming language semantics, debug more effectively, optimize code more intelligently, and learn new languages with greater confidence and insight. Students are expected to have prior knowledge of Data Structures, Automata Theory, Formal Languages, and Logic.						
Course Objectives:	• Introduce the fundamental concepts of language translation and compiler design. • Develop an in-depth understanding of the issues involved in programming language design and implementation. • Provide practical skills required to construct a compiler and build awareness of the functionality and complexity of modern compilers. • Explain the importance of code optimization and introduce techniques for optimizing programs from a compiler's perspective.						
Course Contents	Unit 1: Compilers: Introduction, Lexical Analysis, Top-down Parsers The Language Processing System, The Phases of a Compiler, The Grouping of Phases into passes. Lexical Analysis: The Role of the Lexical Analyzer, Input Buffering, Specification of Tokens, Recognition of Tokens, Design of a Lexical Analyzer Generator. The role of the parser, Syntax Error Handling, Error-Recovery Strategies. Top-down parsing: Recursive-Descent Parsing (without backtracking), LL(1) Parser. 14 Hours Unit2: Syntax Analysis: Bottom-up Parsers, Syntax-Directed Translation Bottom-up parsing: Shift-Reduce Parsing, viable prefixes, CLR, LALR, Error recovery for Bottom-up parsers Syntax- directed definitions, Evaluation orders for SDD"s: S-attributed SDD, L-attributed SDD. 14 Hours Unit3: Implementation of Syntax-Directed Translation Schemes and Intermediate Code Generation Applications of Syntax-Directed Translation - SDD for Syntax Trees, Expressions, Basic Types and flow control statements, Type Checking, Syntax-directed Translation Schemes – Parser Stack Implementation of Postfix SDT's, SDT's with actions inside Productions, SDT's for L-Attributed Definitions. Implementing L-Attributed SDD"s: Bottom-Up Parsing. Variants of Syntax Trees – Directed Acyclic Graphs for Expressions, Three-Address Code – Addresses and Instructions, Quadruples, Triples, Indirect Triples, SSA Form, Control Flow Graph. 14 Hours						
	Unit 4: Machine Independent Code Optimization, Code Generation and Run Time Environment						

	Machine Independent Optimization: Different Optimizations, Optimization of Basic Blocks. Data Flow Analysis: Live-variable analysis, Next-use algorithm. Storage Organization, Different Allocation Strategies, Stack Allocation of space, Access to Non-local Data on the stack. Code Generation: Issues in the design of a code generator, the target language, addresses in the target code, static allocation, stack allocation, run-time addresses for names. A Simple Code generator - The Code generation algorithm, Register allocation problem (Graph coloring) 14 Hours
Text Book(s):	1. “Compilers–Principles, Techniques and Tools”, Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffery D. Ullman, 2nd Edition, Pearson Education, 2023
Reference Book(s):	1. “Modern Compiler Design”, Dick Grune, Kees van Reeuwijk, Henri E. Bal, Cerie J. H. Jacobs, Koen Langendoen, 2nd Edition, 2012. 2. AW Appel, J Palsberg, Modern Compiler Implementation in Java, Cambridge University Press, 2002.
Course Outcome	• Use the knowledge of patterns, tokens, and regex for solving the problems in the field of data mining • Analyse and design the semantic behaviour of a compiler. Choose the appropriate compiler internal representation for different kinds of compiler tasks. • Translate a source-level language into a low-level compiler internal representation. • Optimize the performance of a program in terms of speed and space using new code optimization techniques.

Course Code	UE24CS342BA1	Course Title	Supply Chain Management for Engineers				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -3				
Tools/Languages	SCM applications and tools, ML tools, Case studies, Web resources	Desirable Knowledge	Enterprise Business Systems.				
Prelude	This course explores supply chain management (SCM) processes. It aims for students to develop the skills to understand, design and evaluate techniques in the realm of supply chain management. It covers fundamentals relevant to SCM, current SCM challenges and trends in the industry including planning and prediction, techniques and tools using AI technologies.						
Course Objectives:	• Understand the fundamentals of supply chain management. • Learn the various SCM stages and strategies of planning, sourcing, inventory, distribution and logistics. • Explore case studies of SCM applications to understand various industrial scenarios. • Explore current trends and AI techniques applied to solve real-world SCM challenges.						
Course Contents	Unit 1: Introduction to SCM Overview of supply chain management and it's key Objectives. Understand supply chain scope, strategic fit, its drivers and performance metrics. Building a strategic framework to analyze Supply Chains. Managing Material and Information flows. Intro to supply chain networks and their design. Various strategies of planning, sourcing, inventory, distribution and logistics related to SCM – Opportunities, Competitive advantage. Value Stream Mapping. Key Drivers - Inventory, Transportation/Logistics, Facilities (Warehousing, Manufacturing plants, Docks), Sourcing/Procurement, Pricing Strategies. 14 Hours Unit 2: Procurement and Supply Chain Planning (Demand & Supply) Procurement processes, Supplier selection and performance assessments. Intro to Demand Forecasting models, predictive analytics for supply chain planning. Techniques for aggregate planning of demand and supply in supply chains, managing demand variability and accuracy. Case studies. 14 Hours Unit 3: Planning and Managing Inventory and Logistics Management in SCM Introduction to planning and management of Inventory in an enterprise. Techniques for warehouse management and logistics management for distribution. Logistics stakeholders and modern logistics concepts, Outsourcing logistics, Logistics KPIs like On-Time Delivery, Fill Rate, Inventory Turns, etc. Intro to Reverse Logistics, and Lean Logistics. Enhancing transportation and logistics with real-time tracking and monitoring using IoT and AI-driven route planning and last-mile delivery. Case studies in inventory and logistics management. 14 Hours Unit 4: Drivers, Analytics and Current Trends in Supply Chain Management Understanding and managing cross functional drivers in supply chain. Decision making in supply chains - sourcing, pricing, risk management and compliance. Basics of AI, Blockchain, IoT concepts applied to SCM. Current trends in SCM: E-procurement, Sustainability, Green SCM, Lean and Global SCM, Real-time, Ethical and Transparent Supply Chains, SCM Analytics, Digital Twin Tech. Exploration of AI & Gen-AI techniques, tools and technologies for enhancing SCM like AI driven Demand Forecasting – Case studies. Awareness of current challenges and research areas of SCM in the industry like Ethical and transparent operations. 14 Hours						

Text Book(s):	1. Supply Chain Management: Strategy, Planning and Operation, Sunil Chopra, Pearson, 7th Revised Edition, 2024. 2. Supply Chain Logistics Management, Donald Bowersox, et al, Mc-Graw Hill Publishing, 5th Edition, 2024.
Reference Book(s):	1. Essentials of Supply Chain Management, Michael H Hugos, John Wiley & Sons, 3rd Edition, 2018. 2. Principles of Supply Chain Management: A Balanced Approach, Joel D Wisner, Cengage, 5th Ed, 2019 3. Strategic Supply Chain Management: The Five Core Disciplines for Top Performances, Cohen Shoshanah & Joseph Roussel, Mc-Graw Hill, 2nd Ed, 2013. 4. Logistics and Supply Chain Management, Martin Christopher, Pearson, 2022 5. Supply Chain Management: A Logistic Approach, John J Coyle, Cengage, 10th Ed, 2019. 6. A Roadmap to Green Supply Chains: Using Supply Chain Archaeology and Big Data Analytics, Kevin L Lyons, Industrial Press, 2015 7. Lean Supply Chain and Logistics Management, Paul Myerson, Mc-Graw Hill, 2012. 8. Supply Chain Analytics, T. A. S. Vijayaraghavan, Wiley, 2021.
Course Outcome	• Identify various stages of SCM including Logistics. • Evaluate KPIs and select suitable strategies for optimized supply chain performance. • Evaluate and select the strategies for planning and managing Inventories and Logistics. • Apply ML techniques to predict trends, optimize inventory, and improve logistics.

Course Code	UE24CS342BA2	Course Title	Algorithms for Information Retrieval				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -3				
Tools/Languages	Pytorch, C, C++, Java	Desirable Knowledge	Data structures, Design and Analysis of Algorithms				
Prelude	This course provides a comprehensive introduction to Information Retrieval. By the end of this course, students will gain a strong understanding of how modern information retrieval systems work—from basic keyword-based search to advanced semantic and multimodal retrieval techniques. They will learn how to represent text, images, and audio using embeddings, design efficient retrieval pipelines, and apply neural models for ranking and question answering. Students will also understand how systems like retrieval-augmented generation (RAG) combine search with language models to produce accurate, context-aware answers. Overall, the course equips students with both theoretical foundations and practical insights needed to build intelligent search systems and tackle real-world problems in areas such as web search, recommendation systems, and multimedia retrieval.						
Course Objectives:	- Understand the architecture, models, and algorithms used in Information Retrieval. - Understand the basic principles and implementation of Indexing and Search. - To explore machine learning techniques for Information Retrieval, Web, and multimodal applications. - To study neural models for Question Answering and recent advancements in the field.						
Course Contents	Unit-1: Basics of Information Retrieval Introduction to Information Retrieval, Background, Architecture and Strategies of Information Retrieval (IR) Systems, IR Models, Boolean and Extended Boolean Models, Dictionary, Vocabulary, Positional Postings, Phrase Queries and Tolerant Retrieval, Indexing and Vector Space Model, Evaluation of IR Algorithms for Indexing and Index Compression, Vector Space Model for Scoring, tf-idf and Variants. 14 Hours Unit-2: Ranking, Web Search Basics and Link Analysis, Efficient Scoring and Ranking, Parametric and Zone Indexes, Tiered Indexes, Query Term Proximity, Query Parser, Aggregating Scores, Performance Measurement, Web Applications and Search Algorithms, Relevance Feedback, Query Expansion, Other IR Models, Web Search Basics. Economic Model of Web Search, Improving Search Results- Rank Aggregation, Link Analysis, The Page Rank Algorithm, Other Search Algorithms, Scalability Issues in Search. Search User Experience, Web Crawling and Indices, Link analysis, Building a Complete Search System, Lucene as a Search Engine, and Other search engines like Solr, Everything, and Google 14 Hours Unit-3: Semantic Retrieval, Multimedia Information Retrieval Word Embeddings- CoBoW, SkipGram, Word2Vec, Sentence Embeddings- Universal Sentence Embeddings, Document Embeddings: Doc2Vec, BERT. Embedding based Reranking: Bi-encoders and Cross Encoders. Multimedia Information Retrieval: Basics of Audio, Image and video representation, Audio features, Audio Information Retrieval, content-based visual information retrieval: metadata searching, Query by Example, Semantic Retrieval, Machine Learning Approaches: Image Embeddings, content comparison using image distance measures, Examples of MIR Query languages. Multimodal Information Retrieval: Introduction, Requirements, Applications, challenges, Fusion Strategies, Vision Language Models: CLIP, VisualBERT, BLIP 14 Hours Unit-4: Question Answering, RAG Question Answering, neural models for IR Question Answering, Information Retrieval task, Factoid QA models, Entity Linking models, Knowledge-based QA, and Pretrained models for QA. Retrieval						

	Augment Generation in QA- indexing and generation pipeline, evaluating a RAG system, Advanced RAG strategies 14 Hours
Text Book(s):	1. "Introduction to Information Retrieval", Christopher D. Manning, Prabhakar Raghavan, HinrichSchutze, ISBN: 9781107666399, Cambridge University Press, 2009. 2. "Search Computing,, Challenges and Directions:", chapter 8: Multimedia and Multimodal Information Retrieval. ISBN: 9783642123108, 3642123104. Marco Brambilla, Stefano Ceri. Springer Berlin Heidelberg, 2010 3. "Speech and Language Processing", Third Edition, Daniel Jurafsky, James H.Martin, Chapter 14: Question Answering and Information Retrieval, 2023.
Reference Book(s):	1. "Algorithms of the Intelligent Web", Haralambos Marmanis, Dmitry Babenko, Manning Publishers, 2011 2. "A Simple Guide to Retrieval Augmented Generation", Abhinav Kimothi, June 2025, ISBN 9781633435858, Manning publications
Course Outcome	• Implement an efficient index for document collection. • Perform searches on a document collection, rank, and evaluate results. • Apply machine learning techniques for Information Retrieval, Web, and multimodal applications. • Build and evaluate modern Question Answering systems.

Course Code	UE24CS342BA2	Course Title	Design of RTOS				
Program	B.Tech	Hours per week / Credit Assigned	L	T	P	S	C
			4	0	0	0	4
Semester	6	Type of Course	Elective Course				
Tools/Languages	ARM GCC, OpenOCD, GDB, C, Assembly	Desirable Knowledge	Operating Systems, Microprocessor architecture, C				
Prelude	Have you ever wondered what happens beneath the surface of a while(1) loop, or how a single processor can seemingly do ten things at once without a full operating system like Linux or Windows? The world is rapidly shifting toward smart, connected, and autonomous edge devices—from advanced robotics and automotive control units to wearable IoT sensors. These devices operate under strict physical constraints and unforgiving real-time deadlines, where a late computation is fundamentally equivalent to an incorrect one. In this course, you will peel back the layers of high-level software abstraction and interface directly with the silicon. You will not just learn how to use a Real-Time Operating System (RTOS); you will build one entirely from scratch. By electing to take this course, you will master bare-metal C programming, ARM Cortex-M architecture, and advanced hardware debugging, moving far beyond the safety net of standard libraries. Over the semester, you will engineer your own preemptive scheduler, context switcher, memory protection protocols, and inter-process communication primitives. Whether your career goals lie in firmware engineering, low-level OS kernel development, or architecting secure cyber-physical systems, this course will transform you from a software developer into a systems engineer capable of commanding hardware at its most fundamental level.						
Course Objectives:	- To master the foundational hardware-software interface, enabling students to develop bare-metal C programs, configure interrupt-driven peripheral drivers, and precisely profile hardware execution using cycle-accurate metrics. - To architect and implement the core components of a Real-Time Operating System, focusing on memory layouts, assembly-level context switching, and the mathematical principles of preemptive scheduling algorithms. - To design safe concurrency and synchronization mechanisms, resolving critical execution hazards such as priority inversion and deadlocks using mutexes and inter-process communication primitives. - To integrate advanced OS services including hardware memory isolation and dynamic energy management, and to critically benchmark the custom RTOS against industry-standard commercial implementations.						
Course Contents	Unit 1: Embedded Software Development & Profiling 14 hrs Introduction to Embedded Systems: Microcontrollers vs. Microprocessors. The constraints of embedded software. The RTOS landscape ARM Cortex-M4 Architecture: The RISC pipeline, register set (R0-R15), xPSR, and the memory map. The C Runtime & Bootstrapping: Cross-compilation, Linker Scripts, initializing .bss and .data segments. Memory-Mapped I/O: Using volatile pointers in C to manipulate hardware registers. Advanced Peripherals: Serial Communication (UART), SPI, and I2C protocols. Interrupts and Exceptions: The NVIC, Interrupt Service Routines (ISRs), and hardware preemption. Hardware Profiling: Using the Data Watchpoint and Trace (DWT) cycle counter to measure precise interrupt latencies. Unit 2: RTOS Core Architecture & Context Switching 14 Hrs The Process Model: Threads vs. Tasks. Defining the Task Control Block (TCB). Stack Frames & Exception Planting: Understanding the ARM exception stack frame. What hardware auto-pushes (R0-R3, R12, LR, PC, xPSR) vs. what software must save. Planting fake frames in C arrays. Anatomy of a Context Switch: The PendSV exception. Manipulating the Program Counter (PC) and Stack Pointer (SP). FPU Context & Lazy Stacking. Assembly Implementation: Writing inline ARM Thumb-2 Assembly for PendSV to push/pop registers and swap state. Preemptive Scheduling Concepts: Cooperative vs preemptive execution. Time slicing and priority bitmaps for O(1)						

	highest-priority selection. Real-Time Schedulability Theory: Rate Monotonic Scheduling (RMS), utilization bounds, and optimality proofs. Unit 3: Concurrency, Synchronization & IPC 14 Hrs Concurrency Hazards & Atomic Ops: Data corruption, race conditions, and ARM hardware atomic instructions (LDREX/STREX). Synchronization Primitives: Binary and Counting Semaphores. Moving tasks safely between "Ready" and "Blocked" lists. Mutual Exclusion & Priority Inversion: Mutex ownership tracking. The Mars Pathfinder bug and Priority Inheritance Protocol. Jitter & Response-Time Analysis: Sources of software jitter and computing worst-case response times. Inter-Process Communication (IPC): Designing thread-safe FIFO ring buffers. Deadlock & Safe Signaling: Coffman conditions, resource ordering, and ISR-safe signaling APIs without entering the scheduler directly. Unit 4: Memory Management, and Power 14 Hrs Hardware Memory Isolation: Threat modeling and using the ARM Memory Protection Unit (MPU) for task stack isolation. Heap Allocators in RTOS: First-fit fragmentation vs. deterministic Fixed-Block O(1) allocation. Software Timers & Watchdog: One-shot vs periodic timers, Timer Daemon task, and Independent Watchdog (IWDG) integration. Power Management: Tickless Idle theory—stopping the SysTick during idle periods via the WFI instruction and compensating the tick count upon wakeup. Industry Code Study & Benchmarking: Analyzing FreeRTOS source (tasks.c, queue.c). RHEALSTONE benchmark metrics. Testing & Reliability: MISRA-C 2012 guidelines, static analysis tools (cppcheck), and stress testing.
Text Book(s):	Jonathan Valvano, <i>Embedded Systems: Real-Time Operating Systems for ARM Cortex-M Microcontrollers</i>
Reference Book(s):	Joseph Yiu, <i>The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors</i>
Course Outcome	● Initialize and debug bare-metal systems, effectively configuring hardware descriptions and measuring exact interrupt latencies without relying on external operating systems. ● Write an ARM Thumb-2 assembly context switcher and implement a mathematically verifiable, preemptive, priority-based scheduler. ● Construct thread-safe FIFO queues, semaphores, and mutexes equipped with dynamic priority inheritance to safely coordinate shared resources in highly concurrent environments. ● Deploy a fault-tolerant, power-optimized RTOS utilizing the Memory Protection Unit (MPU) for stack isolation, and execute standardized performance benchmarks against commercial implementations.

Course Code	UE24CS342BA4	Course Title	Generative AI and its Applications				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -3				
Tools/Languages	Python, Hugging Face, LM Studio, Kaggle, Google Colab, AI tools- DeepEval, TruLens	Desirable Knowledge	Machine Learning				
Prelude	Generative Artificial Intelligence and Large Language Models are transforming the field of Artificial Intelligence by enabling machines to understand, reason, and generate human-like text, images, audio, and code. Technologies such as prompt engineering, Retrieval-Augmented Generation, parameter-efficient fine-tuning, multimodal models, and agentic workflows are driving innovations in healthcare, education, finance, legal systems, scientific research, and enterprise automation. These systems are redefining how intelligent applications are designed by combining retrieval, reasoning, planning, and autonomous decision-making capabilities. As organizations increasingly adopt LLM-based solutions, there is a growing need for engineers and researchers who understand not only the theoretical foundations of these models but also the practical techniques required to build reliable, scalable, and responsible AI systems. This course is designed to provide students with a strong foundation in modern Generative AI, covering NLP fundamentals, advanced retrieval architectures, fine-tuning methods, multimodal learning, agentic workflows, evaluation techniques, and ethical, privacy, and security considerations necessary for developing next-generation AI applications.						
Course Objectives:	- To provide a strong foundation in Natural Language Processing, Transformer architectures, and pre-trained language models that form the basis of modern Large Language Models. - To develop an understanding of prompt engineering, Retrieval-Augmented Generation, and advanced retrieval techniques for building accurate and grounded AI systems. - To introduce parameter-efficient fine-tuning methods and multimodal models for adapting LLMs to text, image, and audio tasks. - To design and implement agentic workflows involving planning, memory, tool use, and autonomous decision-making. - To evaluate Large Language Models and understand the ethical, privacy, and security considerations involved in deploying responsible AI systems.						

Course Contents	UNIT 1: NLP Basics, Pre-Trained Models Introduction to GenAI and its Applications, Large Language Models (LLMs), Evolution of LLMs. Tokenization-Introduction, Techniques-BPE, Word Piece, Sentence Piece, Numerical Problems. Word Embeddings - Introduction, Types of Word Embeddings: TF-IDF, Word2Vec – CBoW & SkipGram, GloVe, Numerical Problems. Transformer- Anatomy and Architecture, Numericals on Self and Cross Attention Score computation. Introduction to Pre-trained Models, Exploring BERT and GPT Architectures, Variants of BERT: ELMo, ROBERTa, BART. NLP Basics: Terminologies, N-Gram Models, POS, NER. NLP Tasks: Text Classification, Summarization, Question Answering, Machine Translation. 14 Hours UNIT 2: Prompt Engineering, Basic and Advance RAG Prompt Engineering – Overview and Applications. Prompt Structure, Hard and Soft Prompts, Prompt Engineering Techniques - Zero Shot, One Shot, Few Shot Prompting, Chain of Thought, Tree of Thought, Graph of Thought Prompting, Vector Stores RAG-Introduction, Need for RAG, Naïve RAG, Sparse Retrieval-BM25, Challenges Indexing – Flat Index, Inverted File Index, Hierarchical Navigable Small World, Product Quantization. Advanced RAG- Introduction, Need, Retrieval Enhancement Techniques – Introduction, Sparse Retrieval-BM25, Dense Retrieval-ColBERT/SBERT, Hybrid Retrieval using BM25 and ColBERT/SBERT, Re-Ranking, Query Expansion, Numericals on Hybrid Search, Generation Enhancement Techniques- Introduction, Techniques, Multi-Stage Retrieval, Advanced RAG Architectures including self-reflective, corrective, graph-based, multimodal, adaptive, and agentic retrieval systems for reliable generative AI applications. 14 Hours UNIT 3: Fine Tuning and Multimodal LLMs Fine Tuning - PEFT: Introduction to LLM Fine Tuning, Data Resolution, Precision, Quantization, LLM Pre-training, Fine Tuning Principles, Parameter-Efficient and Alignment-based Fine-Tuning Techniques for LLMs including LoRA, QLoRA, DoRA, SoRA, Adapter Tuning, Instruction Fine Tuning, Prompt Tuning, DPO, ORPO, GRPO, and Multimodal Fine-Tuning approaches. Introduction to Multimodal Models, Exploring CLIP and BLIP, Whisper AI–Architecture, Applications. Mixture of Experts 14 Hours UNIT 4: Agentic Workflow and Evaluation Agentic Development Life Cycle, Agentic Workflow and Design Patterns – Planning-based agent systems, Tool-augmented reasoning, Multi-step Task Decomposition, Memory-augmented agents, Agentic RAG, Self-Reflection and Self-Critique loops, Multi-agent collaboration and coordination, Hierarchical agents, Dynamic prompt generation, Function calling and Tool orchestration, Autonomous Decision-making pipelines, Iterative reasoning loops (ReAct-style), Workflow state management, Long-term memory integration, Agent evaluation and benchmarking, Frameworks for building Agentic Workflow- Lang Chain, LangGraph, Design of AI Harness, LLM Evaluation Techniques. Data behind LLM, LLM Ethics, LLM Security,Recent Developments in LLM. 14 Hours
Text Book(s):	1. Natural Language Processing with Transformers, Lewis Tunstall, Leandro von Werra, Thomas Wolf , O'Reilly Media, Inc., 2022 3. Designing Machine Learning Systems, Chip Huyen, O'Reilly Media, Inc., 2022 4. Hands-On Large Language Models, Jay Alamar and Maarten Grootendorst, O'Reilly Media, Inc., 2024.
Reference Book(s):	1. Natural Language Processing with Transformers, Lewis Tunstall, Leandro von Werra, Thomas Wolf, O'Reilly Media, Inc., 2022.

	2. Speech and Language Processing (3rd Edition Draft), Dan Jurafsky and James H. Martin, Stanford University, 2025. 3. Generative AI with Large Language Models, Shubham Saboo and co-authors, Packt Publishing, 2024. 4. Generative Deep Learning (2nd Edition), David Foster, O'Reilly Media, Inc., 2023. 5. Deep Learning, Ian Goodfellow, Yoshua Bengio, Aaron Courville, MIT Press, 2016. 6. Understanding Large Language Models: Learning Their Underlying Concepts and Technologies, Thimira Amaratunga, Apress, 2023. 7. Retrieval-Augmented Generation (RAG): Empowering Large Language Models (LLMs), Dr. Ray Islam, 2023. 8. Quick Start Guide to Large Language Models: Strategies and Best Practices for Using ChatGPT and Other LLMs, Sinan Ozdemir, Addison-Wesley Professional, 2023. 9. Pretrain Vision and Large Language Models in Python: End-to-End Techniques for Building and Deploying Foundation Models on AWS, Emily Webber, Packt Publishing, 2023. 10. Transformers for Natural Language Processing: Build Innovative Deep Neural Network Architectures for NLP with Python, PyTorch TensorFlow, BERT, RoBERTa, and More, Denis Rothman, Packt Publishing, 2023.
Course Outcome	• Explain the foundations of NLP, tokenization, embeddings, Transformer architecture, and major pre-trained language models. • Apply prompt engineering strategies and implement RAG pipelines using sparse, dense, and hybrid retrieval methods. • Fine-tune and adapt Large Language Models using PEFT techniques and describe the architecture and applications of multimodal models. • Design agentic workflows using frameworks such as LangChain and LangGraph with planning, memory, and tool orchestration. • Evaluate LLM-based systems and address ethical, privacy, and security challenges in real-world Generative AI applications

Course Code	UE24CS342BA5	Course Title	Blockchain				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -3				
Tools/Languages	Solidity, Ethereum, Hyperledger Fabric, Remix IDE, Truffle, Ganache, MetaMask	Desirable Knowledge	Data Structures & Its Applications (for coding)				
Prelude	This course offers a comprehensive understanding of Blockchain technology, covering its foundational concepts, data integrity, cryptographic techniques, and network structures. Students will explore key blockchain models, consensus algorithms, smart contracts, and second-generation applications like DApps and DAOs. Through hands-on demonstrations, including wallet creation and transactions, learners gain practical skills in working with platforms like Etherscan and Hyperledger Fabric. The course also addresses security challenges and real-world use cases across industries, preparing students for research and professional applications in blockchain.						
Course Objectives:	•To provide foundational knowledge of blockchain technology, including its architecture, data integrity principles, and key components such as nodes, tokens, and ledgers. •To explore various consensus algorithms and their roles in achieving distributed agreement in blockchain networks. •To introduce smart contracts and decentralized applications (DApps) using programming tools like Solidity and platforms such as Ethereum. •To examine blockchain security, vulnerabilities, and real-world use cases in sectors like finance, supply chain, and public administration.						
Course Contents	Unit 1: Introduction and Data flow and Integrity Key Blockchain Concepts, Nodes, Cryptocurrency, tokens, Public Ledger, Peer to peerNetwork, Types of Blockchain, Permissioned Blockchain model-Hyperledger Fabric: Architecture and core components, Permission-less Blockchain model- Demonstration of ether scan and Goerli/Sepoliaetherscan. Types of cryptography, Digital signatures, Hash functions, SHA-256, Patterns of hashing, Hash Pointer, Merkle tree. Case study: Bitcoin Blockchain Network, Creation of metamask wallets and performing transactions. 14 Hours Unit 2 : The structure of the network: consensus algorithm Introduction to distributed consensus: What, why, Challenges, Proof of Work, Proof of Stake, Delegated Proof of Stake, Proof of Authority, Proof of Elapsed Time, Proof of Space, Proof of Burn, RAFT, PAXOS, Byzantine Fault Tolerance System, Practical BFT. Smart contracts: origins and how they function, Understanding a sample contract, Decentralized applications, Decentralized Autonomous Organizations (DAOs)- Need, Principal agent dilemma, components, The DAO Story, Legality of DAPPs and DAOs, Legal challenges (India/global). 14 Hours						

	Unit 3: Second generation applications of Blockchain technology Solidity- Variable, Functions, Arrays/Strings, enum/structs, mapping, storage/memory, function visibility - public, private, internal, external, modifiers, view, pure, fallback, overloading, in-built mathematical cryptography functions. 14 Hours Unit 4: Blockchain Application Development, Security, and Performance Optimization SOLIDITY- Withdrawal pattern, Restricted Access, ether units. Creation of a simple DAPP. Gas mechanism in Ethereum, basic gas optimization techniques, Scalability solutions – Rollups (Optimistic and ZK), Sidechains vs Layer 2, Implications for decentralized application deployment. Blockchain vulnerabilities, Smart contract vulnerabilities, Blockchain based DNS security platform, Use cases: Public Sector, Finance, Supply Chain. Research Aspects in Blockchain 14 Hours
Text Book(s):	1. Introduction to Blockchain Technology, Tiana Laurence, 1st edition, Van Haren Publishing, 2019. 2. Blockchain Technology from Theory to Practice, Sudeep Tanwar, 1st edition, Springer, 2022.
Reference Book(s):	1. Hands-On Cyber security with Blockchain: Implement DDoS protection, PKI-based identity, 2FA, and DNS security using Blockchain, Rajneesh Gupta, 1st edition, 2018. 2. Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction, Narayanan, Bonneau, Felten, Miller and Goldfeder, Princeton University Press, 2016. 3. Drescher, Daniel. Blockchain Basics: A Non-Technical Introduction in 25 Steps. Germany: Apress, 2017.
Course Outcome	• Understand and explain the structure, types, and functioning of blockchain systems and cryptographic techniques used for data integrity. • Analyze and compare different consensus mechanisms and their applicability in distributed networks. • Design, code, and deploy smart contracts and DApps using Ethereum-based tools like Metamask and Solidity. • Evaluate blockchain's impact and applicability in various industries, along with identifying and mitigating potential security vulnerabilities.

Course Code	UE24CS342BA6	Course Title	Digital Forensics and Incident Response				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -3				
Tools/Languages	Open source tools on Forensics.Cellebrite,Deepfake Detection Tools	Desirable Knowledge	Machine Learning				
Prelude	Digital Forensics is the core set of principles and processes necessary to produce usable digital evidence and uncover critical intelligence. This course provides a deep understanding of techniques to gather, protect, analyze, and report digital confirmations through a combination of theoretical foundation and practical exercises. The content is organized into four units, each with a focused objective and corresponding outcomes						
Course Objectives:	- Introduce the history, evolution, basic principles, challenges, and incident response techniques in digital forensics. - Develop expertise in Windows file systems, including file structure analysis, registry examination, and related forensic recovery tools. - Build foundational skills in Linux and macOS systems while integrating network forensics and browser-based analysis. - Equip students with specialized knowledge in email forensics, mobile device investigations, anti-forensics techniques, and professional report writing.						
Course Contents	Unit 1: Introduction to Digital Forensics and Incident Response Foundations of Digital Forensics: Introduction to forensic science and digital forensics, Goals, categories, forensic readiness, life cycle, and challenges, Role of the examiner; types of investigations; importance of international standards Incident Response: Attack lifecycle and introduction to incident response, Building an Incident Response Team and drafting an Incident Response Plan (IRP), Forensic tools and methodologies for initial response, Investigations Triad Evidence Acquisition and Analysis: Types of digital evidence (HDD, SSD, RAM), Data recovery, allocated vs. unallocated space, and persistence (page/swap files), Major Concepts, Essential Technical Concepts Initial responder tasks: search and seizure, documentation, packaging, and transporting evidence Labs & Activities: - Activity 1: Understanding and familiarization with Digital Forensic - Lab 1: Acquiring digital evidence (HDD/SSD and RAM) with open-source tools - Lab 2: Analysing digital evidence (HDD/SSD and RAM) with open-source tools 14 Hours Unit 2: Windows Forensics – Filesystems and Registry Analysis Windows File Systems: Overview of FAT and NTFS file systems, Examination of file allocation structures and metadata, Techniques for timeline analysis and file recovery Windows Registry Analysis: Key artifacts: Deleted data, hibernation file (Hiberfile.sys), print spooling, recycle bin, metadata, thumbnail cache, Most Recently Used (MRU) lists, restore points, and shadow copies, Methods for recovering deleted registry keys and performing registry forensics Labs & Activities: - Activity 2: Case study using tools like Autopsy for practical Windows forensics - Lab 3: Data recovery and forensic analysis using EXIF Tool - Lab 4: Examination of the Windows Registry with REGEDIT and open-source forensic tools 14 Hours Unit 3: Linux, macOS, Network, and Browser Forensics Linux Forensics: Linux file systems (Ext2/Ext3), structure layers (file, metadata, data unit), and artifacts, Concepts such as deleted data recovery, logical volume management, boot processes, and principles of file carving macOS Forensics: Exploration of OS X file system artifacts, including HFS+ structures and OS X system logs Network Forensics: Fundamentals of networking and types of networks, Techniques for securing network infrastructures, analyzing network evidence, and applying forensic science to networks,						

	Tools and processes for the seizure and analysis of networking devices Browser Forensics: Overview of web browsers (IE, Firefox, Chrome), Forensic analysis of browser artifacts and investigation tools Labs & Activities: • Activity 3: Practical session on understanding Linux artifacts using both CLI and GUI tools • Lab 5: Analysis of PCAP files and network forensic investigations • Lab 6: Browser forensic investigations using open-source tools 14 Hours Unit 4: Email, Mobile Forensics, Anti-Forensics, and Report Writing Email Forensics: Examination of email communications, protocols, and header analysis, Techniques for recovering and analyzing email evidence Mobile Device Forensics: Architecture, boot processes, and the forensic significance of cellular network data, Procedures related to SIM file systems, device locks, rooting, and jailbreaking Methods for evidence acquisition and challenges associated with mobile devices Anti-Forensics: Overview and classification of anti-forensics techniques, Practices including data wiping, shredding, obfuscation, encryption, and data hiding Report Writing: Preparing and structuring forensic reports best practices for report design, documentation, legal acceptance, and presenting digital evidence Labs & Activities: • Lab 7: Email forensics using open-source tools • Lab 8: Demonstration of mobile forensics techniques Activity 4: Collaborative report writing exercise and peer review 14 Hours
Text Book(s):	1. "The basics of digital forensics: the primer for getting started in digital forensics", Sammons, J. Elsevier, 2012. 2. "Digital Forensics Basics: A Practical Guide Using Windows OS", Hassan, N. A. A press 2019. 3. "Practical Cyber Forensics- An Incident-Based Approach to Forensic Investigations", by Niranjana Reddy, A Press, 2019. 4. "Introductory Computer Forensics-A Hands-on practical Approach", by Xiaodong Lin, Springer, 2018.
Reference Book(s):	1. "Guide to Computer Forensics and Investigations", Bill Nelson, Amelia Phillips and Christopher Steuart, Course Technology, Cengage Learning, 2024 2. "Digital forensics with open-source tools", Altheide, C., & Carvey, H. Elsevier, 2011. 3. "Digital Forensics Workbook _Hands-on Activities in Digital Forensics", Michael K Robinson, CreateSpace Independent Publishing Platform, 2015.
Course Outcome	• Students will understand the fundamental principles of digital forensics, be familiar with its historical evolution, and perform initial incident response and evidence acquisition. • Students will be able to analyze Windows file systems and registry artifacts effectively, perform timeline analyses, and recover deleted data. • Students will acquire and analyze forensic data from Linux and macOS environments and demonstrate proficiency in network and browser forensics. • Students will conduct mobile device and email investigations using proper tools, recognize anti-forensics practices, and compile legally sound forensic reports.

Course Code	UE24CS342BA7	Course Title	Digital Twin				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
AI Tools /Tools/Languages	Co-Pilot	Desirable Knowledge	C, Python, C++ Programming				
Prelude	Digital Twins are revolutionizing how we design, monitor, and optimize complex systems by creating dynamic virtual replicas of physical entities. This course introduces the foundational concepts, modeling techniques, and simulation strategies needed to build and analyze Digital Twins across domains such as healthcare, manufacturing, and transportation. Students will gain hands-on experience with tools like Unity, Blender, and OpenGL, while also exploring critical topics like 3D view synthesis and cybersecurity. By the end of the course, students will be equipped to conceptualize, build, and secure intelligent Digital Twin systems.						
Course Objectives:	- Develop fundamental understanding of Digital Twins and its architecture. With help of case studies provide a structured way to conceptualize any Digital Twin. - Develop understanding on various techniques for structural twinning and 3D view Synthesis. - To build skill in modelling-based system engineering, focusing on discrete event systems - Collaborate with peers on practical projects to gain hands-on experience in developing intelligent systems. Develop understanding about secure designs and privacy concerns in the context of Digital twin						
Course Contents	Unit 1: Digital Twin Essentials The Big Picture of Digital Twins, History of the Digital Twin, Origin of the Digital Twin concept, Digital Twin and Product Life cycle Relationship, Types of Digital Twin: Discrete Vs Composite, Product versus facility, Types of Digital Twin: Simulation versus operational, Analytics versus physics-based, Characteristics of a Digital Twin, Digital Twin Architecture, Industrial Digital Twin applications, Examples of mock, functional, and executable twins, Data Driven Modelling, Physics Driven Modelling, Hybrid (Data-Physics) Modelling, Examples of Physics Driven Modelling: DT of a Propeller of a Drone, DT of a ceiling fan, Bio-Mechanical DT of human body; Examples of Data Driven Modelling: DT for RUL of a Battery, DT for traffic Mobility in Bangalore, DT of RUL of GUT; Hybrid Modelling: Heart Digital Twin, DT for Solar Array, DT for CAR using OBD., Case Study: Manufacturing, Case Study: Healthcare, Case Study: Buildings, Case Study: Transportation & Logistics 14 Hours Unit 2: Foundations and Techniques in 3D View Synthesis Manual 3D modelling and mesh editing, Point cloud to mesh conversion, Poisson surface reconstruction, Ball-pivoting algorithm, Alpha shapes and Delaunay triangulation, Camera models and calibration, Epipolar geometry and depth estimation, Structure from Motion (SfM), Multi-View Stereo (MVS), Depth Image-Based Rendering (DIBR), Forward and backward image warping, Neural Radiance Fields (NeRF) fundamentals, Volume rendering with MLPs, 3D Gaussian Splatting for real-time rendering 14 Hours Unit 3: System Modelling and Simulation Introduction to Simulation, System and System Environment, Discrete Event System Simulation, General Principles, The Event Scheduling/Time Advance Algorithm, Input Modelling, Estimation of Absolute Performance, Verification, Calibration and Validation 14 Hours Unit 4: Digital Twin & Cyber Security. Digital twins and cybersecurity, Security Framework, Digital twins threat modelling, Common attacks						

	on digital twins, Common attacks on digital twins, Digital twin authentication and identification challenge, IDS, IPS, Authentication Methods, Communication Channel Protection, building cyber resilience in digital twins, Privacy Framework, Lack of Privacy, and trust, Privacy by Design, Enhancing trust with block chain integration. 14 Hours
Text Book(s):	1. Building Industrial Digital Twins by Shyam Varan Nath & Pieter van Schalkwyk, by Packt Publishing Ltd. 2. Jerry Banks, John S. Carson II, Barry L. Nelson, David M. Nicol: Discrete-Event System Simulation, 5th Edition, Pearson Education, 2010. 3. Girod, Bernd, Günther Greiner, and Heinrich Niemann, eds. Principles of 3D image analysis and synthesis. Vol. 556. Springer Science & Business Media, 2013
Reference Book(s):	1. Diab, W. W., A. Ferraro, B. Klenz, S. W. Lin, E. Liongosari, W. E. Tannous, and B. Zarkout. "Industrial IoT Artificial Intelligence Framework." (2022): 1-59. 2. El Saddik, Abdulmotaleb, ed. Digital Twin for Healthcare: Design, Challenges, and Solutions. Elsevier, 2022. Web References: 1. https://in.mathworks.com/campaigns/offers/next/digital-twins-for-predictive-maintenance/modeling-methods.html 2. https://learn.microsoft.com/en-us/training/paths/build-mixed-reality-azure-digital-twins-unity/
Course Outcome	• Students will be able to conceptualize a Digital Twin and build its characteristics and input output parameters • Students will be able to use either manual or algorithmic techniques to model the structural twin of the DT. • Students will be able to translate the conceptualized Digital twin application into a Simulation model – a Time advanced discrete event system simulation. • Students will be able to collaborate effectively with peers on practical projects and demonstrate the ability to apply Digital Twin and Extended Reality solutions to real-world problems, such as improving operational efficiency, enhancing security, and reducing costs.

Course Code	UE24CS342BA8	Course Title	Responsible AI				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
Tools/Languages		Desirable Knowledge					
Prelude	AI is transforming cybersecurity through intelligent threat detection and automation. However, it also introduces risks such as bias, misuse, and adversarial attacks. This course explores both AI capabilities and responsible, secure usage in cybersecurity.						
Course Objectives:	- To introduce core principles of Responsible AI, including fairness, accountability, transparency, safety, and trustworthiness across the AI lifecycle. - To analyze real-world risks in AI systems, including bias, privacy leakage, misalignment, and societal impacts. - To understand security challenges in AI, including adversarial machine learning and vulnerabilities in generative AI systems. - To equip students with techniques for designing robust, secure, and reliable AI systems. - To familiarize students with AI governance, ethical practices, and regulatory frameworks such as the Digital Personal Data Protection Act 2023 and General Data Protection Regulation.						
Course Contents	Unit 1 – Foundations, lifecycle and real-world risks Responsible AI principles – Fairness, accountability, transparency, safety, and trustworthiness; Societal impact of AI systems across domains; AI system lifecycle – Design, development, deployment, monitoring, and continuous evaluation of AI systems; Integration of Responsible AI principles across lifecycle stages; Interpretability and Transparency – Understanding how AI systems make decisions and making decision processes explainable to humans; Evaluating explainability methods and their limitations; Consistency and Robustness – Ensuring predictable behavior across inputs; Identifying instability and edge-case failures; Alignment Problem – Ensuring AI systems behave according to human intent; reward mis-specification and specification gaming; Advanced AI risk concepts – Instrumental Convergence (power-seeking, deception, resource acquisition behaviors), goal misgeneralization, unintended optimization; Understanding AGI – Definition, capabilities, current progress, and debates on timelines and feasibility; Privacy and Fairness in AI – Bias sources, fairness metrics, trade-offs; privacy risks in data-driven systems; Human-AI interaction risks – Automation bias, over-reliance, hallucination acceptance, trust calibration; Real-world AI failures – Case studies across sectors highlighting bias, privacy violations, and unsafe deployment 14 Hours Unit 2:Adversarial Machine Learning – attacks and defenses Introduction to adversarial machine learning – Threat models, attacker goals, and attack surfaces in ML systems; Evasion attacks – Crafting adversarial examples to mislead models at inference time; Poisoning attacks – Training-time attacks through data manipulation and backdoor insertion; Model extraction and inversion attacks – Stealing model parameters and inferring sensitive training data; Membership inference attacks – Privacy leakage from trained models; Robustness and generalization issues – Sensitivity of models to perturbations and distribution shifts; Defense strategies – Adversarial training, input preprocessing, certified robustness, and anomaly detection; Secure ML pipeline design – Data integrity, training safeguards, and validation mechanisms;						

	Evaluation of adversarial robustness – Metrics and testing strategies for secure ML systems 14 Hours Unit 3: LLM vulnerabilities and security in Generative AI systems Introduction to LLM-specific security risks – Differences between traditional ML and Generative AI threat models; Prompt injection attacks – Direct and indirect injection, instruction override, and context manipulation; Jailbreaking techniques – Bypassing safety mechanisms and generating restricted outputs; Data leakage and privacy risks – Extraction of sensitive information through prompts and context exploitation; Hallucinations and reliability failures – Confident incorrect outputs, fabrication, and reasoning errors; Context handling vulnerabilities – Instruction drift, long-context failures, and prioritization errors; Security risks in tool-integrated and agent-based systems – Tool misuse, cross-component attacks, and multi-agent vulnerabilities; AI-assisted offensive security – Use of LLMs in generating attacks, phishing, and exploit development; Mapping LLM threats using MITRE ATLAS; Defense strategies – Input/output filtering, prompt hardening, secure system design, monitoring, and red teaming of LLM systems; Case studies of advanced AI systems (e.g., Claude Mythos) to analyze emergent behavior, alignment challenges, and security vulnerabilities. 14 Hours Unit 4: AI Governance, Ethics and societal impact Regulatory and governance landscape for AI – Digital Personal Data Protection Act 2023 and Recent Developments; General Data Protection Regulation, EU AI Act, US AI policy directions and executive initiatives Compliance, auditing, and accountability – Risk classification, documentation, auditing mechanisms, and governance processes; Ethical design and deployment practices – Ethical approvals, informed consent, participatory design, and stakeholder inclusion; Responsible AI in key domains: Legal domain – Bias, explainability, accountability, Healthcare domain – Safety, fairness, privacy, reliability, Education domain – Personalization risks and over-automation, Other domains – Finance, hiring, and governance systems Policy issues in Responsible AI – Innovation vs regulation, accountability, liability, and ethical trade-offs across domains; Responsible AI in the Indian context – Public sector AI, data governance challenges, digital inclusion, and regulatory considerations; Future of work and AI – Impact of automation, human-AI collaboration, workforce transformation, and ethical implications; Sustainable and cost-aware AI deployment – Efficient system design, resource optimization, and environmental considerations. 14 Hours
Text Book(s):	1. Virginia Dignum, “Responsible Artificial Intelligence: How to Develop and Use AI in a Responsible Way” Springer Nature, 04-Nov-2019; ISBN-10 : 3030303705, ISBN-13 : 978-3030303709 2. Christoph Molnar “Interpretable Machine Learning”.Lulu, 1st edition, March 24, 2019; eBook. ISBN-10 : 0244768528, ISBN-13 : 978-0244768522 [available online]
References	Practical AI Security No Starch Press https://www.bcs.org/media/e2ih402j/bcs-foundation-award-in-understanding-the-role-of-ethics-in-the-responsible-use-of-ai-syllabus.pdf https://exec.cs.cmu.edu/custom/responsible-ai https://onlinecourses.nptel.ac.in/noc24_cs132/preview https://scholar.google.com/citations?hl=en&user=G9pIW1AAAAAJ&view_op=list_works&sortby=pubdate https://llm-vulnerability.github.io/ https://portswigger.net/web-security/llm-attacks https://github.com/harriethacks/aisecurityhandbook

Course Outcome	● Apply Responsible AI principles in the design and deployment of AI systems. ● Identify and evaluate risks such as bias, privacy violations, and robustness issues in AI models. ● Analyze and mitigate adversarial attacks and security vulnerabilities in machine learning and LLM systems. ● Design AI solutions that are secure, interpretable, and aligned with human intent. ● Assess AI systems in the context of legal, ethical, and societal requirements.
-----------------------	--

Course Code	UE24CS342BA9	Course Title	Quantitative Trading Systems				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
Tools/Languages	C++ (C++17/20), Python, Google Benchmark, CMake, perf	Desirable Knowledge	Data Structures, C++, OS, Linear Algebra, Probability				
Prelude	This course is built for CSE students with an engineering first mindset. The emphasis is 80% quant development (systems, implementation, C++ performance, financial algorithms in code) and 20% finance theory. By the end, students will have an in-depth understanding of financial systems that operate on nanosecond latency scales.						
Course Objectives:	• Explain exchange microstructure and design cache-efficient limit order books in C++. • Implement core statistical estimators, Monte Carlo engines, and price options. • Design and implement a fully event-driven backtesting framework. • Build lock-free data structures and understand HFT systems and market making.						
Course Contents	Unit 1 – Systems Foundations & Financial Data Structures Market Microstructure & the Order Book, C++ Memory Model for Performance, Limit Order Book Implementation in C++, NASDAQ ITCH 5.0 Feed Handler. 14 Hours Unit 2: Quantitative Methods & Statistical Engines Probability, Statistics & Numerical Methods, Time Series Analysis: Stationarity, AR Models & Volatility, Options Pricing: Black-Scholes, Statistical Arbitrage: Cointegration to Signal.. 14 Hours Unit 3: Strategy Infrastructure & Back testing Systems Backtesting Engine Architecture, Momentum & Pairs Trading Strategies, Performance Analysis & Avoiding Overfitting (Deflated Sharpe Ratio, Purged K-Fold CV). Case studies of advanced AI systems (e.g., Claude Mythos) to analyse emergent behaviour, alignment challenges, and security vulnerabilities. 14 Hours Unit 4: HFT Systems, Market Making & Capstone Concurrency & Lock-Free Data Structures (SPSC Ring Buffer), Kernel Bypass Networking & HFT Infrastructure, Avellaneda-Stoikov Market Making Model, Systems Design. 14 Hours						
Text Book(s):	1. Harris, L. (2003). Trading and Exchanges: Market Microstructure for Practitioners. 2. Meyers, S. (2014). Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14. 3. Hull, J. C. (2022). Options, Futures, and Other Derivatives.						
References	1. Fog, A. (2023). Optimizing software in C++. 2. Cartea, Á., Jaimungal, S., & Penalva, J. (2015). Algorithmic and High-Frequency Trading. 3. de Prado, M. L. (2018). Advances in Financial Machine Learning.						
Course Outcome	• Explain exchange microstructure and build cache-efficient limit order books in C++. • Implement core statistical estimators, Monte Carlo engines, and options pricing models. • Design a robust event-driven backtesting engine to evaluate algorithmic strategies. • Develop lock-free concurrency structures and evaluate ultra-low latency HFT systems.						

Course Code	UE24CS342BA10	Course Title	Quantum ML & AI				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
Tools/Languages	PennyLane and Qiskit	Desirable Knowledge	Linear Algebra				
Prelude	Quantum Machine Learning (QML) sits at the confluence of two transformative disciplines: quantum information processing and modern machine learning. This advanced course equips students to design, implement, and critically evaluate supervised quantum learning algorithms on near-term NISQ hardware. Beginning with quantum data-encoding strategies, the course progresses through variational quantum circuits, kernel methods, and quantum neural networks before broadening to generative models and reinforcement learning. Theoretical rigour is maintained throughout, with careful attention to complexity, sample complexity, and the prospects for genuine quantum advantage. Hands-on programming in PennyLane and Qiskit bridges mathematical formalism with practical deployment, preparing graduates for research and industry at the quantum-AI frontier.						
Course Objectives:	- To introduce students to the theoretical foundations of quantum information processing and its application to machine learning tasks, including complexity analysis and data encoding paradigms. - To enable students to understand and apply quantum-enhanced inference and training techniques such as kernel methods, amplitude amplification, and adiabatic optimization. - To equip students with the knowledge to design and evaluate quantum learning models, including Ising-type formulations, variational classifiers, and hybrid architectures. - To develop proficiency in building, training, and optimizing Quantum Neural Networks using Parameterized Quantum Circuits with practical tools like PennyLane and Qiskit - To familiarize students with real-world challenges in quantum ML deployment, including noise modeling, error mitigation, and the design of scalable hybrid classical-quantum pipelines.						

Course Contents	Unit 1: Quantum Data Encoding Introduction, Background, How quantum computers can classify data, Computational complexity of learning, Sample complexity, Model complexity, Information Encoding: Basic Encoding, Amplitude Encoding, Qsample Encoding, Hamiltonian Encoding. Encoding Strategies in PennyLane 14 Hours Unit 2: Quantum Computing for Inference and Training Linear Models, Kernel Methods, Probabilistic Models, Quantum Bias, Search and Amplitude Amplification, Hybrid training for Variational Algorithms, Quantum Adiabatic Machine Learning. 14 Hours Unit 3: Learning with Quantum Models Quantum Extensions of Ising-Type Models, Variational Classifiers and Neural Networks, Other Approaches to Build Quantum Models. 14 Hours Unit 4: Quantum Neural Networks Parameterised Quantum Circuits (PQC) & Ansatz Design, Gradient Estimation — Parameter-Shift Rule, Quantum Neural Networks (QNN) — Architecture, Training QNNs — Optimisation Landscape, Hybrid Classical-Quantum Pipelines, Noise Models & Error Mitigation, VQC Classification & Regression in PennyLane / Qiskit. 14 Hours
Text Book(s):	1. M. Schuld and F. Petruccione, Supervised Learning with Quantum Computers, Springer, 2018. ISBN: 978-3-319-96423-2. 2. M. Schuld, Machine Learning with Quantum Computers (with Petruccione), Springer, 2nd ed., 2021. ISBN: 978-3030830977
References	1. M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, Cambridge University Press, 10th Anniversary Edition, 2010. ISBN: 978-1107002173. 2. P. Wittek, Quantum Machine Learning: What Quantum Computing Means to Data Mining, Academic Press, 2014. ISBN: 978-0128009536. 3. W. Humble, A. McCaskey et al., Quantum Computing: An Applied Approach (J. Hidary), Springer, 2nd ed., 2021. ISBN: 978-3030832742
Research & Web References	• Cerezo et al. (2021), 'Variational quantum algorithms', Nature Reviews Physics 3, 625-644. • Biamonte et al. (2017), 'Quantum machine learning', Nature 549, 195-202. • Huang et al. (2021), 'Power of data in quantum machine learning', Nature Communications 12, 2631. • PennyLane QML Demos: https://pennylane.ai/qml/demonstrations • Qiskit Machine Learning Documentation: https://qiskit-community.github.io/qiskit-machine-learning
Course Outcome	• Encode and represent classical datasets using quantum encoding schemes (Amplitude, Qsample, Hamiltonian) and implement them programmatically in PennyLane. • Analyze and compare quantum and classical approaches to supervised learning, kernel methods, and probabilistic inference, articulating the advantages and limitations of each. • Design quantum learning models using variational circuits, Ising-type formulations, and hybrid architectures tailored to specific classification and regression problems. • Train and optimize Quantum Neural Networks by applying the Parameter-Shift Rule for gradient estimation and navigating the challenges of barren plateaus in quantum optimization landscapes.

	• Implement and evaluate end-to-end quantum ML pipelines using Qiskit and PennyLane, incorporating noise-aware models and error mitigation techniques for near-term quantum hardware.
--	---

Course Code	UE24CS343BB1	Course Title	Heterogenous Parallelism				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -4				
Tools/Languages	Pthread, OpenMP, CUDA, Vivado HLS	Desirable Knowledge	Problem Solving with C, Microprocessor and Computer Architecture				
Prelude	This course focuses on parallel heterogeneous architecture as well as programming models and imparts pragmatic skills to program using parallel programming languages and frameworks trending in the industry.						
Course Objectives:	- Familiarize with various parallel heterogeneous architectures ILP, and Optimization techniques and cache-efficient programming. - Explore multi-core architectures, parallel algorithms, and GP-GPU-based computing models. - Learn OpenMP, memory models, and Acquaint with Concurrency Bugs and Resolution debugging techniques for parallel programming. - Study CUDA programming, hardware accelerators (FPGAs & TPUs), and to explore HLS programming						
Course Contents	Unit 1:Fine Grained Parallelism Review on Parallelism and Performance, Instruction Level Parallelism and Enhancement Techniques, Prediction and Speculation, Cache optimized, Cache coherency, Programming. Laws of Parallelism, Data, Task and Pipeline Parallelism, Pthreads. 14 Hours Unit 2: Coarse Grained Parallelism Parallel Algorithms for regular architecture – Mesh, Pyramids, Task Decompositions and Mapping, Multithreaded and Multi-Core architectures, Many-Core Heterogeneous Architectures, GPU Architectures. 14 Hours Unit 3:Parallelism Frameworks OpenMP, Race Conditions, Deadlocks & Debugging, Memory Models for Parallel Programming, Memory Consistency Models. 14 Hours Unit 4: GPU Programming & Hardware Accelerators CUDA Programming Frameworks, Introduction to Nvidia/SIMT, AI math(matrix multiplication) using systolic arrays and TPU.Hardware acceleration platforms (FPGAs and TPUs), Architecture and organization of modern FPGAs, HLS Programming, Case Studies. 14 Hours						

Text Book(s):	1. "Multiprogramming and GPU Programming – An Integrated approach" 2nd Edition by Gerassimos Barlas- Morgan Kaufmann Publications, 2022 2. "Parallel Programming: for Multicore and Cluster Systems", Thomas Rauber, Gudula Runger, 3rd Ed., Springer, 2023.
Reference Book(s):	1. "Computer Architecture: A Quantitative Approach: John Hennessy, David Patterson", 6th Edition, Morgan Kaufmann, 2017. 2. "Programming Massively Parallel Processors", David Kirk and Wen-mei Hwu, 3rd Ed, Morgan Kaufmann, 2016. 3. "Computer Systems: A Programmer's Perspective", Randal E. Bryant, David R. O'Hallaron, Pearson, 2nd Ed, 2016. 4. "Parallel Programming for FPGAs", Ryan Kastner, Janarбек Matai, and Stephen Neuendorffer, Creative Commons, 2018. 5. Introduction to Parallel Computing, Second Edition-Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar-Pearson Publication, 2003
Course Outcome	• Understand the underpinnings of Parallel Heterogeneous Architectures and Techniques, Memory Consistency Models and Coherence Techniques. • Develop and analyze parallel algorithms using many-core architectures. • Program using popular Parallel Programming Frameworks and Languages trending in the industry. • Implement parallel programs on GPUs and analyze real-world acceleration techniques.

Course Code	UE24CS343BB2	Course Title	Topics in Deep Learning				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective IV				
Tools/Languages	Pytorch, Tensorflow, Kaggle, Google Colab, Hugging Face models	Desirable Knowledge	Machine Learning				
Prelude	Deep Learning has become a key technology in Artificial Intelligence and is widely used by companies such as Google, Microsoft, IBM, Meta, etc.. for solving problems in Computer Vision and Natural Language Processing. This course introduces the fundamental concepts and architectures of Deep Learning, including neural networks and modern deep architectures used in Vision and NLP applications. By the end of the course, students will gain knowledge of designing and applying Deep Learning models to real-world problems.						
Course Objectives:	- To understand the fundamental concepts, architectures, and optimization techniques used in Deep Learning. - To analyze and implement deep learning models for Computer Vision, Natural Language Processing, and generative tasks. - To understand advanced generative and representation learning models for modern AI applications. - To explore advanced learning paradigms including Reinforcement Learning, Federated Learning, TinyML, and Explainable AI.						
Course Contents	Unit 1: Introduction to Deep Learning: Activation functions, loss functions- Loss function- For Balanced Datasets, Categorical Cross-Entropy Loss, Sparse Categorical Cross-Entropy, Hinge Loss for Imbalanced Datasets- Weighted Cross-Entropy Loss, Focal Loss, Dice Loss, Batch Normalization, Regularization and Optimization, Convolution Architectures - Alexnet, ZFNet, VGGNet, GoogleNet, ResNet. RCNN, FRCNN, Faster RCNN, YOLO. Transfer Learning: Introduction, Motivation, Variations, TL Architecture of CNNs. Mixture of Experts. 14 Hours Unit 2: Recurrent Neural Networks (RNN) and Transformers: Introduction-Recurrent Neurons, Memory Cells, Variable-Length Input-Output Sequences, RNN Architecture, Sequence learning problem, BPTT-Back Propagation Through Time, truncated BPTT, Vanishing and Exploding Gradient, Bidirectional RNN, LSTM Cell and GRU Cell, Text Classification with RNN, Encoder/Decoder architecture, Seq2Seq model with Attention, Transformer architectures including self-attention, encoder-decoder models, Introduction to Vision Transformers 14 Hours Unit 3: Generative Models and Advanced Representation Learning Introduction to Autoencoders, Regularization in autoencoders, Denoising autoencoders, Sparse autoencoders, Contrastive autoencoders, Variational Auto Encoders(VAEs). Generative Adversarial Networks(GANs)- Architecture and Training Methods, Image Generation, DCGAN, Style GAN, WGAN, Applications. Diffusion Model, Stable diffusion architectures. Meta Learning: Introduction to Meta Learning, MAML, FOMAML, Adaptive Neural inductive Learning, Siamese Networks. Graphical Neural Networks (GNN): Introduction to GNNs, Graph Convolution Networks, Applications. 14 Hours Unit 4: Advanced Learning Paradigms: Reinforcement Learning, Federated Learning, TinyML, and Explainable AI Introduction, Basic Framework of RL, Learning to Optimize Rewards, Credit Assignment Problem,						

	Temporal Difference, Learning and Q Learning. Deep RL: Deep Q Learning. Federated Learning: Horizontal, Vertical and FTL (Federated Transfer Learning). Tiny ML: Model optimization (quantization, pruning, knowledge distillation), resource-efficient neural networks, and data handling for edge devices (sensor-based time-series data preprocessing). Explainable AI (XAI) – Introduction and need for interpretability, model transparency vs explainability, feature importance methods, SHAP, LIME, saliency maps, Grad-CAM, attention visualization, rule-based explanations, surrogate models, counterfactual explanations, evaluation metrics for explainability. 14 Hours
Laboratory	NA
Text Book(s):	1. “Advanced Deep Learning with Python” - Ivan Vasilev, Packt Publishing, 2019. 2. “Neural Network and Deep learning” by Charu C Agarwal. Springer International Publishing, 2nd edition, 2023.
Reference Book(s):	1. “Hands-on Machine Learning with Scikit-Learn and TensorFlow”, Aurelian Geron, O’REILLY, 3rd Edition, 2022. 2. “Pattern Recognition and Machine Learning”, Christopher Bishop, Springer, 1st Edition, 2016 (Reprint). 3. Handouts: Transfer Learning / Latest Deep Learning Techniques / Vision Transformers/ GPT / FL 4. https://www.datacamp.com/blog/what-is-tinyml-tiny-machine-learning 5. Tiny Machine Learning Quickstart (2025) – Simone Salerno 6. “<i>Agents of Chaos</i>” Natalie Shapira et al. (38 authors, including David Bau, Maarten Sap, Reuth Mirsky, Tomer Ullman), arXiv (preprint server), DOI: 10.48550/arXiv.2602.20021, February 23, 2026
Course Outcome	• Apply deep learning fundamentals, CNN architectures, and transfer learning techniques for vision-based applications. • Design and implement RNNs, LSTMs, Transformers, and attention-based models for sequential and NLP tasks. • Analyze and utilize generative and representation learning models such as Autoencoders, GANs, VAEs, and GNNs. • Evaluate and apply advanced learning paradigms including Reinforcement Learning, Federated Learning, TinyML, and Explainable AI techniques for intelligent systems.

Course Code	UE24CS343BB3	Course Title	Database Technologies				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -4				
Tools/Languages	MySQL, postgres, Oracle, Apache Spark, Apache Kafka, Amazon Kinesis.	Desirable Knowledge	Database Management Systems.				
Prelude	The last decade saw enormous advancements in the design of large-scale data processing systems due to the rise of Internet services. This course covers the architecture of modern data management systems. Topics include storage management, query optimization, distributed, parallel and stream processing, with a focus on the key design ideas shared across many types of data intensive systems.						
Course Objectives:	• Data Storage Techniques, Indexing Mechanisms. • Query Processing and Query Optimization Techniques. • Parallel and Distributed Databases. • Data Streaming Systems, current Trends in Database Systems.						
Course Contents	Unit 1- Relational Data Model and Storage Formats and Indexing Review of Relational Design Theory Relational Algebra. Secondary Storage Management: The Memory Hierarchy, Flash Storage and Database Buffer, Architecture, Accelerating Access to Secondary Storage, Disks and Disk Failures, Arranging Data on Disk, Block and Record Addresses, Variable-Length Data and Records Column Stores, Record Modifications. Index Structures: Concepts, Types of Indexes, B+ Tree Indexes, Hash Tables, Multidimensional Data and Indexes and It's Applications, Multiple-key Indexes and R-trees, Bitmap Indexes, Indexes in SQL. 14 Hours Unit 2 - Query Processing and Optimization Query Execution: Physical-Query-Plan Operators, One-Pass Algorithms and Two-Pass Algorithms (based on Sorting and Hashing), Clustered and Non-clustered indexes, Buffer Management. The Query Compiler: Parsing and Preprocessing, Algebraic Laws for Improving Query Plans, From Parse Trees to Logical Query Plans, Cost based Plan Selection, Choosing an Order for Joins. 14 Hours Unit 3- Parallel and Distributed Databases Avenues for Parallelism: Parallel Database Architectures, Models of Parallelism, Intra-query and Inter-query Parallelism, Parallel Algorithms on Relations. Distributed Databases Distributed Data Storage, Types, Advantages, Data Fragmentation and Replication, Distributed Transactions, Distributed Query Processing, Distributed Commit Two-phase Commit, Distributed Locking, Distributed Catalog Management, Concurrency Control, Peer-to-Peer Distributed Search Chord Circles. 14 Hours Unit 4 - Data Stream Management & Trends in Database Systems Introducing Stream Processing, Stream Processing Model, Streaming Architectures, Apache Spark as a Stream Processing Engine, Spark's Distributed Processing Model, Spark's Resilience Model, Apache Kafka, and Amazon Kinesis. Introduction to multi-model databases and memory optimized tables. Current Trends in Database Systems and Decision Support Systems; Data Warehousing and Data Mining Applications; Introducing Data Lake and Data Fabric. 14 Hours						
Text Book(s):	1. "Database Systems: The Complete Book", H Garcia-Molina, JD Ullman and J Widom, 2nd Ed., Pearson, 2018. 2. "Stream Processing with Apache Spark", Gerard Maas & Francois Garillot, O'Reilly, June 2019.						

Reference Book(s):	1. "Fundamentals of Database Systems", Elmasri and Navathe, Pearson Education, 7th Ed., 2016. 2. "Streaming Systems" by Tyler Akidau, Slava Chernyak, Reuven Lax, O'Reilly, July 2018.
Course Outcome	• Design and Deploy Storage Solutions and Indexing mechanisms for optimized performance of databases. • Perform Query Optimization. • Apply Parallel and Distributed Database approaches to solve problems of large databases. • Manage "Data Streams" using latest tools.

Course Code	UE24CS343BB4	Course Title	Machine Learning on Graphs				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -4				
Tools/Languages	Pytorch, C, C++, Java	Desirable Knowledge	Data structures, Graph Theory and its Applications				
Prelude	This course introduces the students to the three meta landscapes of representation, optimization, and objective for the learning tasks on graph data. Graphs are ubiquitous in almost everything today –in social graphs, large-scale biological systems, road networks, polypharmacy, and user-item relations. Graphs, being non-Euclidean, require different machine-learning approaches. This course starts by exploring graph features for traditional machine learning on graphs and discussing a few noteworthy classical graph ML approaches. The second unit focuses on shallow embedding methods for graphs in general, and then the same for knowledge graphs. The third unit focuses on representation learning using graph neural networks and discusses key algorithms along with extensions like temporal graphs and multi-relational graphs. The fourth unit discusses the deep generative models on the graph and model-building techniques using graph neural networks. The theory input concludes with a discussion on the interdisciplinary applications of Graph Neural Networks, interpretability, and meta-learning perspectives of graph AI.						
Course Objectives:	• To understand the theoretical and practical aspects of graph feature-based traditional machine learning and implement the same • To understand the theoretical and practical aspects of and to implement shallow embedding approaches for graphs and multi-relational knowledge graphs. • To understand the theoretical and practical aspects of and to implement neural methods of representation learning on graphs. • To understand the theoretical and practical aspects GNN modeling techniques and deep graph generative models with interdisciplinary applications.						

Course Contents	Unit 1: Traditional machine learning on graphs Types of complex Graphs and computational tasks on Graphs. Graph Features for traditional machine learning– Node-level features. Graph kernel features- Graphlet & Motif, Weis Feiler-Lehman kernel. Measures for neighborhood overlap. Graph clustering and spectral methods –Graph Laplacians, Graph Cuts and clustering, Spectral clustering. Encoder-Decoder perspective – encoder, decoder, and optimization of the encoder-decoder model. Factorization based approaches. 14 Hours Unit 2: Graph Representation Learning Based on Shallow Embedding Methods Random Walk-based embedding –DeepWalk and Node2Vec. Limitations of shallow embedding. Shallow embedding in multi-relational Knowledge Graph- embedding as a reconstruction task, loss function, and decoders. Knowledge Graph Completion Task-TransE, TransR, DistMult, ComplEx. Semi-supervised Learning on Graph - Label Propagation Algorithm and Label Spreading Algorithm . 14 Hours Unit 3: Graph Learning Using Graph Neural Network Vanilla GNN - Neural message passing framework. Generalized neighborhood aggregation. Generalized update methods. Graph Convolution Networks (GCN): Spectral GCN, LGCN, and GraphSAGE), Multi-relational GCN and Graph Attention Networks (GAT), Graph Transformer, Graph pooling. Expressiveness, designing Maximally expressive GNNs, and Expressive Power by aggregate functions. Applications and loss functions. 14 Hours Unit 4: Deep Generative Models, GNN on complex graphs, and interdisciplinary applications Deep Generative models on graph – Variational autoencoder on the graph, Generative adversarial network on the graph, Auto-regressive methods. GNN on complex graphs – Heterogeneous GNN, Dynamic GNN, Hypergraph GNN. Application of GNN to Biology and Chemistry, Interdisciplinary Applications of GNN: Unstructured data – Text and Image. Structured Data - Social Network, Recommender System, Cyber Security. An introduction to trustworthy graph AI, Meta-Learning perspectives of graph ML 14 Hours
Text Book(s):	1. “Graph Representation Learning”, William L Hamilton, Morgan and Claypool Publishers, 2020. 2. “Deep Learning on graphs”, Yao Ma and Jiliang Tang, Cambridge University Press, 2021 3. “Introduction to Graph Neural networks”, Zhiyuan Liu and Jie Zhou, Synthesis Lectures on Artificial Intelligence and Machine learning, Morgan and Claypool Publishers, 2020.
Reference Book(s):	1. “Social Media Mining”, Reza Zafarani, Cambridge University Press, 2015 (Asian Economic edition available) 2: “ Network Science” by Albert Barabasi , 2016
Course Outcome	• feature engineering-based graph ML tasks. • downstream machine learning tasks on graphs using shallow embedding. • solve a downstream machine-learning task using a graph neural network. • graph modeling techniques and appreciate deep generative models on graphs.

Course Code	UE24CS343BB5	Course Title	Embedded system programming with micro python				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -4				
Tools/Languages	Microcontroller: Pyboard, ESP32, BBC micro:bit	Desirable Knowledge	Python for computational problem solving, Microprocessor and Computer Architecture				
Prelude	This course introduces learners to the world of embedded systems using MicroPython, a lightweight implementation of Python 3 designed to run on microcontrollers. Students will develop a strong foundation in embedded programming principles using accessible and readable Python code. The course guides students from basic setup and scripting to more advanced interfacing with sensors, hardware protocols, and simple Internet of Things (IoT) applications. Through practical labs, real-world examples, and mini-projects, learners will gain hands-on experience in building responsive and connected embedded systems.						
Course Objectives:	- Understand the MicroPython ecosystem and its advantages for embedded systems. - Set up and configure MicroPython on supported hardware platforms such as Pyboard, ESP32, or BBC micro:bit. - Write efficient MicroPython code to interact with digital and analog peripherals. - Utilize communication protocols (I2C, SPI, UART) for sensor and device integration. Develop basic IoT applications using networking features in MicroPython. Apply debugging and development tools to test, monitor, and deploy MicroPython programs.						
Course Contents	Unit 1: Getting Started with MicroPython and Microcontrollers Introduction to Embedded Systems and MicroPython, Overview of Microcontrollers (Pyboard, ESP8266, ESP32, micro:bit), Setting up the Development Environment: Flashing MicroPython firmware, Using REPL, WebREPL, and editors (Thonny, Mu), MicroPython Syntax Review: Basic Python features (control structures, data types, functions), Working with the Filesystem, Executing scripts on boot (boot.py and main.py) Lab Activities: - LED blink and GPIO toggling - Button input and LED output - Running Python scripts using REPL and script files 14 Hours Unit 2: Digital and Analog Interfacing with Hardware Digital Inputs/Outputs (GPIO), Pull-up/down resistors and debounce logic, Using PWM (Pulse Width Modulation), Analog Input and Output using ADC/DAC, MicroPython modules: machine, time, and utime, Creating reusable modules, Timer-based programming and event loops Lab Activities: - Dim an LED using PWM - Read a potentiometer using ADC - Build a tone generator with PWM and DAC - Debounce button-controlled LED 14 Hours Unit 3: Sensor Interfacing and Communication Protocols Understanding serial communication basics, I2C, SPI, and UART in MicroPython, Interfacing common sensors (temperature, light, accelerometer), Working with displays (OLED, LCD), Using external libraries and drivers, Managing asynchronous data streams Lab Activities: - Display sensor readings on an OLED via I2C						

	- Communicate between two devices using UART - Read accelerometer data using SPI - Develop a multi-sensor data display system 14 hours Unit 4: Networking and IoT with MicroPython Introduction to IoT with MicroPython, Wi-Fi networking and internet access, Creating HTTP servers and web APIs, Using MQTT protocol for data publishing, Sending sensor data to cloud platforms (ThingSpeak, Adafruit IO), Remote device control via mobile/web interface Lab Activities: - Build a Wi-Fi-connected temperature logger - Create a web server to control LEDs via browser - Use MQTT to publish and subscribe sensor data - Final Project: Complete IoT-enabled embedded system Recommended Hardware: - Microcontroller: Pyboard, ESP32, BBC micro:bit - Components: LEDs, resistors, buttons, temperature sensors (e.g., TMP36, DHT11), OLED display, potentiometer, buzzer, jumper wires, breadboard, USB cable 14 Hours
Text Book(s):	1. Programming with MicroPython: Embedded Programming with Microcontrollers and Python by Tollervey, Nicholas , Publisher: Shroff/o' reilly
Reference Book(s):	1. Python for Microcontrollers: Getting Started With MicroPython by Donald Norris, Publisher: Tab Books
Course Outcome	- Understand and apply the fundamentals of MicroPython programming for embedded systems, including setting up environments and writing code for basic I/O operations. - Interface microcontrollers with sensors, actuators, and displays using digital, analog, and communication protocols such as I2C, SPI, and UART. - Develop and deploy IoT-enabled embedded applications using networking features in MicroPython (Wi-Fi, HTTP, MQTT) for real-time monitoring and control. - Design and implement a complete embedded system project, demonstrating integration of hardware and software with proper documentation and presentation.

Course Code	UE24CS343BB6	Course Title	Information Security				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
Tools/Languages	SEED Labs VM, Scapy, Burp Suite, Metasploit, Nmap, etc.	Desirable Knowledge	Computer Networks, Operating System				
Prelude	This course will present security aspects from a secure software life cycle process – requirement, architecture, design, coding, and testing. Students will have opportunity to dwell well in to technical "how to" with hands-on sessions, assignments, and some case study discussions						
Course Objectives:	● To understand various cyber threats and attacks and secure software development process. ● To learn attack and defence mechanisms for buffer overflow, shellshock attack, etc. ● To understand the concept of threat modelling and its application. ● To learn about the most common web application security vulnerabilities. ● To understand and apply various penetration testing techniques and tools.						
Course Contents	Unit 1: Introduction and Privilege Escalation Attacks Security Concepts and Relationships. Use cases and Misuse cases, Misuse case legend, Security use case vs Misuse case, Secure Software Development Life Cycle (SDL). Basics of Linux for Security, Set-UID program: Need for privileged programs, Set-UID mechanism, Superman story, Attack surfaces, Invoking other surfaces, Principle of least privilege. Environment variables and attacks: Environment variables, Attack surface, Attacks via Dynamic linker, External program, and Library. Lab: Set-UID program & Environment variables and attacks. Buffer overflow attack-I : Program memory layout, Stack and function invocation, Stack buffer-overflow attack 14 Hours Unit 2: Secure Coding Buffer overflow attack-II and Shellcode:, Attacks with Unknown address and Buffer size, Shellcode, Countermeasures & Defeating it. Return-to-libc, Return-oriented programming. Format string vulnerability: Introduction to functions and format string, Vulnerable program, Exploiting the vulnerability, Code injection attack, Countermeasures. Race-condition vulnerability, Case study: Target case study. 14 Hours Unit 3: Threat modelling and Web Vulnerabilities SSRF, Broken Access Control, JWT attacks, OAuth / authentication flaws, API security. Threat Modelling: Trust Boundaries, Attack Surfaces, Modelling Methods : STRIDE model and variants. Case study: Apple - Privacy vs Safety. 14 Hours Unit 4: Malware Analysis and Penetration Testing Malware : its types, analysis: Static and Dynamic , MorrisWormAttack, Reverse engineering basics (IDA/Ghidra), Penetration testing: Introduction, Benefits, Drawbacks. 14 Hours						
Text Book(s):	“Computer & Internet Security: A Hands-on Approach”, Wenliang Du, 2nd Edition/3 rd Edition.						
Reference Book(s):	1: “Computer Security: Principles and Practice”, William Stallings and Lawrie Brown, Pearson Education, 3rd Edition, 2014. 2: “Secure Programming with Static Analysis”, Brian Chess and Jacob West, Pearson Education, 2007						

Course Outcome	At the end of this course, the student will be able to: ● Identify possible misuse cases in the context of software development. ● Defend against various attacks and how to write secure code. ● Apply threat modelling techniques to expose inherent vulnerabilities in applications. ● Design and develop secure web applications. ● Exploit software vulnerabilities and launch attacks.
-----------------------	---

Course Code	UE24CS343BB7	Course Title	Mobile and Autonomous Robot				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective Course				
Tools/Languages	ROS/ROS2, Python, C++	Desirable Knowledge	Introduction to Autonomous Systems with ROS Linear Algebra and its Applications				
Prelude	This course provides an introduction to autonomous robots, focusing on the principles and techniques behind their design, programming, and control. Students will explore key topics such as perception, planning, and navigation, while gaining hands-on experience through projects and simulation using ROS/ROS2. The course combines theoretical foundations with practical applications across both terrestrial and aerial robotic platforms.						
Course Objectives:	- Understand the evolution, current trends, and research directions in autonomous mobile robotics. - Learn the theoretical and practical fundamentals of designing and operating autonomous robots. - Explore core topics including perception, localization, planning, and control. - Gain hands-on experience with robot simulation and programming using ROS/ROS2. - Apply knowledge to develop and test autonomous systems across terrestrial and aerial platforms.						
Course Contents	Unit 1: Introduction to Robotics Robotics overview: Past, Present and Future; Sensors for Robots (Visual and inertial measurements: Gyroscope, accelerometers, IMU, GPS, Range sensors, and LiDAR.) AI in Robotics: Machine Learning Basics for Robotics. ROS overview: ROS/ROS2 for robotics, ROS architecture and communication protocols, ROS packages for robot hardware control. 14 Hours Unit 2: Locomotion and Perception Introduction to Locomotion: The basics of SOTA locomotion systems, Legged Robots, Wheeled Robots, Aerial Robots. Robot Kinematics, Introduction to Perception, camera vision, Fundamentals of Computer Vision and Image Processing and filtering, Feature Extraction: Object detection and Place Recognition, Stereo vision and 3D perception, Feature Extraction Based on Range Data (Laser, Ultrasonic). 14 Hours Unit 3: Localization Introduction and Challenges of Localization, To Localize or Not to Localize, Belief Representation, Map representation, Probabilistic Map-Based Localization: Markov localization, Kalman Filter localization, Autonomous Map Building: SLAM, EKF SLAM, Particle filter SLAM, GraphSLAM, Open challenges in SLAM. 14 Hours Unit 4: Planning and Navigation Introduction Path Planning and Navigation, Path planning: Graph search and Potential field path planning. Obstacle avoidance: Bug algorithm, Vector field histogram, bubble band technique, Curvature velocity techniques, Dynamic window approaches. Navigation Architectures. Revisiting Navigation using Reinforcement Learning and Imitation learning: A Robotics Perspective, Applications and Social Implications. 14 Hours						
Laboratory	1: ROS/ROS2 installation and setup 2: Using turtlesim, ros2, and rqt 3: Learn how to build a visual model of a robot that you can view in Rviz 4: Building a movable robot model 5: Working With URDF Files,						

	6: Using a Camera Sensor in Gazebo with ROS 2 7: Introducing tf2 8: turtlebot3 simulations
Text Book(s):	1. Introduction to Autonomous Mobile Robots (Intelligent Robotics and Autonomous Agents series) second edition by Roland Siegwart, Illah Reza Nourbakhsh, Davide Scaramuzza 2. Probabilistic Robotics By Sebastian Thrun, Wolfram Burgard and Dieter Fox. ISBN-13: 978-0262201629, ISBN-10: 0262201623. Intelligent Robotics and Autonomous Agents series; 1st Edition
Reference Book(s):	1. Introduction to Autonomous Robots: Nikolaus Correll, Magellan Scientific, 2016. 2. ROS Robot Programming, ROBOTIS Co., Ltd. From the basic concept to practical programming and robot application. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, and TaeHoon Lim. 3. Introduction to Robotics: Mechanics and Control 4th Edition, John Craig, ISBN-13: 978-0133489798, Pearson; 4th edition 4. Artificial Intelligence for Robotics Build Intelligent Robots that Perform Human Tasks Using AI Techniques By Francis X. Govers · 2018
Course Outcome	● Explain the principles and applications of autonomous mobile robots, including their key components and system architecture. ● Identify and integrate appropriate sensors and actuators for robot perception and control. ● Develop and simulate robotic systems using ROS/ROS2 for navigation, planning, and hardware interfacing. ● Apply algorithms for localization, path planning, and obstacle avoidance in real or simulated environments. ● Design and implement autonomous robotic solutions for terrestrial and aerial platforms.

Course Code	UE24CS343BB8	Course Title	Cybersecurity in Cloud				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
Tools/Languages	AWS-vulnerable platform	Desirable Knowledge	Computer Networks, Operating System				
Prelude	This course focuses on the essentials of Cloud security and offers a practical exposure to securely create or access the cloud services. The course emphasizes on securing data and applications in a public cloud environment.						
Course Objectives:	- Understand the major security challenges and risks associated with shared cloud computing environments. - Analyze cloud deployment practices and evaluate the security implications of misconfigurations in cloud services. - Apply appropriate data protection mechanisms and security controls to safeguard cloud-based networks and resources. - Examine cloud application security, governance models, and industry standards relevant to secure cloud operations						
Course Contents	Unit 1 – Cloud Security Essentials and IAM in Public Cloud Infrastructure Introduction to Cloud Computing, IaaS, PaaS and SaaS and Service models, Private/Public/ Hybrid and Multi Cloud models, Introduction to Cloud Security, NIST Cyber security Framework, Shared Responsibility Model-cloud Provider and Cloud Customer, Cloud Security Fundamentals-Types of Misconfigurations. IAM Life cycle, Building and Managing IAM in AWS, Access Policy and Authentication 14 Hours Unit 2 – Network and Data Security in Public Cloud Infrastructure Zero trust architecture in Single Cloud/Hybrid/Multi Cloud infrastructure. AWS Network Security: Amazon VPC security groups, AWS shield, AWS Firewall Manager, AWS Network firewall, Network ACLs, DNS firewall, Data Security- Securing data at rest, Data Security- Securing data at transit, Data in process. 14 Hours Unit-3: Securing Cloud Applications in Public Cloud Infrastructure Container services, what is Microservice Architecture? Challenges of Microservice Architecture, Service Mesh Architecture, Serverless Architecture/Framework, FaaS, CI/CD Pipeline, DevSecOps. Secure IaC 14 Hours Unit 4: Cloud Security Governance Cloud Controls Matrix (CCM) – cybersecurity control framework for cloud computing, CIS (Center for Internet Security) Critical Security Controls, Cloud logging and Auditing, Overview of CSPM, CWPP Solutions, Security guidance for critical areas of focus in cloud computing by CSA (Cloud Security alliance), Case Study: Exploration of AWS Cloud Security Dashboard 14 Hours						
Text Book(s):	1. Mastering AWS Security by Albert Antony 2. Practical Cloud Security by Chris Dotson Copyright © 2019 Chris Dotson. All rights reserved. Printed in the United States of America. Published by O'Reilly Media						

Reference Book(s):	1. Hands-On AWS Penetration Testing with Kali Linux (Karl Gilbert Benjamin Caudill PACKT publishers) 2. CSA Guide to Cloud Computing Implementing Cloud Privacy and Security Raj Samani Brian Honan Jim Reavis 3. https://cisecurity.wistia.com/medias/tcelnntxig 4. https://aws.amazon.com/security/
Course Outcome	• Explain the security risks and challenges inherent in cloud computing environments. • Analyze cloud configurations to identify vulnerabilities and recommend secure setup practices. • Apply appropriate security controls to protect cloud data, applications, and network infrastructure. • Evaluate cloud governance policies and industry standards to ensure compliance and secure cloud operations.

Course Code	UE24CS343BB9	Course Title	Quantum Computer System Design				
Program	B.Tech	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective				
Tools/Languages	IBM Qiskit	Desirable Knowledge	Linear Algebra				
Prelude	This course offers a rigorous exploration of quantum computing from first principles to fault-tolerant architectures. Students will journey through the mathematical foundations of quantum mechanics, master quantum algorithm design, and engage hands-on with real quantum hardware using modern SDKs. Bridging theory and practice, the curriculum addresses the full quantum software stack — from high-level programming models down to physical gate synthesis. With focused attention on the NISQ era's challenges, including noise, decoherence, and error correction, this course equips students to navigate the rapidly evolving landscape of quantum computing and contribute meaningfully to its transformative applications.						
Course Objectives:	- Understand the postulates of quantum mechanics and apply them to model quantum states, unitary evolution, and measurement in quantum computing contexts. - Analyze and implement foundational quantum algorithms — including QFT, QPE — evaluating their computational advantages over classical counterparts. - Design and compile quantum programs using modern SDKs such as IBM Qiskit, targeting real quantum backends through hardware-aware optimization techniques. - Examine the full quantum software stack, including qubit mapping, routing, ISA design, and classical-quantum microarchitectural control. - Evaluate noise models, apply error mitigation strategies, and understand the principles of quantum error correcting codes and fault-tolerant computation.						
Course Contents	Unit 1: Quantum Programming Postulates of Quantum Mechanics: State representation, unitary time evolution, and projective measurements. Quantum Information Theory: No-cloning theorem, dense coding, quantum teleportation, and quantum parallelism. Quantum Programming Models: Working with Quantum Instruction Set Architectures (Q-ISA) and executing programs on real quantum backends using modern SDKs like IBM Qiskit. 14 Hours Unit 2: Quantum Algorithms and Quantum Architecture Arithmetic and Signal Processing: Quantum Fourier Transform (QFT), Quantum Phase Estimation (QPE). Quantum Complexity Theory: Bounded-Error Quantum Polynomial-time (BQP) and connections to classical complexity classes. The Quantum Compiler: Compiling higher-level quantum code to physical gates; gate synthesis and approximation (e.g., Solovay-Kitaev algorithm). 14 Hours Unit 3: Quantum Architecture and the Software Stack Qubit Mapping and Routing: Solving hardware connectivity constraints, SWAP gate insertion, and layout-aware scheduling. Classical-Quantum Control: Microarchitecture for classical and quantum control; managing coherence						

	limits. Instruction Set Architectures (ISA): Designing interfaces that map logical operations to hardware-native physical gates. 14 Hours Unit 4: Noisy Intermediate-Scale Quantum (NISQ) & Error Correction Quantum Noise: Decoherence, gate errors, readout errors, and T1 / T2 relaxation times. Error Mitigation and Characterization: Randomized benchmarking, cross-entropy benchmarking, and dynamical decoupling. Quantum Error Correcting Codes (QECC): Classical linear codes, stabilizer codes, surface codes, and concatenated codes. Fault-Tolerant Computation: Threshold theorem and fault-tolerant gate operations. 14 Hours
Text Book(s):	1. "Quantum Computer Systems: Research for Noisy Intermediate-Scale Quantum Computers" by Yongshan Ding & Frederic T. Chong (University of Chicago), 2020 2. "Quantum Computation and Quantum Information" by Nielsen, M. A., & Chuang, I. L. — (10th Anniversary Edition), Cambridge University Press.
Reference Book(s):	3. Preskill, J. — <i>Lecture Notes on Quantum Computation</i>, California Institute of Technology (freely available at theory.caltech.edu). 4. Mermin, N. D. — <i>Quantum Computer Science: An Introduction</i>, Cambridge University Press. 5. Krantz, P. et al. — "A Quantum Engineer's Guide to Superconducting Qubits," Applied Physics Reviews, 2019 (for hardware-layer and NISQ context).
Course Outcome	• Formulate quantum programs using state vector representations and execute them on real or simulated quantum hardware with appropriate SDK tooling. • Derive and explain the workings of key quantum algorithms and position them within quantum complexity theory (BQP and classical class relationships). • Apply compiler techniques such as gate synthesis, the Solovay-Kitaev algorithm, and layout-aware scheduling to optimize quantum circuits for physical hardware. • Characterize quantum noise sources including T1/T2 relaxation, gate errors, and readout errors, and apply mitigation techniques such as randomized benchmarking and dynamical decoupling.

Course Code	UE24CS343BB10	Course Title	Video Processing and understanding				
Program	B.Tech CSE	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	0	4	4
Semester	6	Type of Course	Elective -4				
Tools/Languages	Python with libraries like OpenCV, PyTorch, TensorFlow FFmpeg, OpenCV, Hugging Face Transformers (DeepSort with YOLO integration)	Desirable Knowledge	Image Processing and Machine Learning.				
Prelude	The rise of video as a dominant medium in digital communication and content creation has made the ability to analyze and understand video data a vital skill in the age of AI. This course provides a comprehensive introduction to the theory and application of video processing and video understanding, spanning low-level video enhancement and segmentation techniques to high-level video analytics such as semantic scene understanding, and vision-language integration. Students will explore cutting-edge techniques in action recognition and vision-language models, with applications in video captioning, anomaly detection, and visual question answering, preparing them for real-world challenges in video analytics and artificial intelligence.						
Course Objectives:	- To introduce the foundational concepts and techniques used in digital video processing including representation formats, compression techniques, resolution and frame rate. - To explore motion analysis methods such as optical flow, motion tracking, and temporal feature extraction from video streams. - To equip students with techniques for video understanding including object recognition, scene parsing, and semantic analysis. - To provide exposure to deep learning-based approaches and modern AI and for video analytics, including action recognition, anomaly detection and vision-language models.						
Course Contents	Unit 1: Fundamentals of Video Processing Introduction to digital video: Spatial resolution and frame rate, color spaces (RGB, YUV), video enhancement techniques (denoising, sharpening), video sampling and interpolation, video formats and compression techniques (H.264 (MPEG-4 AVC)), frame extraction and representation (I-frames, P-frames, B-frames), separating audio and video from an input video. 14 Hours Unit 2: Video segmentation, motion estimation, and tracking Video segmentation: Scene cut detection, spatio-temporal segmentation, Motion Detection and estimation (frame differencing, background subtraction, optical flow methods: Lucas-Kanade, Horn-Schunck), Spatial Feature extraction: 2D CNN, Skeletal keypoint detection, Temporal feature extraction: 3D CNN, temporal pooling, Object tracking: Kalman filter, DeepSORT. 14 Hours Unit 3: Video Understanding-1 Key frame extraction and scene segmentation, video-based object recognition: YOLOV++, semantic segmentation in video and its challenges: SAM2, pose estimation (for blurring of faces for privacy, analyzing human movement, orienting a robot for navigation, etc.), extraction and processing text content in video (for indexing and retrieval), audio and video semantic analysis, scene understanding. 14 Hours Unit 4: Video Understanding-2 Action Recognition: C3D, R2+1 D, MViTv2, Anomaly recognition: Real-time anomaly detection in dynamic environments, GNN for activity recognition, vision language models: video captioning, visual question answering and reasoning, visual dialog: video CLIP, Llava. 14 Hours						

Text Book(s):	1. M. Tekalp, Digital Video Processing. Signal Processing Series, Prentice Hall, 2015 (2nd edition). 2. Szeliski, Richard. Computer vision: algorithms and applications. Springer Nature, 2022.
Reference Book(s):	1. Awais, Muhammad, et al. "Foundation Models Defining a New Era in Vision: a Survey and Outlook." IEEE Transactions on Pattern Analysis and Machine Intelligence (2025). 2. Jerry D. Gibson and Al Bovik. "Handbook of Image and Video Processing". Academic Press, Inc., USA (2005). 1. "Computer Vision A Modern Approach", D. A Forsyth and J. Ponce, Pearson Education, 2015 2. A. Bovik (Ed.), The Essential Guide to Video Processing. Academic Press, 2009.
Course Outcome	• Apply video preprocessing and enhancement techniques to improve video quality and extract frames based on compression methods, • Assess the suitable solution and implement motion estimation and tracking algorithms, including Kalman filter and DeepSORT, for object tracking, • Implement methods to detect and segment objects in video, followed by scene understanding for effective video analysis and • Recognize actions and detect anomalies in video using advanced deep learning models, and apply vision-language models for tasks like video captioning and visual question answering.

Course Code	UE24CS342BB12	Course Title	SOC & Threat Intelligence				
Category	Elective-3 Group	Hours per week/ Credit Assigned	L	T	P	S	C
			4	0	2	5	5
Semester	6	Type of Course	Elective				
Tools/Languages	Wazuh, Shuffle/ Python	Desirable Knowledge	basic OS + basic networking + fundamental cybersecurity concepts + analytical thinking				
Course Objectives:	- Understand SOC structure, network fundamentals, and security tools. - Master SIEM for monitoring and basic alert triage. - Apply MITRE ATT&CK for threat identification. - Execute primary Incident Response (IR) and containment steps.						
Course Contents	Module 1: Fundamentals of Cyber Defense and SOC Operations Introduction to Cybersecurity: CIA Triad, Threat Actors, Vulnerability vs. Risk.Network & OS Foundation: TCP/IP, Common Protocols (HTTP, DNS, DHCP).Windows Event Logs & Linux Syslogs (basic log types).SOC Structure & Tools: Tier 1, 2, 3 Roles, Daily Workflow, Key Metrics (MTTD, MTTR).Introduction to SOC Tools: SIEM, Ticketing, EDR (Conceptual). Module 2: Log Management and Security Monitoring (SIEM Essentials) Log Sources & Collection: Log types (Firewall, Endpoint, OS), Log Normalization Security Information and Event Management (SIEM): SIEM Architecture (Collectors, Database, Correlator - simplified view).Basic Search Query Language and syntax.Alert Triage and Handling: Alert lifecycle, False Positives/True Positives.Basic Correlation Rules (Conceptual design). Module 3: Threat Detection and Threat Intelligence Cyber Threat Intelligence (CTI) Basics: CTI Sources and Types (Tactical, Operational), Indicators of Compromise (IoCs) and Indicators of Attack (IoAs), Adversarial Tactics & Models: MITRE ATT&CK Framework (Introduction).Cyber Kill Chain (Basic Phases) ,Introduction to Threat Hunting: Difference between Alerts and Hunting.Simple Hunt Hypotheses (e.g., "Is PowerShell being used unusually?"). Module 4: Incident Response and Containment Incident Response (IR) Lifecycle: Preparation, Identification, Containment, Eradication, Recovery Lessons , Incident Prioritization and Classification (Severity, Impact), Chain of Custody, Data Acquisition (Disk & Memory - simplified),Security Automation (SOAR Basics): Introduction to SOAR , Need for Automation and Response Playbooks.						
Laboratory	Lab1: Basic Command-Line Forensics: Use Linux/Windows commands (e.g., netstat, grep, dir) to find suspicious files/connections. Lab2: Network Traffic Analysis (Wireshark): Capture and analyze common protocol traffic (DNS, HTTP) to identify anomalies. Lab3: Introduction to a Ticketing System: Create, assign, and document the resolution of a simple security alert ticket (e.g., Phishing Report). Lab4: Log Ingestion & Normalization: Configure a simple log forwarder (e.g., Syslog-NG/Winlogbeat) to send logs to a centralized server. Lab5: SIEM Basic Searching (Splunk/ELK): Run fundamental search queries using fields, time ranges, and boolean logic (e.g., `source=firewall`)						

	Lab6:Creating Dashboards & Reports: Build simple graphical reports for executive consumption (e.g., Top 10 blocked IPs, Users with most login failures). Lab7:Alert Triage & False Positive Handling: Investigate 5 pre-generated alerts (e.g., high-volume traffic alert, impossible travel alert) and determine if they are true/false positives. Lab8:Using Threat Feeds: Research a known malware IoC (IP address, hash) using public tools like VirusTotal and an open-source threat feed. Lab9:Mapping to MITRE ATT&CK: Given a security incident description, map the attacker's actions to specific techniques in the MITRE ATT&CK framework. Lab10:Network Threat Hunting Scenario: Use SIEM data to search for a pre-defined IoC (e.g., known C2 server IP) and identify all compromised hosts. Lab11:EDR Triage Simulation: Review Endpoint Detection and Response (EDR) alerts (e.g., "suspicious process execution") and follow a playbook to contain the system. Lab12:Containment Exercise: Practice isolating a compromised host from the production network using a simulated firewall or network access control tool. Lab13:Digital Evidence Acquisition: Use a forensic tool (e.g., FTK Imager Lite, KAPE) to acquire an image of a simulated infected disk drive (focus on practical steps, not deep analysis). Lab14:Malware Triage: Analyze a benign suspicious file (e.g., an email attachment) in an online sandbox to identify its basic behavior and extracted IoCs. Lab15:Playbook Execution and Documentation: Follow a step-by-step SOAR playbook (printed or simple flow chart) for a "Credential Theft" incident, focusing on the decision points and documentation required.
Text Book(s):	1. Security operations center (SOC) https://unidel.edu.ng/focelibrary/books/Open-Source%20Security%20Operations%20Center%20(SOC)%20(%20etc.)%20(Z-Library).pdf 2. Framework for Improving Critical Infrastructure Cybersecurity https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.04162018.pdf
Reference Book(s):	1. Computer Security:Incident Handling Guide https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf 2. The Threat Intelligence Handbook https://soclibrary.futa.edu.ng/books/The%20Threat%20Intelligence%20Handbook%20by%20Za%20ne%20Pokornoy%20(z-lib.org).pdf